Faculty of Engineering

Department of Informatics Engineering

Software Engineering and Information Systems

Integrated Social Media Platform (NOONIFY)

A Senior 1 project report - submitted to complete the requirements for obtaining a bachelor's degree in informatics engineering – Software Engineering and Information Systems

Prepared by

Sara Eyad Attah

Aya Al-salamat

Supervised by

Dr. Eng. Akram Massouh

May 2026

I Certify that the preparation of this project entitled

[Integrated Social Media Platform (NOONIFY)]

Prepared by

[Sara Eyad Atta - Aya Alsalamat]

was made under my supervision at Faculty of Informatics Engineering in partial Fulfillment of the Requirements for the Degree of Bachelor of Software Engineering and Information System

Name: .....

Signature: .....

Date: .....

Abstract

This project aims to design an integrated social media platform that allows users to create their accounts, share text and visual content, and interact with others in a safe and seamless way, the system relies on the MERN Stack (MongoDB, Express.js, React.js, Node.js) to cover the front and back sides, in addition to supporting tools such as Clerk for account management and user documentation, **Image Kit** for image processing and optimization, and **Ingest** for managing scheduled back-end tasks.

The platform offers a wide range of functions including Sign Up/Sign In, Profile Management, Posts, Stories, Social Interaction via Likes, Comments, and Unfollow, as well as a real-time chat system.

The platform also enables Search & Discover, real-time notifications, and media management using efficient cloud storage solutions.

Technically, the system is highly scalable thanks to the Node.js and MongoDB architecture, with an interactive and user-Friendly user interface built with React.js.

Overall, the project offers a practical and integrated model of a modern social media app, combining a pleasant user experience with powerful performance powered by the latest Full Stack Development technologies.

Table of Contents

1.Chapter One ……..8

1.1-Project Importance……..9

1.2-Project Objectives……...9

1.3-Related Studies for social media platforms…...……..10

1.3.1-first study……..10

1.3.2-second study…...……11

1.3.3-Comparison tables between related studies…….....12

1.4-Grant char……13

1.5-Project development methodology……..13

1.6-Applying SCRUM methodology in the social media project……..……14

1.7-System Architecture……15

1.7.1-Client_Server Architecture……15

1.7.2-MVC Architecture…….16

1.7.3-Intergration between Client_Server and MVC in the project…….17

1.8-Tools used……..20

1.8.1-Front_End Tools……20

1.8.2-Back_End Tools……21

1.8.3-DataBase Tools……..…22

2-Chapter Two……24

2.1-General Requirments……..……25

2.2-Non Functional Requirments……..27

2.3-Requirments to be implemented in the project……..…27

2.4-Use Case Diagram ……28

2.5-Use Case Specification……29

2.6-Activity Diagram……..29

2.7-Sequence Diagram……29

3.Chapter three…….....73

3.1-Site_Map ……..74
3.2-Data Base ERD……..75
3.3-Class Diagram……76
3.4-Test Case ……...……77
3.5-RTM ……..……84
4.Chapter four……...87
4.1-User Interfaces……88
4.2-Conclusion……...95
4.3-Future Work……95

Lists of Table

2.1-Table Comparison between Related Studies……12
2.2-Table Start New Conservation……..29
2.3- Table Send and Receive Text Message and Emojis……32
2.4-Table Support Real-Time Message……..35
2.5-Table Display Message Status (Sent / Delivered / Read)……38
2.6-Table Delete or Mute Coversation……41
2.7-Table Block or Unblock User……44
2.8-Table Online Status (Online/Offline)……47
2.9-Table Search for Posts using Hashtags……49
2.10-Table View Explore Page……51
2.11-Table Send Real-Time Notification……53
2.12-Table View All Notification……55
2.13-Table Delete Notification or Mark as Read……57
2.14-Table Upload and Optimize Images……59
2.15-Table Protect Media……61
2.16-Table Log errors and System Activities……...63
2.17-Table Execute Scheduled Tasks……...65
2.18-Table Report Post……..67

2.19-Table Suggest Friends……69
2. 20-Table Discover |Trending……..71

Lists of Figures

1.1-Grant char……..…13
1.2-Scrum Methodology…….
1.3-Cient-Server…….
1.4-MVC……..……...
1.5-Integration between two architecture……..…….
2.6-Activity Start New Conservation……..30
2.7-Sequence Start New Conservation……..……..31
2.8- Activity Send and Receive Text Message and Emojis……33
2.9- Sequence Send and Receive Text Message and Emojis……..……34
2.10-Activity Support Real-Time Message……..36
2.11- Sequence Support Real-Time Message……..37
2.12-Activity Display Message Status (Sent / Delivered / Read)…… …..…….39
2.13- Sequence Display Message Status (Sent / Delivered / Read)…….39
2.14-Activity Delete or Mute Coversation……...……42
2.15- Sequence Delete or Mute Coversation……...……42
2.16-Activity Block or Unblock User…….……45
2.17- Sequence Block or Unblock User……..……45
2.18-Activity Online Status (Online/Offline)…..…… ……47
2.19- Sequence Online Status (Online/Offline)……...……47
2.20-Activity Search for Posts using Hashtags……50
2.21- Sequence Search for Posts using Hashtags…..……50
2.22- Activity View Explore Page……...……52
2.23- Sequence View Explore Page……52
2.24- Activity Send Real-Time Notification……..…….54

2.25- Sequence Send Real-Time Notification…….54
2.26- Activity View All Notification……..……56
2.27- Sequence View All Notification……56
2.28- Activity Delete Notification or Mark as Read……..…….58
2.29- Sequence Delete Notification or Mark as Read…….58
2.30- Activity Upload and Optimize Images……..…….60
2.31- Sequence Upload and Optimize Images…….60
2.32- Activity Protect Media…..…….62
2.33- Sequence Protect Media……62
2.34- Activity Log errors and System Activities……..……...64
2.35- Sequence Log errors and System Activities……...64
2.36- Activity Execute Scheduled Tasks……..……...66
2.37- Sequence Execute Scheduled Tasks……...66
2.38- Activity Report Post……..…..……..68
2.39- Sequence Report Post……..……..68
2.40- Activity Suggest Friends…….…..……70
2.41- Sequence Suggest Friends…….……70
2.42- Activity Discover Trending……..72
2.43- Sequence Discover Trending…..……..72
3.44-SiteMAP…….74
3.45-ERD……75
3.46-Class Diagram…….77

1. Chapter One: Introduction

1.1- Project Importance

The importance of this project stems from the growing need for modern and secure social media platforms that enable users to interact and share content easily and efficiently. With the rapid advancement of web technologies and the increasing reliance on digital applications for daily communication, it has become essential to develop an integrated social networking system that combines high performance with an outstanding user experience.

This project contributes to enhancing developers' skills in the field of Full Stack Development through a practical implementation based on MERN Stack technologies (MongoDB, Express.js, React.js, Node.js). These technologies enable the development of a comprehensive system that covers all aspects: Front-End for designing an interactive user interface, Back-End for handling requests and managing data, and Database for storing information in an efficient and secure manner.

The importance of the project also lies in its integration of modern concepts and technologies such as Real-time Communication, media management using ImageKit, secure user authentication through Clerk, as well as background task management using Inngest.

Therefore, the project is not limited to being merely an educational experience; rather, it represents a comprehensive model that can be further developed into a real-world social media platform characterized by security, reliability, ease of use, and alignment with the latest technological advancements in the web domain.

1.2- Project Objectives

This project aims to develop a comprehensive social media platform that allows users to create accounts, manage their profiles, and share both textual and media content in a secure and seamless manner. It also seeks to provide an interactive environment that combines attractive design, high performance, and ease of use across various devices and platforms.

The primary objective is to build a web application based on the MERN Stack (MongoDB, Express.js, React.js, Node.js) to cover all system layers, from the Front-End interface to the Back-End system and the Database. The project also aims to enhance its technical aspects by integrating modern tools and services such as Clerk for user management and authentication, ImageKit for image processing and optimization, and Inngest for executing scheduled background tasks such as notifications and temporary data deletion.

Another important objective is the implementation of Real-time Features such as instant messaging (Real-time Chat) and real-time notifications (Real-time Notifications), in order to provide a dynamic and interactive user experience.

In addition, the project aims to build a Responsive User Interface (Responsive UI) using React.js and CSS, ensuring smooth navigation and user interaction.

In summary, the project aims to deliver a comprehensive practical model that bridges the gap between academic and practical aspects, enabling developers to apply their skills in building a professional Social Media Platform that highlights the core concepts of modern web technologies and best practices in full-stack development.

1.3- Related Studies for Social Media Platforms

1.3.1 - First Study

Social Network Data Architecture and Storage

“TAO: Facebook's Distributed Data Store for the Social Graph”

This study discusses the TAO system developed by Facebook to efficiently manage relationships between users (Social Graph). The system aims to improve query speed and data access through a specialized, read-optimized data store, along with a unified API interface that simplifies querying without requiring an understanding of the underlying data structure. TAO is considered a model for designing databases tailored to large-scale social applications, as it combines MySQL with a caching layer to reduce response time and enhance performance.

Key Takeaways:

- Designing a specialized data layer tailored to the nature of social applications.
- Using caching and an API layer to reduce the load on databases.
- Geographical distribution of data improves access speed for users worldwide.

Source: Nathan Bronson et al., USENIX ATC, 2013 – Facebook Research.

1.3.2- Second Study

System Architecture and Scaling of Communication Platforms

"Scaling Big Data Mining Infrastructure: The Twitter Experience"

This study presents Twitter's experience in building an infrastructure capable of handling billions of messages daily while maintaining real-time performance. Twitter utilized technologies such as Kafka, HBase, and distributed storage systems to process and analyze continuous data streams efficiently.

The study emphasizes the importance of building a scalable system that allows component updates without service interruption, and achieving a balance between latency and throughput to provide a stable and interactive user experience.

Key Takeaways:

- Adopting streaming technologies and message queues such as Kafka is essential for real-time features.
- Building a flexible, scalable, and maintainable architecture without downtime.
- The importance of monitoring and logging tools in performance analysis.

Source: Twitter Engineering Reports, *Scaling Big Data Systems*.

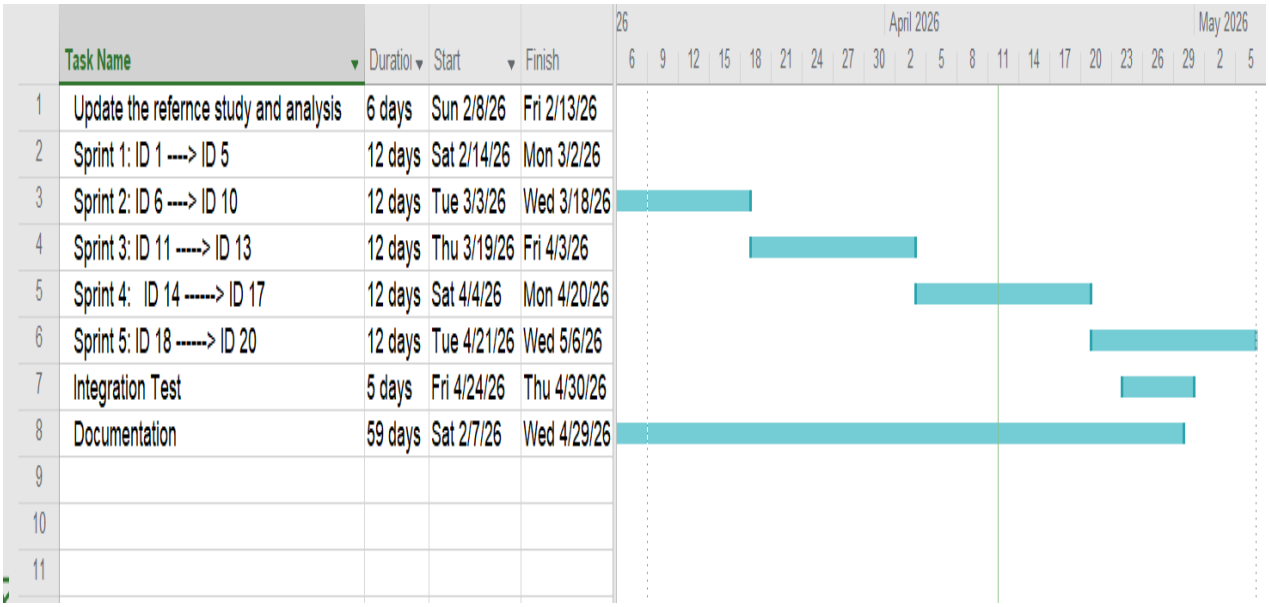
1.3.3- Comparison Table Between Related Studies

Table 1.1: Comparison Between Related Studies

Features	Twitter	Instagram	Facebook	Our System(Nonnify)
User identity verification	✓	✓	✓	✓
Publish and share content	✓	✓	✓	✓
Multimedia support	✓	✓	✓	✓
Immediate interaction(comment/share /like)	✓	✓	✓	✓
Reporting illegal publications	✓	✓	✓	✓
Support for real-time correspondence	✓	✓	✓	✓
Access protection and session mangement	✓	✓	✓	✓

Search through the site or description	✗	✗	✗	✓

1.4- Gantt Chart



	Task Name	Duration	Start	Finish
1	Update the refernce study and analysis	6 days	Sun 2/8/26	Fri 2/13/26
2	Sprint 1: ID 1 ----> ID 5	12 days	Sat 2/14/26	Mon 3/2/26
3	Sprint 2: ID 6 ----> ID 10	12 days	Tue 3/3/26	Wed 3/18/26
4	Sprint 3: ID 11 -----> ID 13	12 days	Thu 3/19/26	Fri 4/3/26
5	Sprint 4: ID 14 ------> ID 17	12 days	Sat 4/4/26	Mon 4/20/26
6	Sprint 5: ID 18 ------> ID 20	12 days	Tue 4/21/26	Wed 5/6/26
7	Integration Test	5 days	Fri 4/24/26	Thu 4/30/26
8	Documentation	59 days	Sat 2/7/26	Wed 4/29/26
9				
10				
11				

Figure 1.1: Gantt Chart

1.5- Project Development Methodology (SCRUM Methodology)

The SCRUM methodology was adopted as an Agile Framework for implementing this project due to its high flexibility and its ability to organize the development process in an interactive and incremental manner (Iterative and Incremental Process). SCRUM is considered one of the most widely used Agile Software Development methodologies in modern software projects, as it focuses on dividing the work into short phases known as Sprints. During each Sprint, a set of high-priority tasks is implemented and continuously reviewed to improve the quality of the final product.

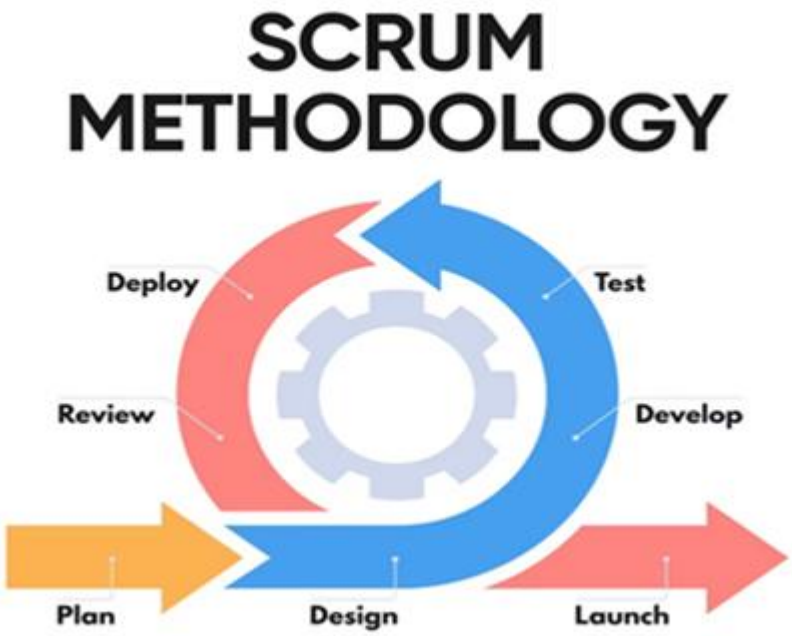


Figure 1.2: Scram Methodology Diagram

1.6- Applying SCRUM Methodology in the Social Media Platform Project

The development process was divided into multiple Sprints, where each Sprint focuses on a specific set of core functionalities within the platform.

The Sprints are as follows:

- **Sprint 1:** Setting up the development environment (Setup Environment) and implementing the Authentication System (Sign Up / Sign In).
- **Sprint 2:** Developing the Front-End Interface using React.js and integrating it with Back-End APIs.
- **Sprint 3:** Implementing search features and the Friends system (Search & Friends System).
- **Sprint 4:** Developing the posts and comments module (Posts & Comments Module).
- **Sprint 5:** Adding the stories feature (Stories System) and notifications (Notifications).

After each Sprint, a **Sprint Review Meeting** is conducted to present progress and evaluate completed tasks, followed by a **Sprint Planning Meeting** to define the tasks for the next iteration.

1.7- System Architecture

1.7.1- First: Client–Server Architecture

The Client–Server architecture was adopted as the foundation for building the platform. It is one of the most widely used architectures in modern web applications, especially those based on communication between the user interface (Client) and the server (Server) via HTTP/HTTPS protocols.

In this project:

- The **Client** represents the user interface built using React.js.
- The **Server** represents the back-end developed using Node.js and Express.js, connected to the MongoDB database.

Data is exchanged between the client and server in JSON format through RESTful APIs.

Workflow

1. The user (**Client**) sends a request via the browser, such as logging in or viewing posts.
2. The **Server** receives the request through Express.js controllers and processes it based on the application logic.
3. The server communicates with the MongoDB database to retrieve or modify the required data.
4. The server sends a response back to the client, and the result is displayed through the interface using React.js.

Advantages of Client–Server Architecture

- Clear separation between the Front-end and back-end (**Separation of Concerns**), which simplifies development and maintenance.
- Enhanced security, as core data processing occurs on the server rather than the client side.
- High scalability (**Scalability**), allowing independent scaling of servers or updates to the user interface.
- Reusability of back-end services through APIs that can be used in other applications (e.g., mobile apps).

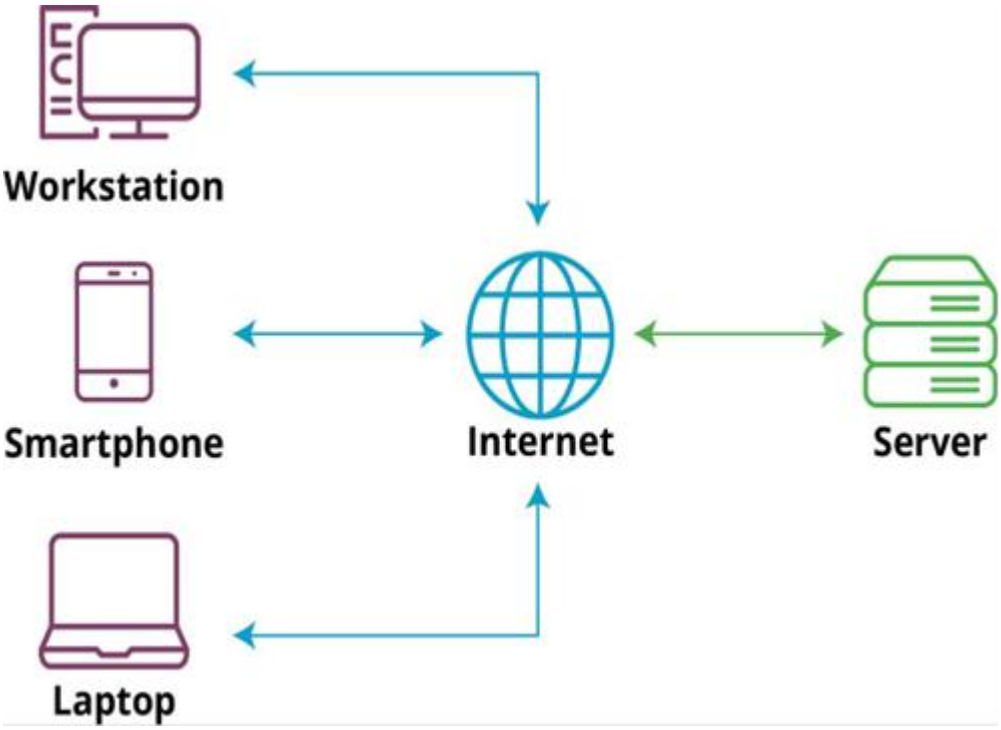


Figure 1.3: Client-Server

1.7.2- Second: MVC Architecture (Model – View – Controller)

In addition to the Client–Server architecture, the MVC pattern is implemented on the server side to organize the codebase and improve maintainability and scalability.

The MVC architecture divides the system into three main layers:

1. **Model:** Represents the data and its associated structures, including all operations related to data storage and retrieval from the database.
2. **View:** Represents the user interfaces displayed on the Front-end using React.js.
3. **Controller:** Manages the interaction between the Model and the View. It handles user requests, executes the required business logic, and returns the results to the interface.

MVC Implementation in This Project

Controllers Used:

- **MessageControl:** Responsible for managing messages and conversations.
- **PostControl:** Responsible for creating, updating, and deleting posts.
- **StoryControl:** Responsible for uploading and managing stories.
- **UserControl:** Responsible for user registration and profile updates.

Models Used:

- **Email, Message, Post, Story, User, Connection**

These represent the collections in the MongoDB database and define the attributes and relationships between them.

View:

The user interfaces built using React.js and CSS, which receive data from the server through RESTful APIs and display it to users in an interactive and user-friendly manner.

Advantages of MVC Architecture

1. **Code Organization:** Dividing the code into separate components makes it easier to read and maintain.

2. **Facilitates Team Development:** front-end and back-end teams can work in parallel without conflicts.
3. **Reusability:** Models can be reused in other services or future projects.
4. **Flexibility:** Any part of the system can be updated without affecting other components.
5. **Testability:** Each layer can be tested independently (Unit Testing).

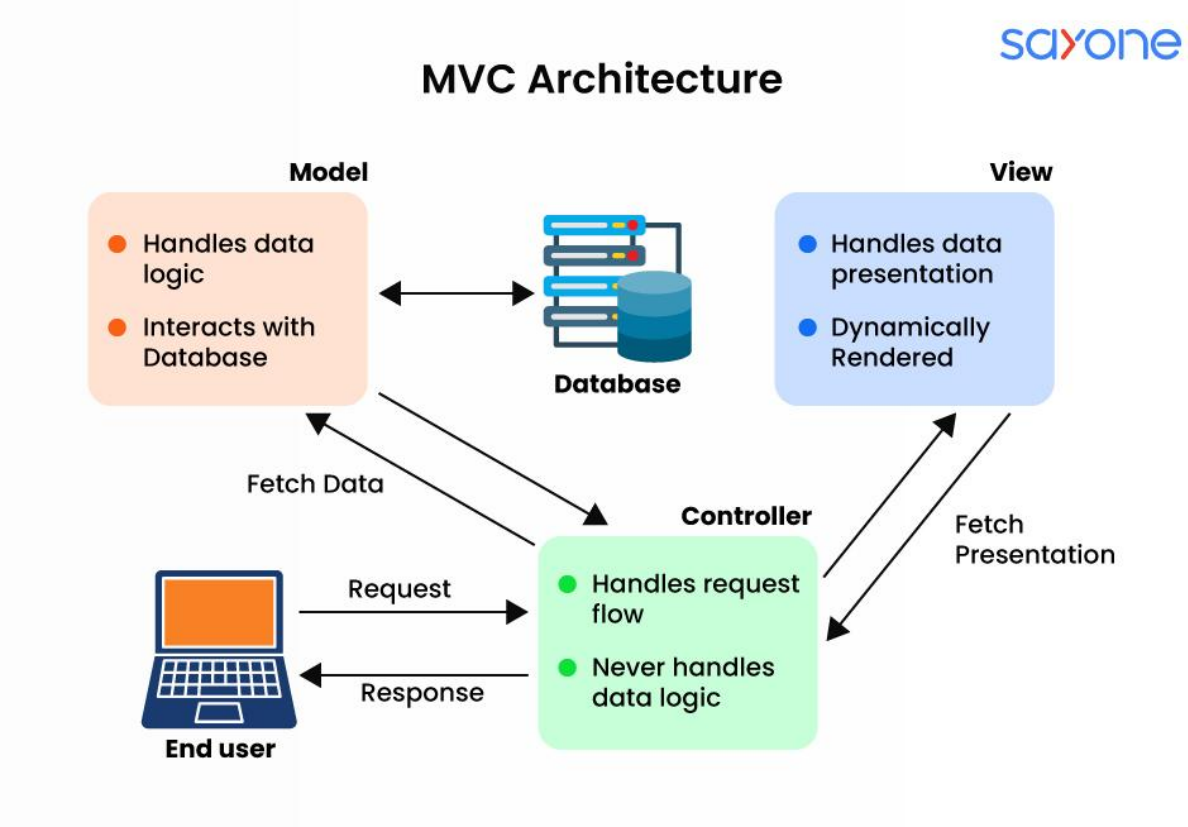


Figure 1.4: MVC

1.7.3- Third: Integration of Client–Server and MVC in the Project

The system in this project is considered a hybrid of both architectures:

Externally, the system is based on the **Client–Server Architecture**, where the user interacts with the application through the client interface, which communicates with the server over the internet. Internally, the server is structured according to the **MVC pattern**, which organizes the data flow within the system between the Model, Controller, and View layers. This combination makes the system scalable and easy to maintain.

System Layers

- **Controller Layer:**
- Handles incoming requests and executes the required logic through modules such as MessageControl, PostControl, StoryControl, and UserControl.
- **Model Layer:**

Manages the data represented by models such as User, Post, Story, Message, Connection, and Email, and interacts directly with the MongoDB database.

- **View Layer:**

Sends the processed data to the Front-end, where it is presented to the user.

External Integrations

The system is also integrated with external services such as:

- **Clerk** for authentication and user management
- **ImageKit** for media handling and optimization
- **Inngest** for background task processing

Data Flow

The data flows through the system in a clear sequence:

User → Interface (Client) → Server (Controllers & Models) → Database → Back to Client (View)

Conclusion

This architecture achieves an optimal balance between **network-based distribution (Client–Server Architecture)** and **internal system organization (MVC Pattern)**. As a result, the platform becomes robust, secure, scalable, and easier to maintain and extend in future development stages.

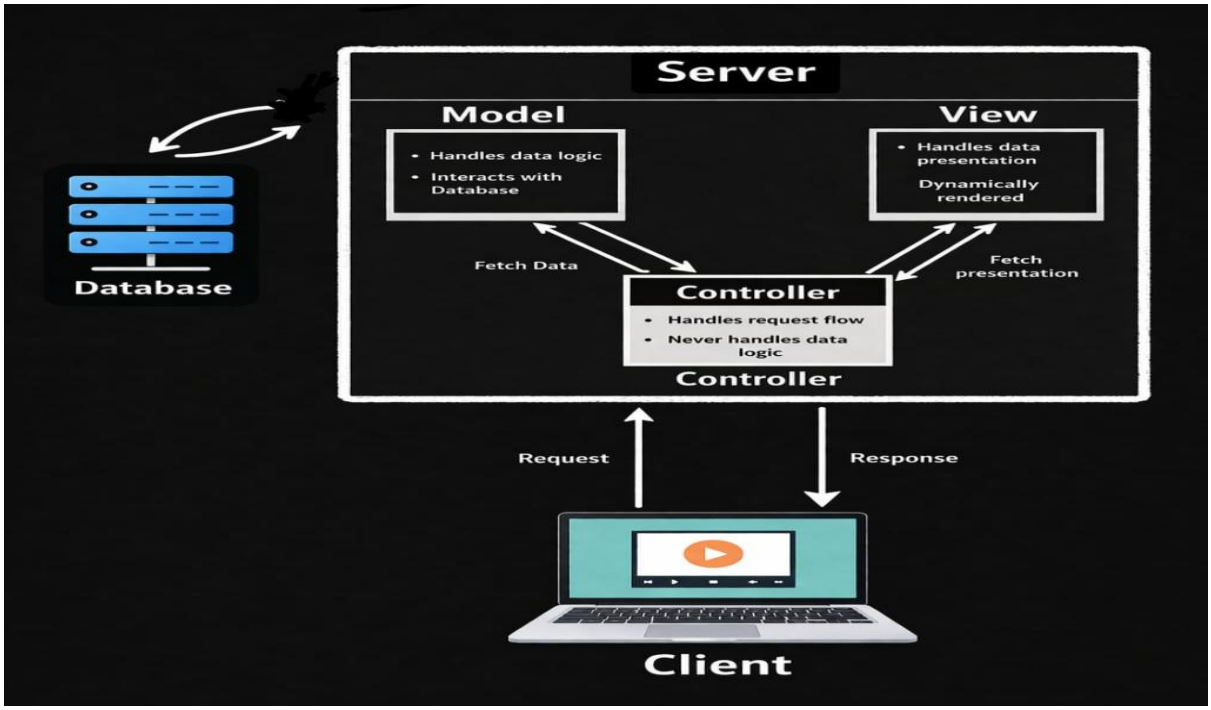


Figure 1.5: Integration of the Two Architectures

1.8- Tools Used

This chapter presents the tools and technologies utilized in developing the Noonify social media platform. The tools are categorized into three main groups: Front-End tools, Back-End tools, and Database tools, in alignment with the MERN Stack architecture.

1.8.1- Front-End Tools

The Front-end of the application was developed using a set of modern tools aimed at building a responsive and interactive user interface :

- **React.js**

- A JavaScript library used for building component-based user interfaces, which facilitates code reusability and improves maintainability.

- **Vite**

A modern build tool used to accelerate the development process and run the project more efficiently compared to traditional tools.

- **JavaScript (ES6+)**

The primary programming language used to implement front-end logic, handle events, and manage user interactions.

- **HTML5**

Used to structure web pages.

- **CSS3**

Used to style user interfaces and enhance the visual experience.

- **Axios**

A library used to send and receive HTTP requests between the front-end and back-end.

- **ESLint**

A tool used for code analysis, detecting errors, and improving code quality.

- **Vercel**

A platform used to deploy and host the front-end application in a production environment.

1.8.2- Back-End Tools

The back-end was developed using technologies that ensure high performance and structured system logic. Key tools include:

- **Node.js**

A JavaScript runtime environment used to execute server-side code efficiently.

- **Express.js**

A Framework built on Node.js used to create RESTful APIs and manage routing.

- **JavaScript**

Used as the primary language for implementing server-side logic.

- **JWT (JSON Web Token)**

Used to implement authentication and verify user identity.

- **Multer**

A library used for handling file uploads (such as images) from users to the server.

- **NodeMailer**

A library used for sending emails, such as verification and notification messages.

- **ImageKit**

A service used for managing, uploading, and storing images in the cloud.

- **Middleware**

Used for handling intermediate requests such as authorization and route protection.

1.8.3- Database Tools

A flexible and scalable database solution was used to store user data and content. The tools include:

- **MongoDB**

A NoSQL database based on documents, known for its flexibility and ease of handling unstructured data.

- **Mongoose**

An Object Data Modeling (ODM) library for MongoDB that simplifies schema creation and data operations.

Summary of Tools Used

The project adopted the MERN Stack as its primary architecture, achieving integration between the front-end, back-end, and database. This integration enabled the development of a modern, well-structured, and scalable social media platform that meets both academic and practical project requirements.

2. Chapter Two: Analytical Study

2.1- General Requirements

Authentication System

- Ability to create a new account (Sign Up) using email or phone number.
- User login (Sign In) using credentials or via social login (Google / GitHub).
- Password reset functionality (Reset Password) in case it is forgotten.
- Logout from the current session (Log Out).

User Profile Management

- Display basic user information (name, profile picture, bio, number of followers).
- Edit profile (Edit Profile) and update user data.
- View other users' profiles (View Other Profiles).
- Display online/offline status (Online / Offline Status).
- Secure authentication and session management using Tokens and Clerk Authentication.

Posts Management

- Create new posts (Create Post) with text or images.
- Edit or delete posts (Edit / Delete Post).
- Display posts on the home page (Feed) in chronological order.
- Interact with posts through likes (Like) and comments (Comment).
- Save favorite posts (Save Post) or share them (Share Post).
- Report inappropriate posts (Report Post).

Stories / Status System

- Upload a new story (Upload Story) as an image or short video.
- View stories from followed users (View Stories).
- Delete stories manually or automatically after 24 hours.
- Display viewers list (Viewers List).

Real-time Chat & Messages

- Start a new conversation (Start Conversation) between users.
- Send and receive text messages, images, and emojis.
- Support real-time chat using Socket.io.
- Display message status (Sent – Delivered – Read).
- Mute or delete conversations (Mute / Delete Chat).

Connections System

- Follow or unfollow users (Follow / Unfollow).
- Send and receive friend requests (Friend Requests).
- Display followers and following lists (Followers / Following Lists).
- Suggest Friends based on shared interests (Friend Suggestions).
- Block or unblock users (Block / Unblock).

Search & Discover System

- Search for users by name or username (Username Search).
- Search for users based on location.
- Search for posts using hashtags (Hashtags).
- Display a Discover page containing trending posts and suggested users.

Notifications System

- Send real-time notifications when interacting with posts or receiving new follows.
- Display all notifications on a dedicated page (Notifications Page).
- Ability to delete notifications or mark them as read (Mark as Read).

Media Management

- Upload images using ImageKit with automatic optimization.
- Support multiple image formats (JPEG, PNG, WEBP).
- Compress images to reduce size without losing quality.
- Protect media URLs from unauthorized access (Access Control).

Backend Operations

- Provide APIs for interacting with users, posts, and messages.
- Execute scheduled tasks using Inngest (e.g., deleting expired stories).
- Perform error logging and activity monitoring (Error Logging & Monitoring).

2.2- Non-Functional Requirements

- **Security:** Use Clerk for authentication and HTTPS for data encryption.
- **Performance:** High loading speed, real-time responsiveness, caching, and real-time chat support.
- **Scalability:** Support horizontal scaling using Node.js and MongoDB.
- **Usability:** Interactive interface with responsive design (Responsive UI).
- **Reliability:** Stable operation with system monitoring (Logging & Monitoring).
- **Maintainability:** Well-structured modular code with proper documentation.
- **Compatibility:** Support for all browsers and devices (Cross-Platform).
- **Privacy:** Protection of user data with flexible privacy settings.

2.3- Requirements to Be Implemented in the Project

Requirement ID	Functional Requirement	Priority
ID-01	Start a new conversation between users	1
ID-02	Send and receive text messages and emojis	1
ID-03	Support real-time messaging	1
ID-04	Display message status (Sent – Delivered – Read)	1
ID-05	Delete or mute conversations	1
ID-06	Follow or unfollow users	2
ID-07	Block or unblock users	2
ID-08	Display online status (Online / Offline)	2
ID-09	Search for posts using hashtags	2
ID-10	View the Explore page containing highly interactive posts and suggested users	2
ID-11	Send real-time notifications when interacting with posts or when new followers appear	3
ID-12	View all notifications on a dedicated page	3
ID-13	Delete notifications or mark them as read	3
ID-14	Upload images and automatically optimize them with support for multiple formats	4
ID-15	Protect media links from unauthorized access	4
ID-16	Log errors and system activities	4
ID-17	Execute scheduled tasks such as deleting expired stories	4
ID-18	Report inappropriate or violating posts	4
ID-19	Suggest friends or groups based on shared interests	5
ID-20	Discover trending content in real time	5

2.4- Use Case Specifications

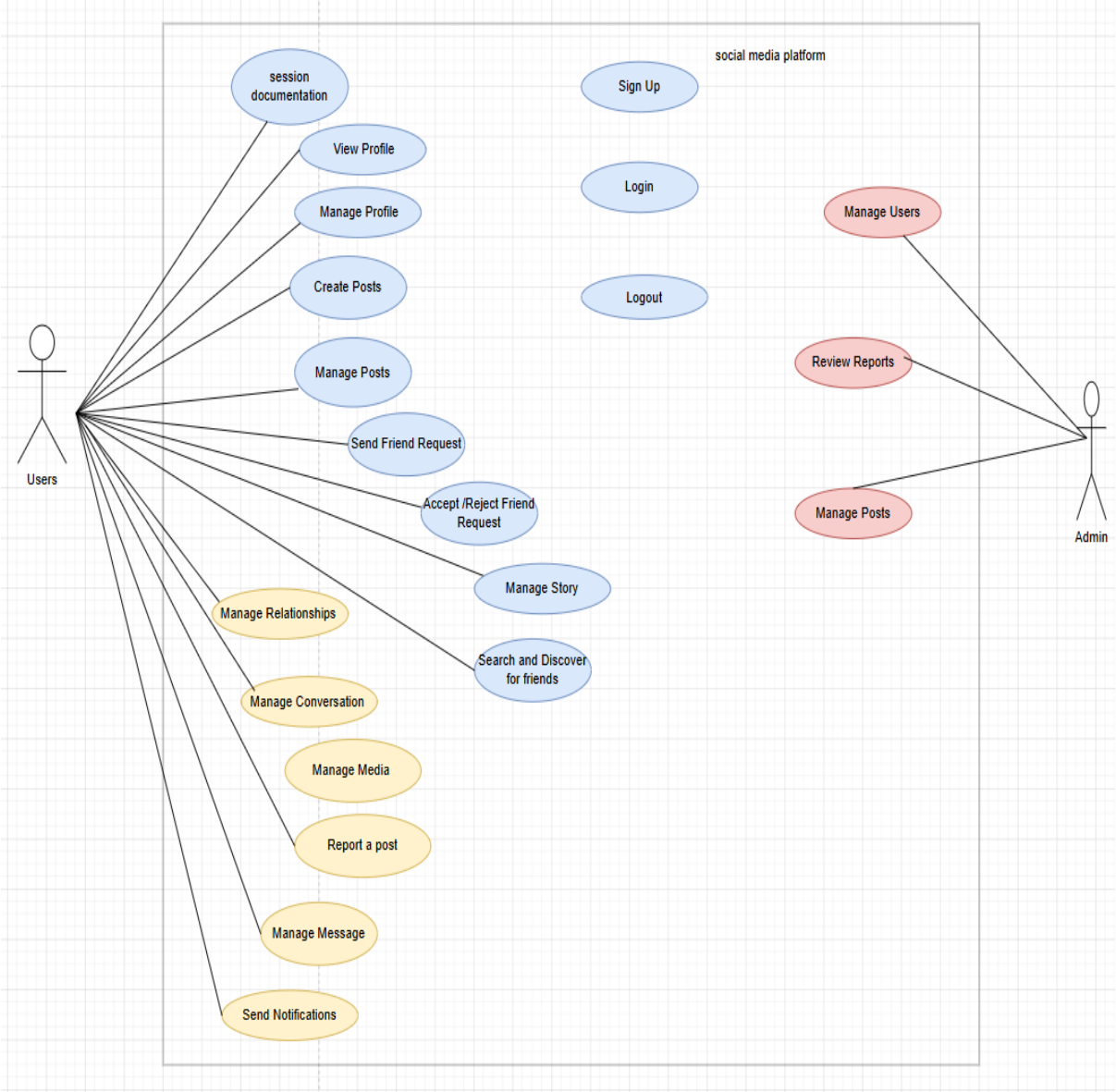


Figure 2.6: Use Case Diagram

2.5-Use Case Specifications

2.6-Activity Diagrams

2.7-Sequence Diagrams

Table 2.2: Use Case Specifications Start New Conservation

Use Case Name/ID	Start New Conversation UC_19
Actors	User
Description	This case allow the user to start a new conversation with another user by selecting the user and sending the first message.
Precondition	1. The user must be logged into the system. 2. The selected user must be exist in the system.
Main flow	1.The user open the Message Page and start new conversation. 2.The system displays a list of search field to find user. 3.The user selects the user. 4.The system opens a chat window. 5.The user writes the first message and click send. 6.The system saves the message and creates a new conversation and it's appear in both user's chat list.
Alternative flow	A3: The system cannot find the user and displays "User Not Found". A5:The user clicks Send without writing a message and display "Message Is Empty".
Postconditions	A new conversation is created in the system and the conversation appears in the user's chat list.

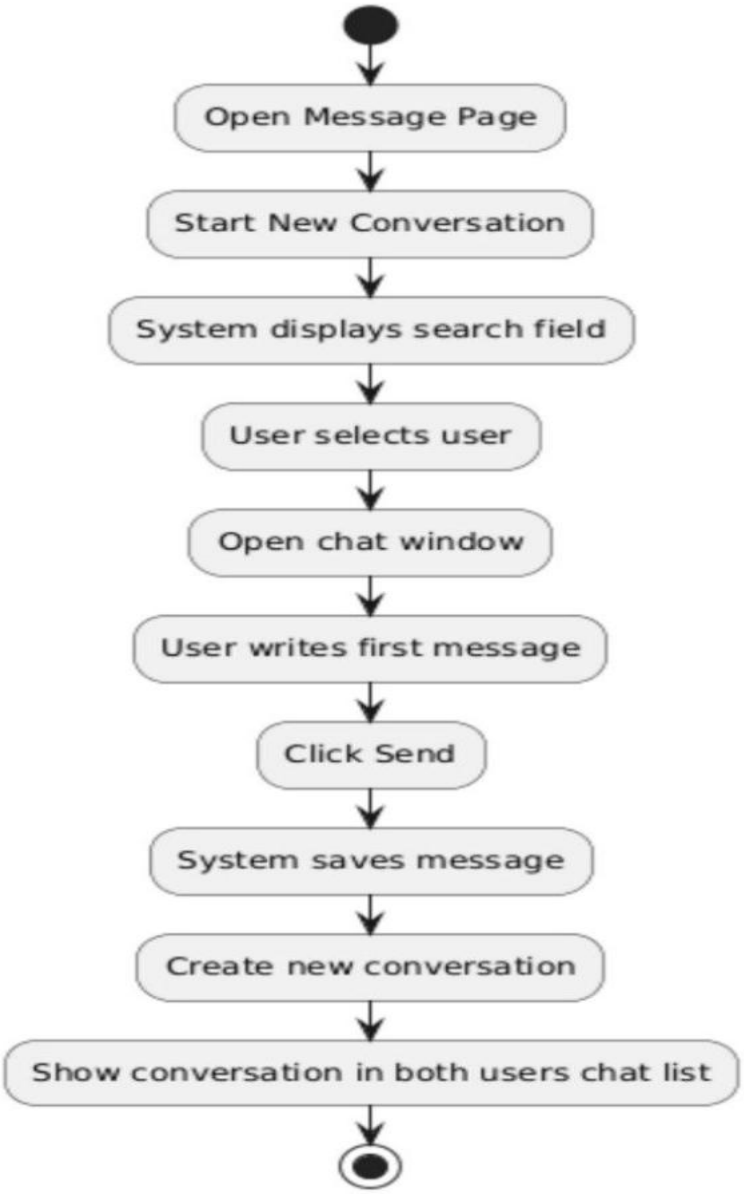


Figure 2.7: Activity Diagram Start New Conservation

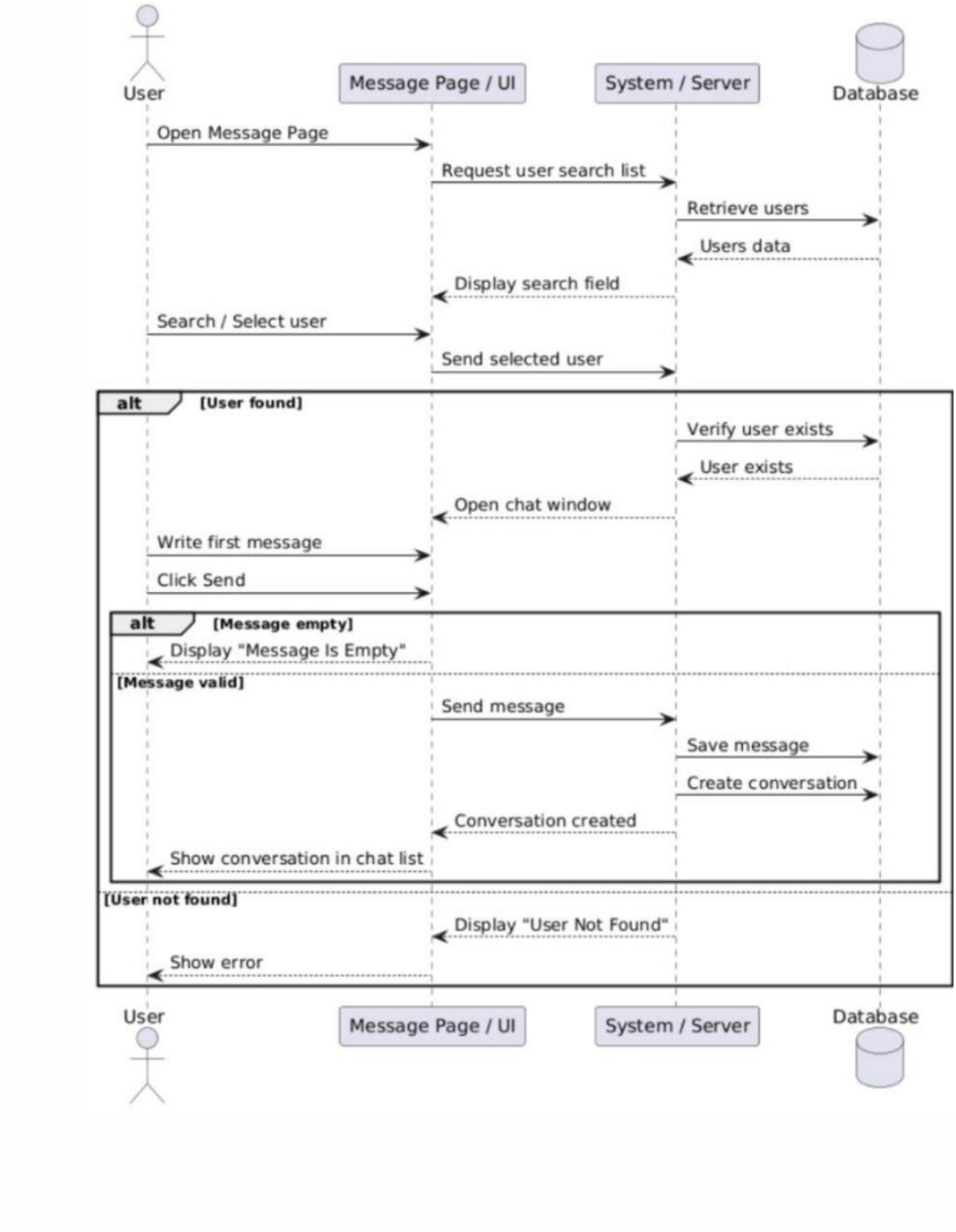


Figure 2.8: Sequence Diagram Specifications Start New Conservation

Table 2.3: Use Case Specifications Send and Receive Text Message and Emojis

Use Case Name/ID	Send and Receive Text Message and Emojis UC_20
Actors	User
Description	This case allow the user to start send and receive text message and emojis within an existing conversation in the chat system.
Precondition	1.The user must be logged into the system. 2.A conversation between users must already exist.
Main flow	1.The user open the Message Page and select an existing conversation. 2.The system displays previous message. 3.The user writes a text message or selects an emoji and click Send. 4.The system sends the message to the server and saves in the database and the message displays in the chat window. 5.The receiver receives the message.
Alternative flow	A3:The user clicks Send without writing a message and display ”Message Is Empty”.
Postconditions	The message appears in the conversation window.

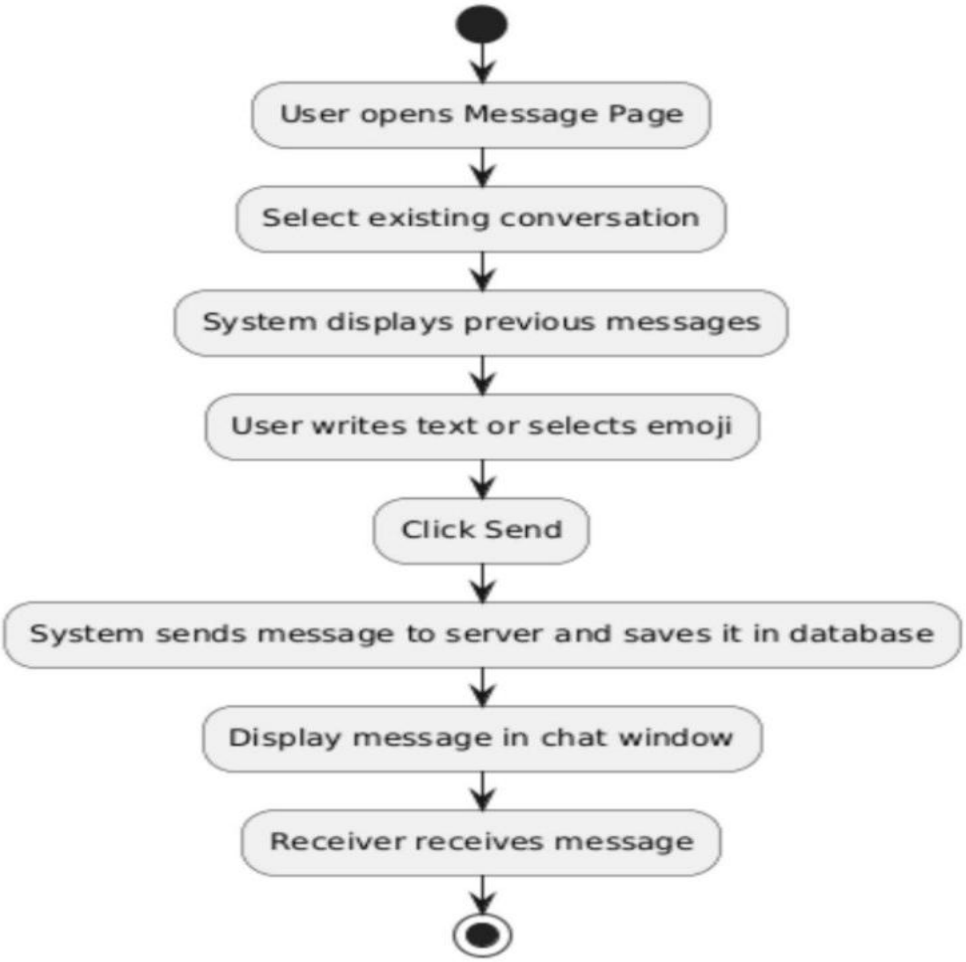


Figure 2.9: Activity Diagram Send and Receive Text Message and Emojis

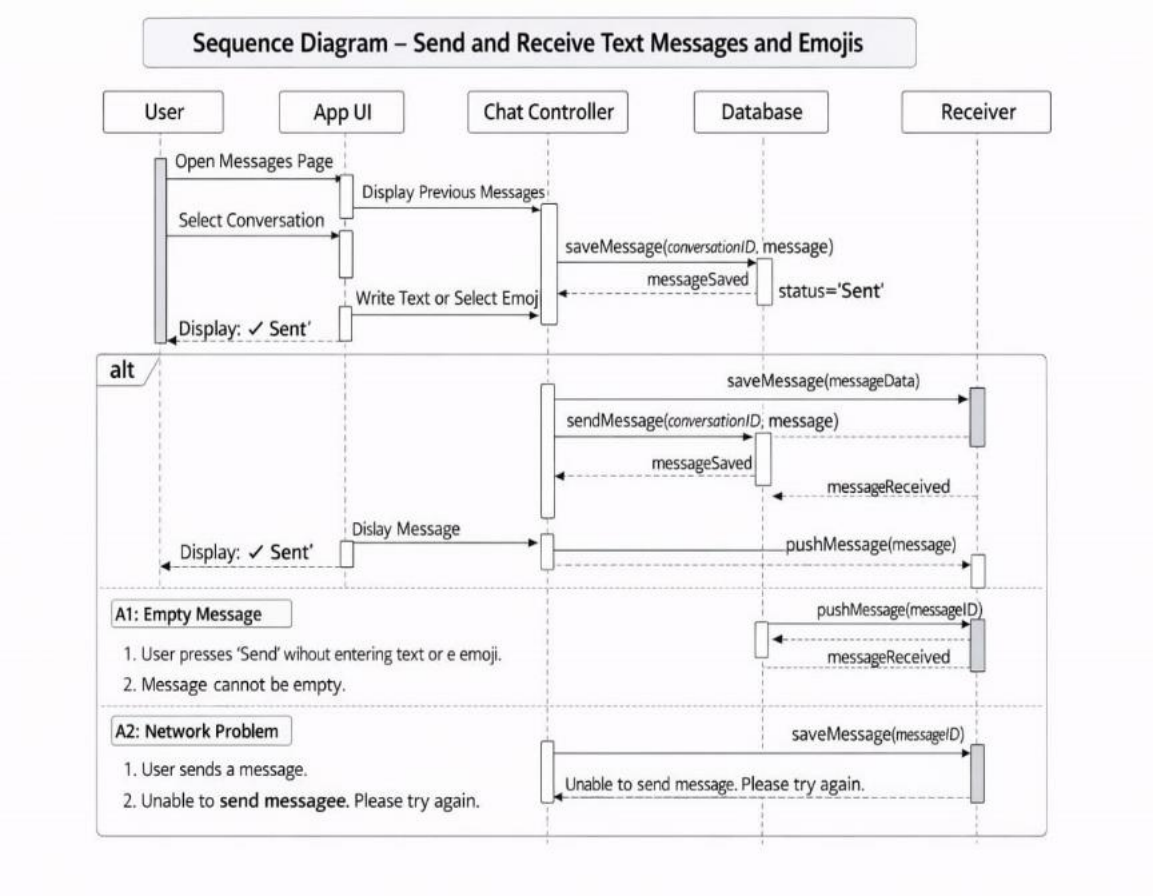


Figure 2.10: Sequence Diagram Send and Receive Text Message and Emojis

Table 2.4: Use Case Specifications Support Real-Time Message

Use Case Name/ID	Support Real-Time Message UC_21
Actors	User
Description	This case allow the user to start send and receive text message instantly in real time within an active conversation without refreshing the page.
Precondition	1. The user must be logged into the system. 2. The user must open an existing conservation.
Main flow	1.The user open the chat conversation. 2.The system connect to the real time message service. 3.The user writes a message and click send. 4.The system saves the message to the server that stores it in database and pushed the message instantly to the receiver. 5.The receiver sees the message immediately in the chat window.
Alternative flow	A5: The receiver is offline it receives the message when they open the chat.
Postconditions	The message appears immediately in chat window.

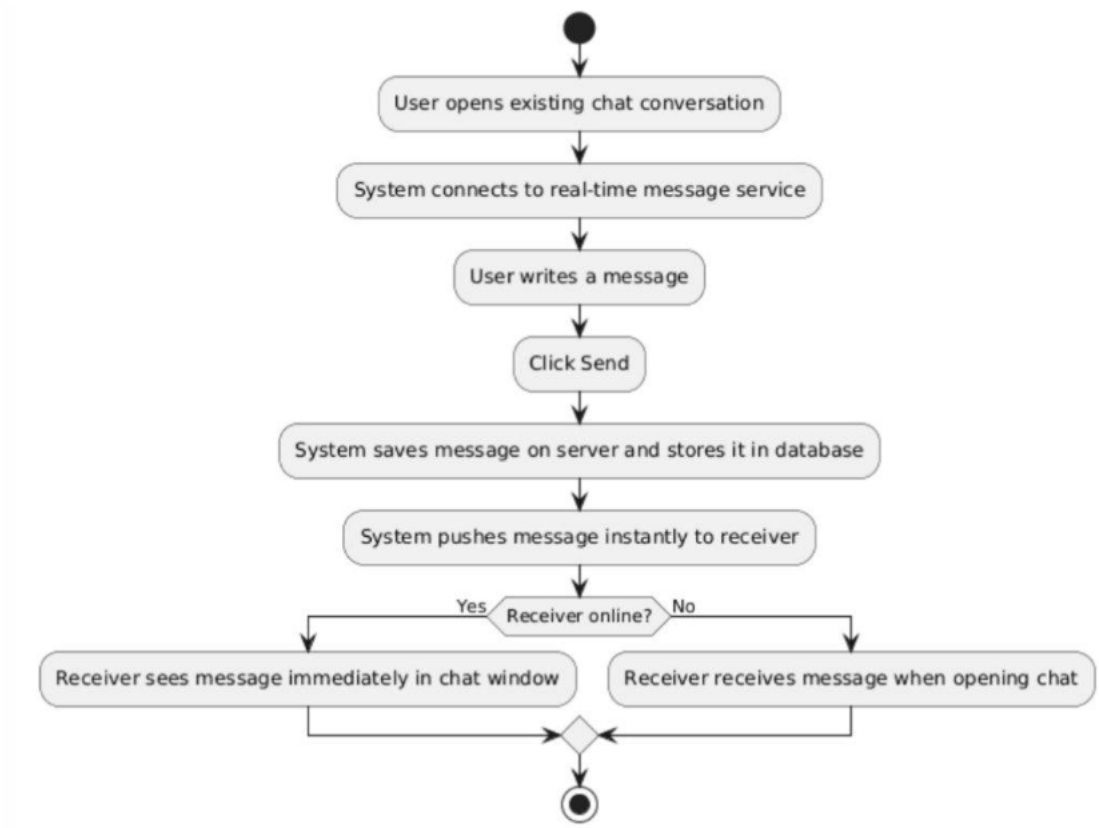


Figure 2.11: Activity Diagram Support Real-Time Message

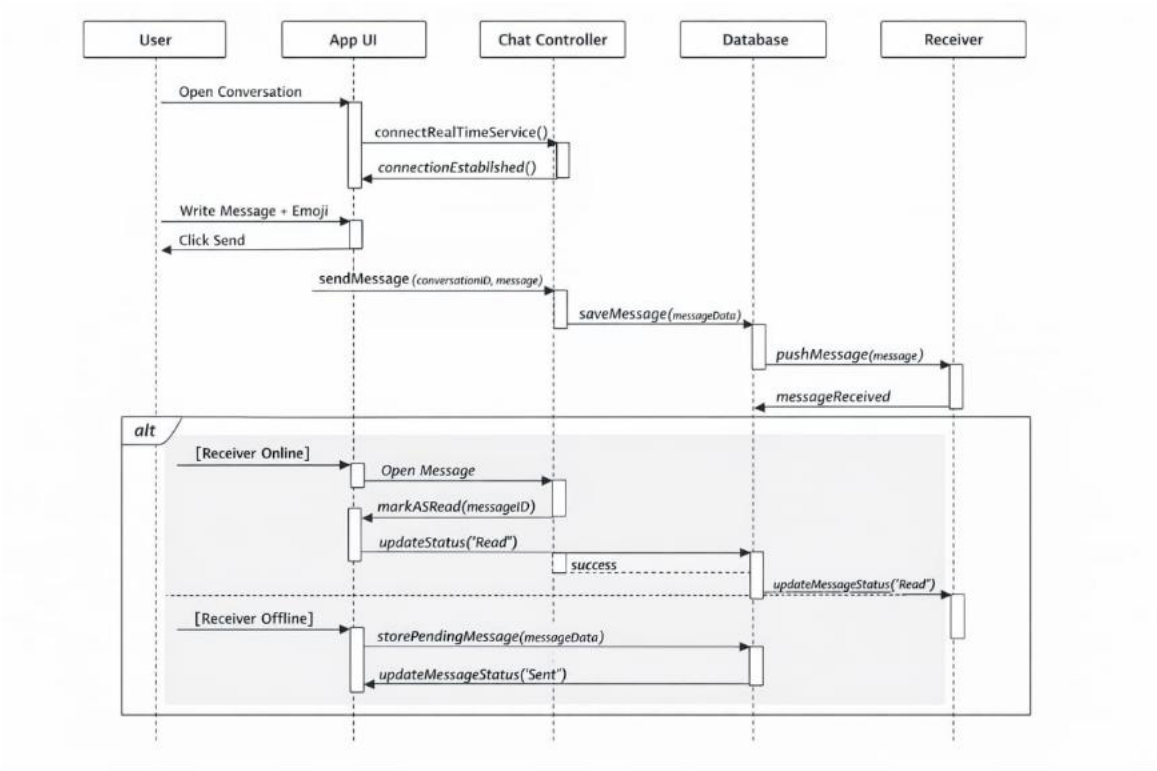


Figure 2.12: Sequence Diagram Support Real-Time Message

Table 2.5: Use Case Diagram Display Message Status (Sent –Delivered – Read)

Use Case Name/ID	Display Message Status (Sent –Delivered – Read) UC_22
Actors	User
Description	This case allow the system to display the status of a message in a conversation, indicating whether the message is sent ,delivered ,read.
Precondition	1. The user must be logged into the system. 2. The user must send a message.
Main flow	1. The user writes a message and click send. 2.The system send the message to the server that stores it in database and displays Sent status. 3.When the receiver receives the message, the system updates the status to Delivered. 4. When the receiver opens the message, the system updates the status to Read.
Alternative flow	A3: The receiver is offline it receives the system displays Sent until the receives connects when its connect ,the status change to Delivered.
Postconditions	The sender can see the message status and the receiver's action update status automatically.

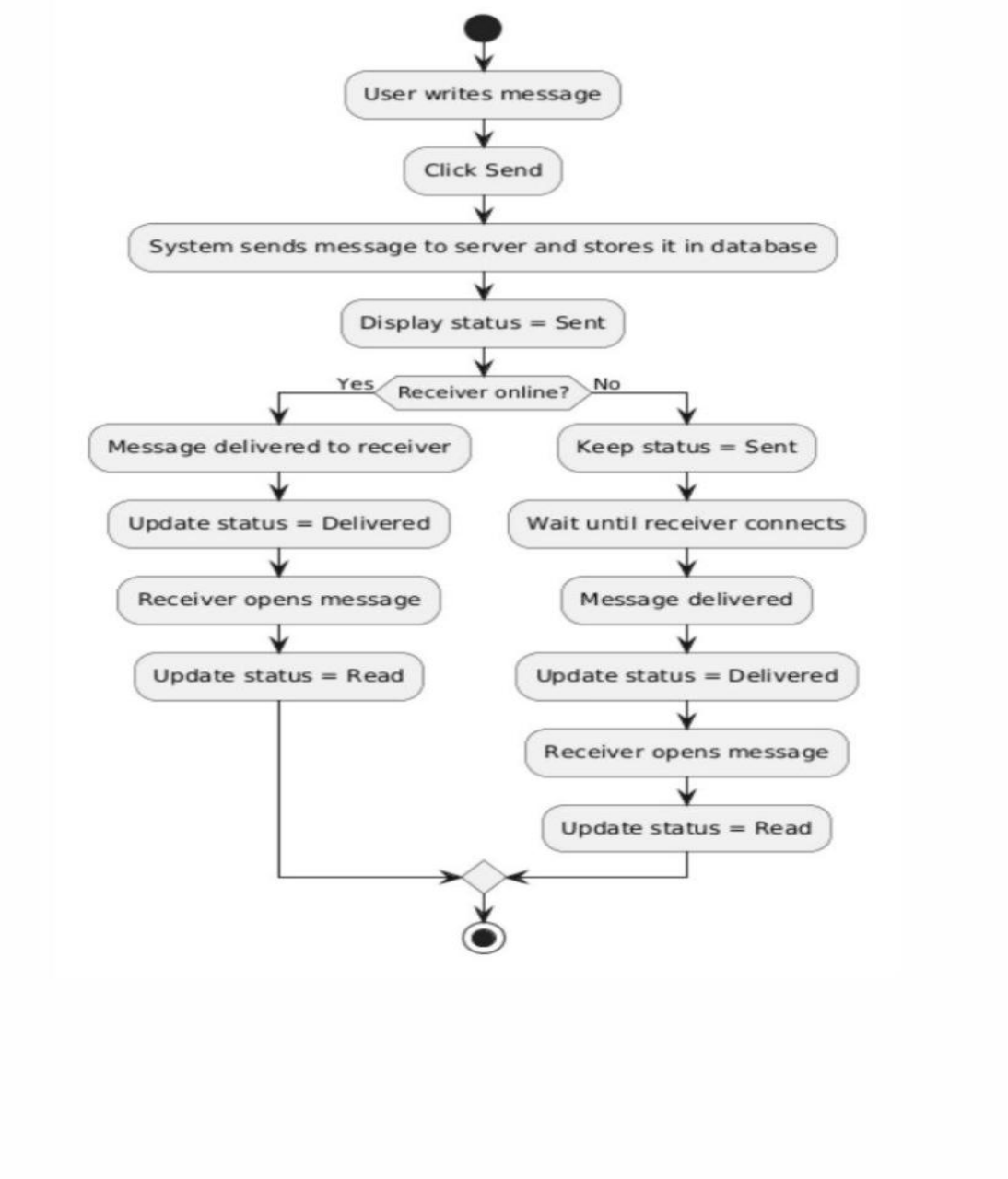


Figure 2.13: Activity Diagram Display Message Status (Sent –Delivered – Read)

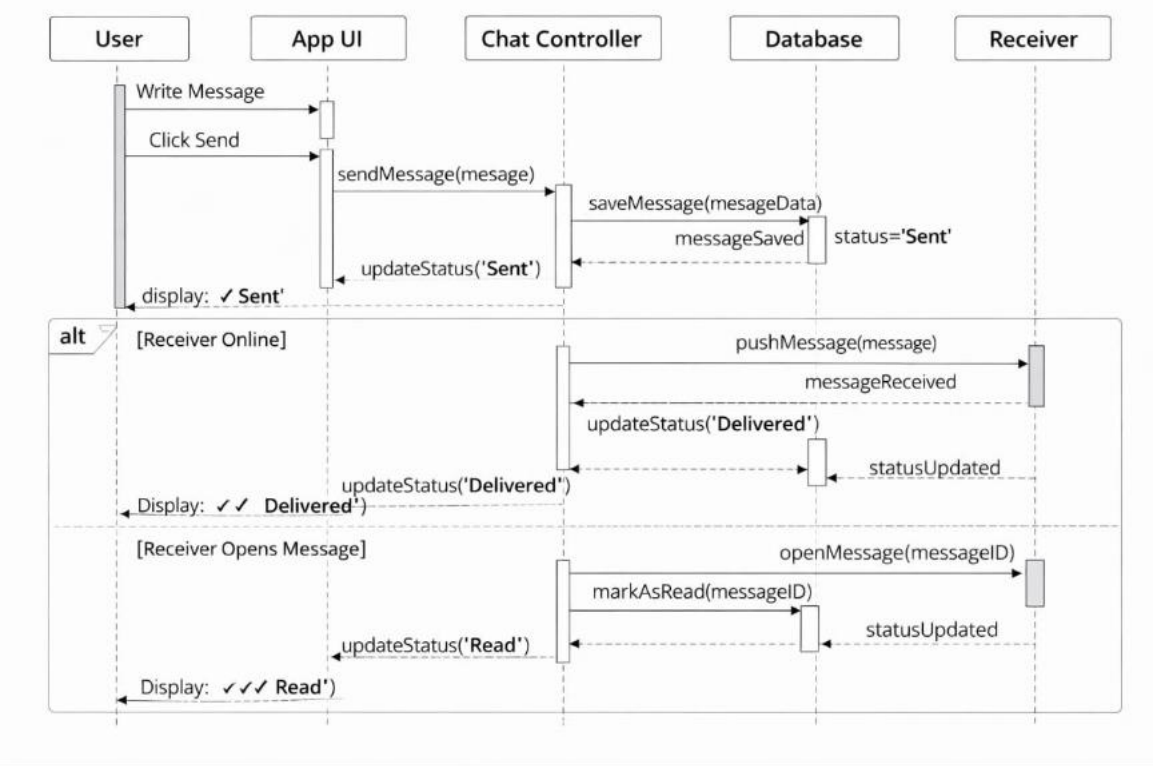


Figure 2.14: Sequence Diagram Display Message Status (Sent –Delivered – Read)

Table 2.6: Use Case Diagram Delete or Mute Conversation

Use Case Name/ID	Delete or Mute Conversation UC_23
Actors	User
Description	This case allow the system to delete a conversation or mute notifications from a specific conversation.
Precondition	1. The user must be logged into the system. 2. The conversation must already exist.
Main flow	1. The user opens a Message Page, select a conversation and clicks Options. 2. The system displays available actions . 3. The user chooses Delete Conversation or Mute Conversation. 4. The system process the request , updates the database and update the UI .
Alternative flow	A3: if the user selects a delete conversation the system sends the request to the server , the system removes the conversation from the database ,the conversation disappears from the chat list. A3: if the user selects Mute Conversation, system sends a mute request to the serer, updates the conversation settings in the database ,notifications for that conversation are disabled.
Postconditions	The conversation is deleted from the user's chat list or notification for the conversation is muted.

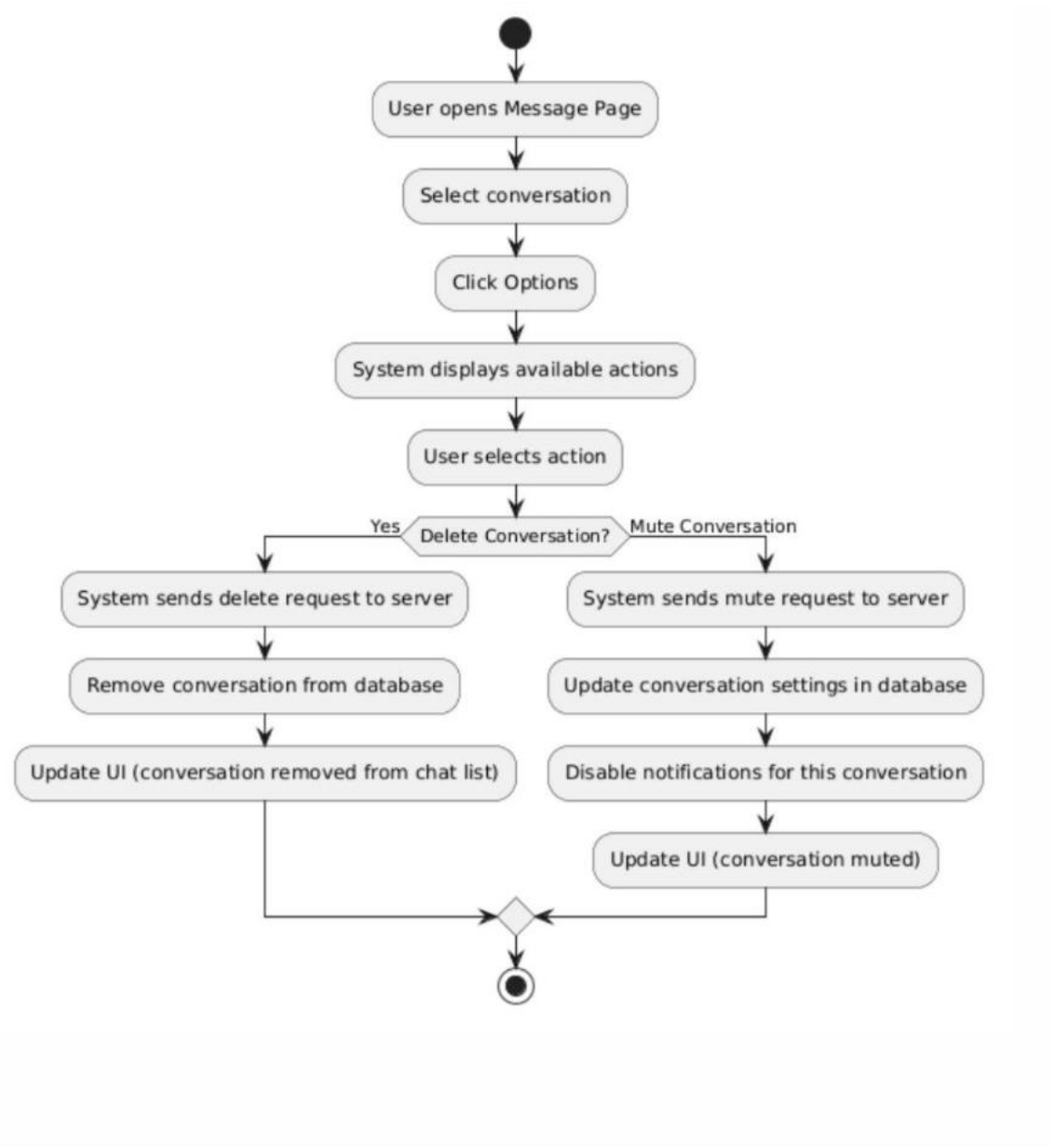


Figure 2.15: Activity Diagram Delete or Mute Conversation

Figure 2.16: Sequence Diagram Delete or Mute Conversation

Table 2.7: Use Case Diagram Block or Unblock User

Use Case Name/ID	Block or Unblock User UC_24
Actors	User
Description	This case allow the user to block or unblock another user.
Precondition	1. The user must be logged into the system. 2. A relation or interaction between users exists.
Main flow	1. The user opens a Connection and click view Profile. 2. The system displays available actions. 3. The user selects Block or Unblock . 4. The system processes the requests , update the database, and updates UI.
Alternative flow	A3: if the user selects block the system blocks the user , Message prevented between the users. A3: if the user selects unblock the system removes the block , Communication is restored.
Postconditions	The blocked user cannot send message , if unblocked communication is restored.

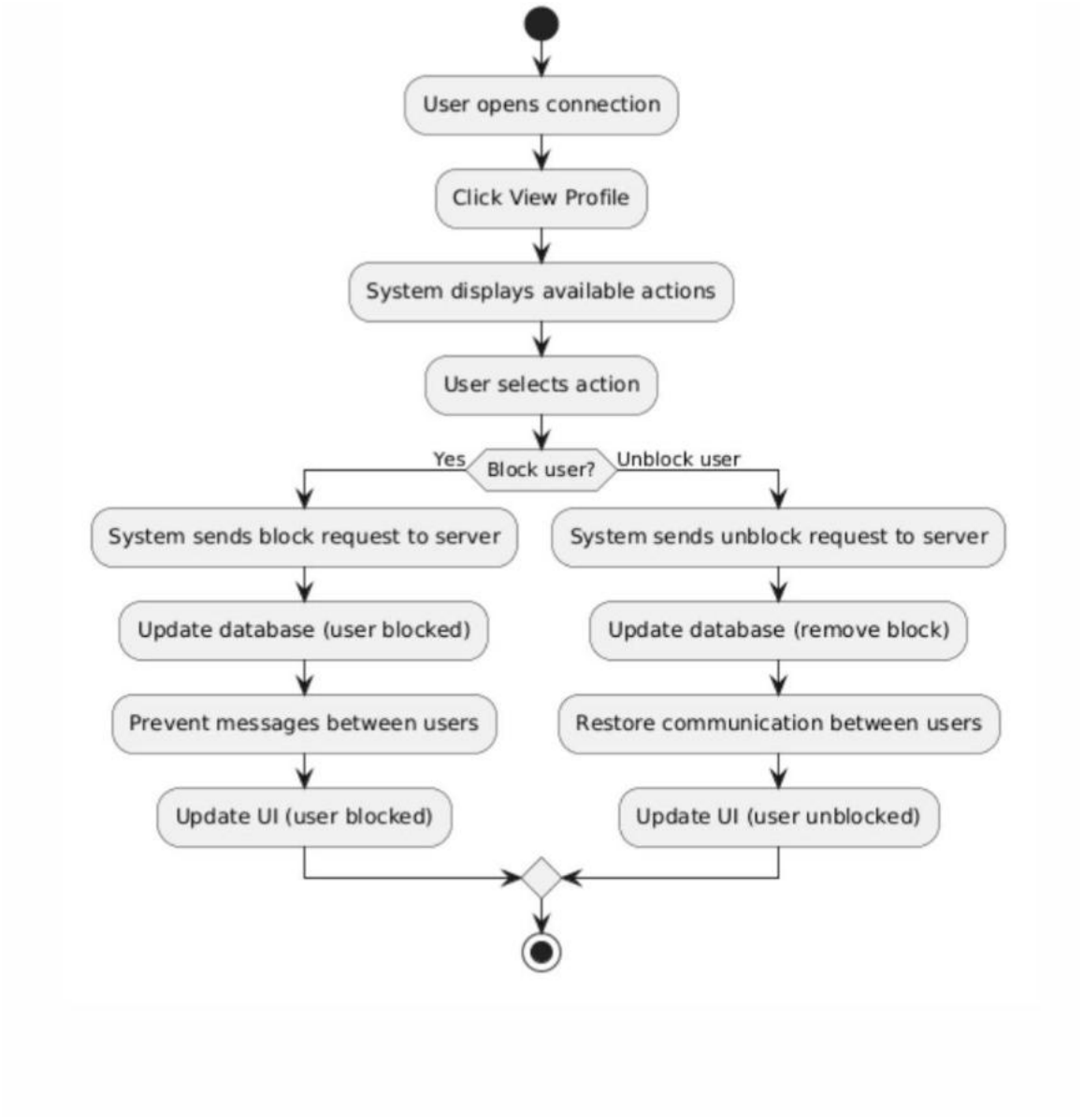


Figure 2.17: Activity Diagram Block or Unblock User

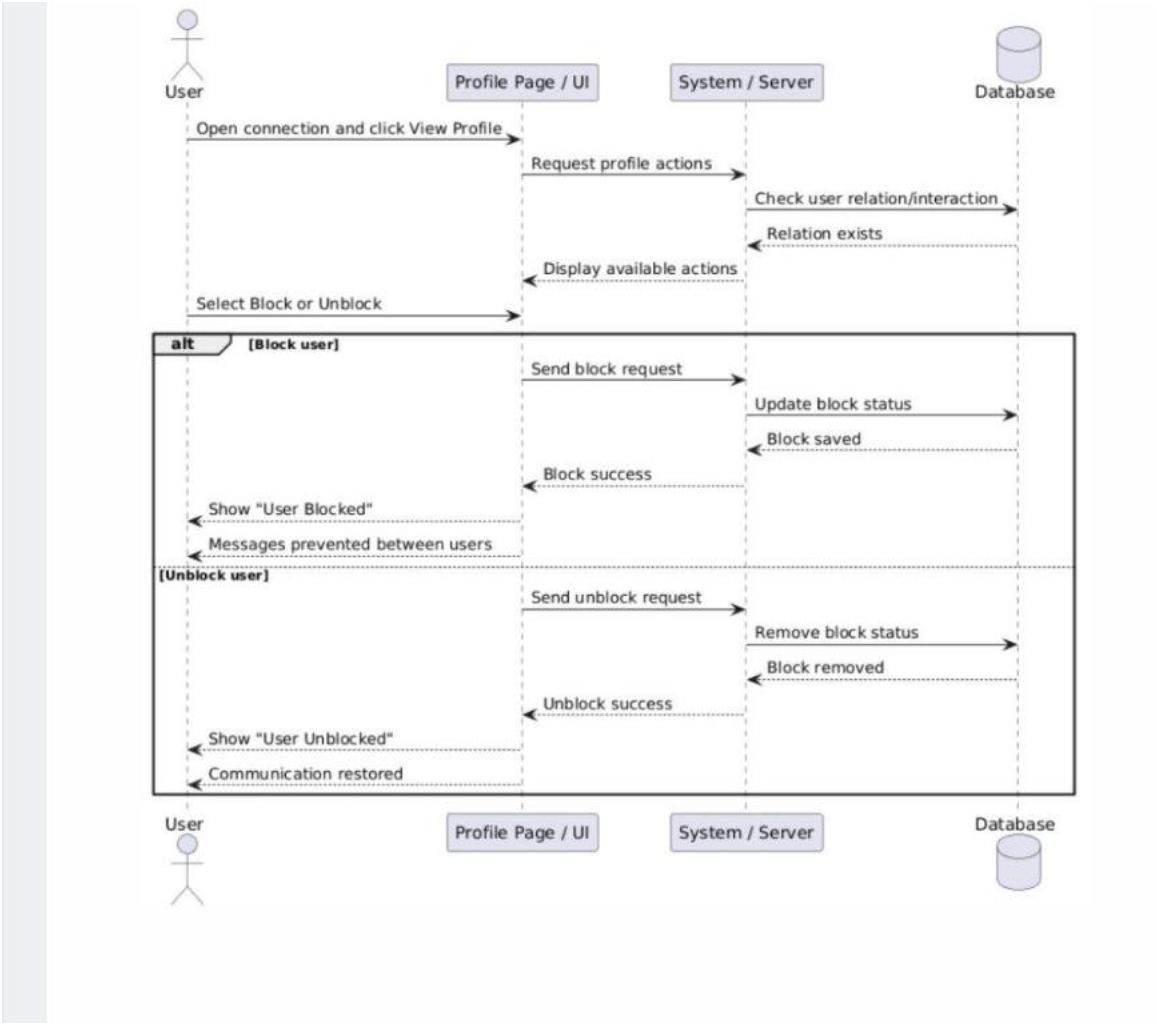


Figure 2.18: Sequence Diagram Block or Unblock User

Table 2.8: Use Case Diagram Display online Status(Online / Offline)

Use Case Name/ID	Display online |Status (Online / Offline) UC_25
Actors	User
Description	This case allow the user to display whether a user is online or offline in real time.
Precondition	1. The user must be logged into the system.
Main flow	1. The user opens the chat or Mesasage . 2. The system requests the status of other users , retrieves status from the server , display Online or Offline .
Alternative flow	A2: User Online : the system displays Online A2: User Offline : the system displays |Offline .
Postconditions	The user status is displayed correctly , and the system updates the status dynamically.

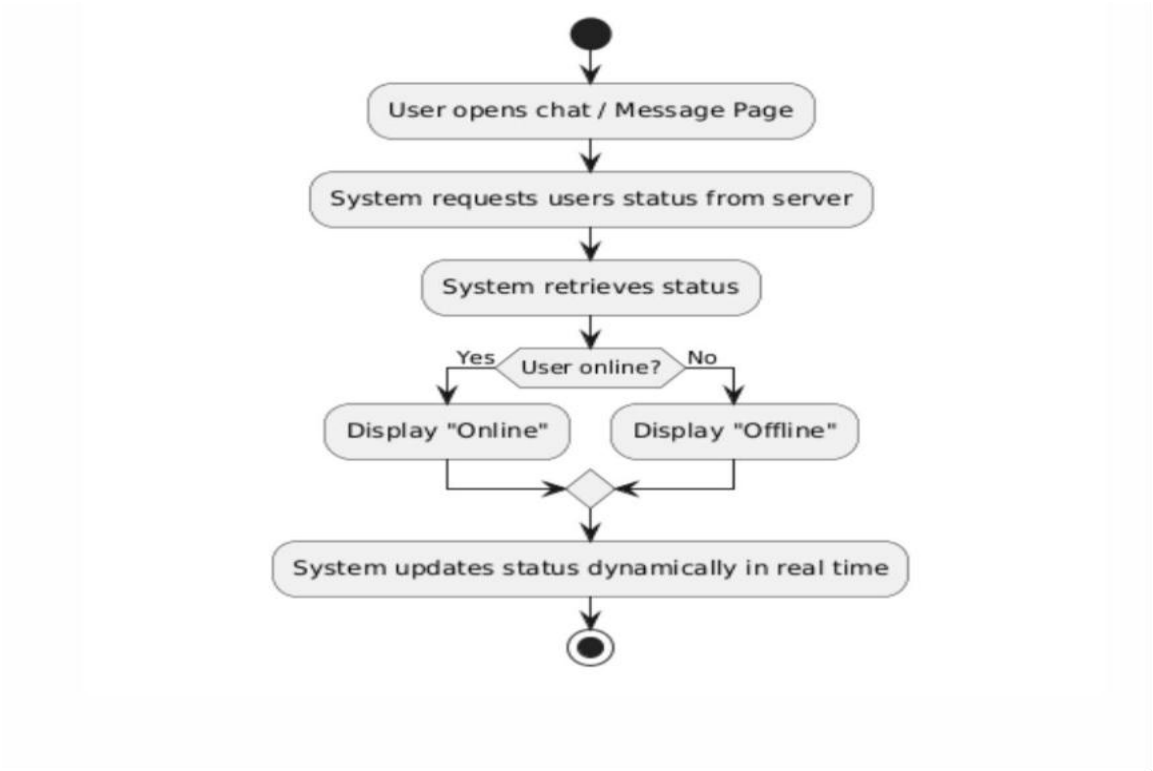


Figure 2.19: Activity Diagram Block or Unblock User

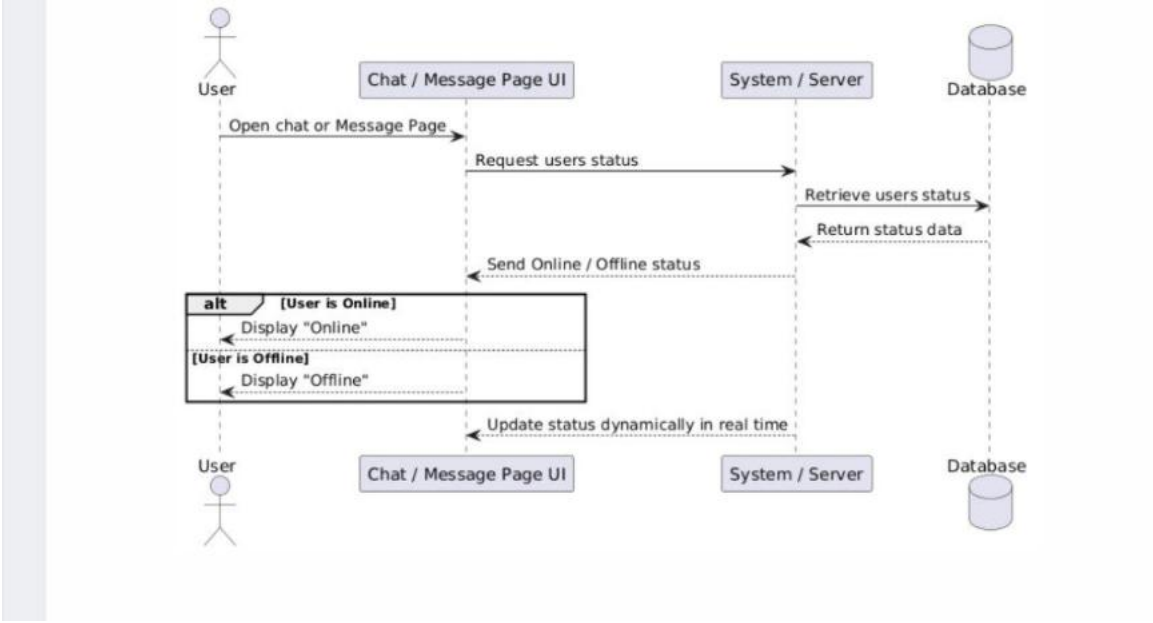


Figure 2.20: Sequence Diagram Block or Unblock User

Table 2.9: Use Case Diagram Search For Posts Using Hashtags

Use Case Name/ID	Search For Posts Using Hashtags UC_26
Actors	User
Description	This use case allows the user to search for posts using a specific hashtag and view matching results.
Precondition	1. The user must be logged into the system. 2. Posts with Hashtags must already exists in system.
Main flow	1. The user opens the Search Page enters a hashtags and click Search. 2. The system sends the request to the server , retrieves matching posts from the database and displays the results.
Alternative flow	A2: Posts Found : the system displays a list of posts. A2: Posts Found: the system displays “No result found”.
Postconditions	The system displays posts matching the hashtags if no posts are found an appropriate message is shown.

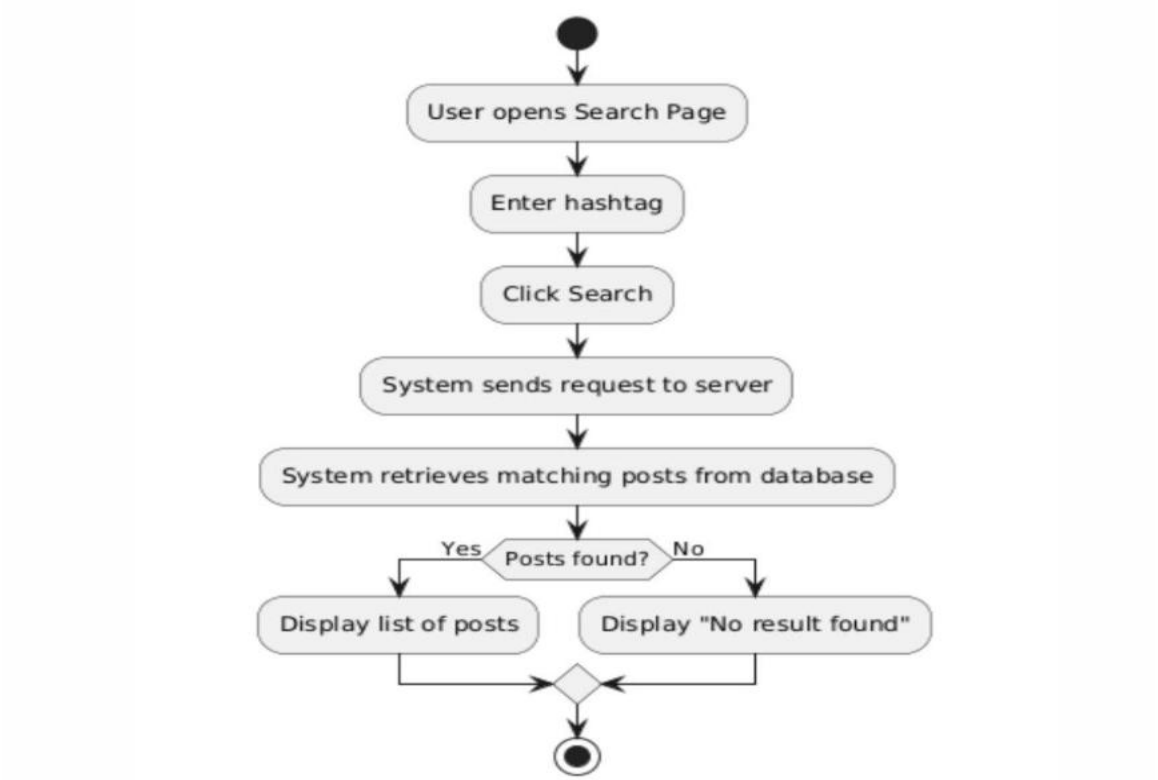


Figure 2.21: Activity Diagram Search |For Posts Using Hashtags

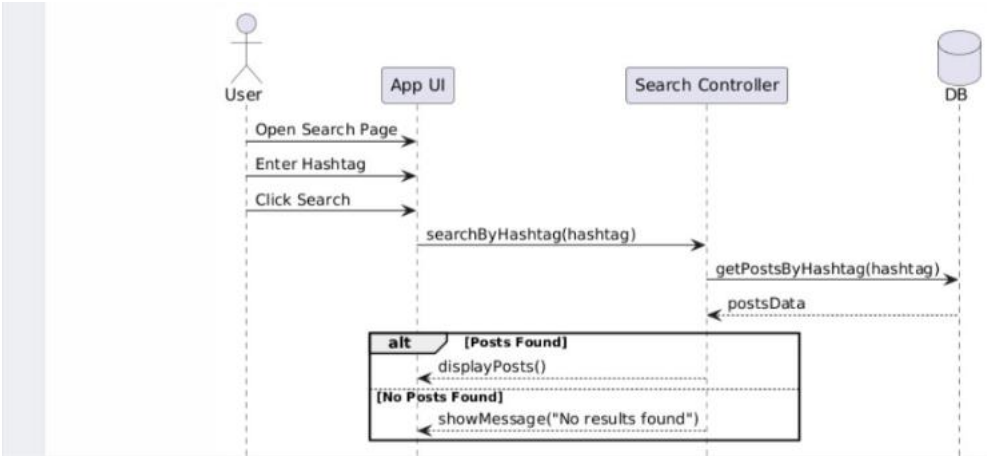


Figure 2.22: Sequence Diagram Search |For Posts Using Hashtags

Table 2.10: Use Case Diagram View Explore Page

Use Case Name/ID	View Explore Page UC_27
Actors	User
Description	This use case allows the user to view the Explore page, which contains highly interactive posts and a list of suggested users to follow.
Precondition	1. The user must be logged into the system. 2. The system must have posts and user data available.
Main flow	1. The user opens the Explore page. 2. The system sends a request to the server, retrieves trending or interactive posts, retrieves suggested users, processes and ranks the data and displays posts and suggestions.
Alternative flow	A2: Data Available : the system displays posts and suggested users. A2: Data Not Available: the system displays “No content available”.
Postconditions	Interactive posts are displayed , suggested users are shown and the content is updated dynamically.

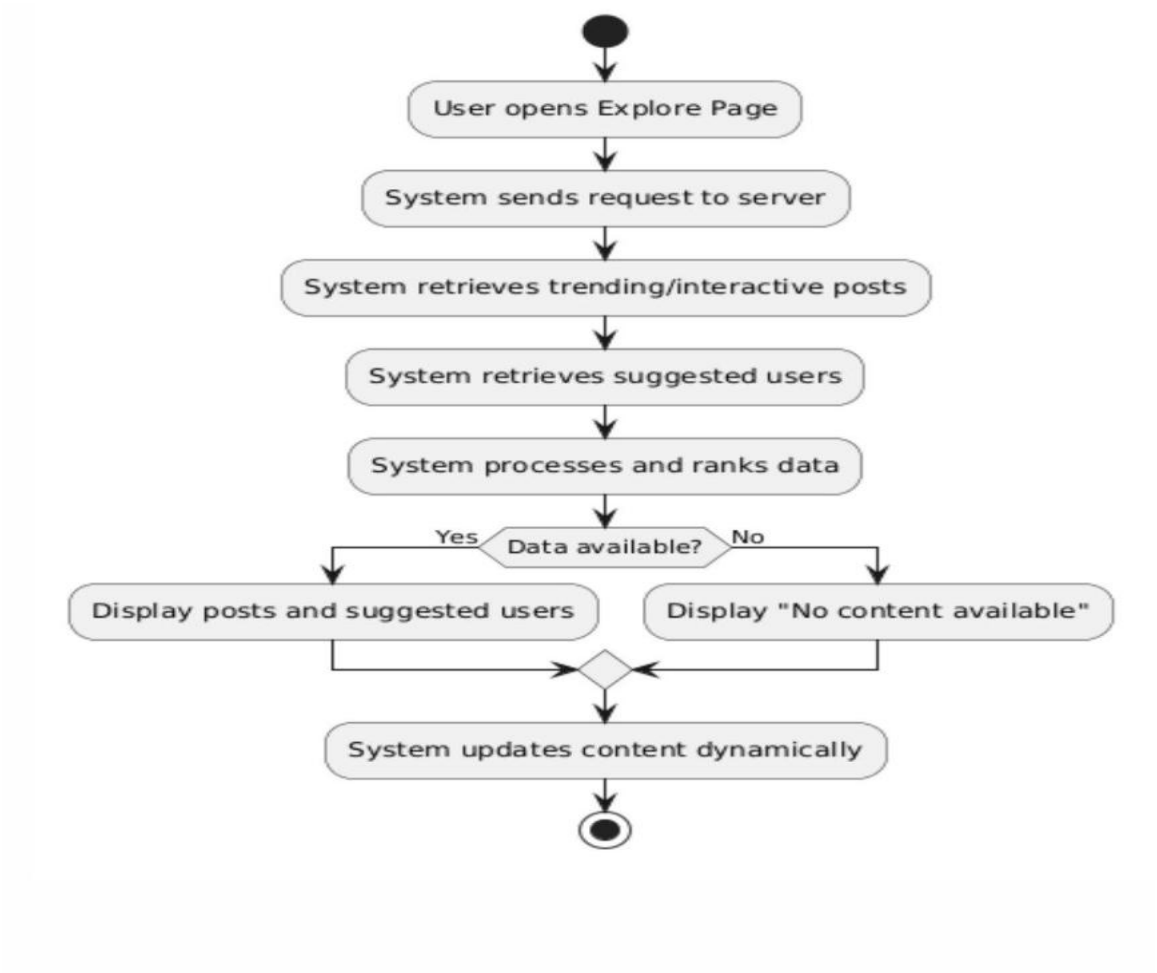


Figure 2.23: Activity Diagram View Explore Page

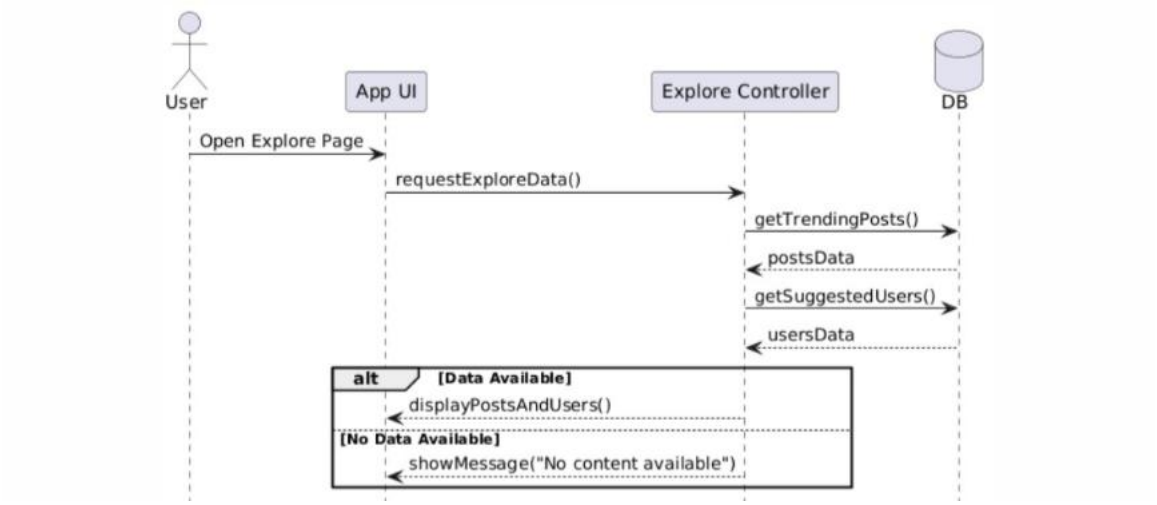


Figure2.24: Sequence Diagram View Explore Page

Table 2.11: Use Case Diagram Send Real-Time Notification

Use Case Name/ID	Send Real-Time Notification UC_28
Actors	User
Description	This use case allows the system to send real-time notifications to user when someone interacts with their posts or new user follows them .
Precondition	1. The user must be logged into the system.
Main flow	1. The user performs an action (like , comment ,share ,follow). 2. The system sends the request to the server , process the action , creates a notification , stores the notification in the database and pushes the notification to the real-time. 3. The target user receives the notification instantly.
Alternative flow	A3: User Online : The system pushes the notification instantly. A3: User Offline : The system stores the notification it receives to the user when they login.
Postconditions	The notification is delivered instantly to the target user and the user sees the notification in real-time .

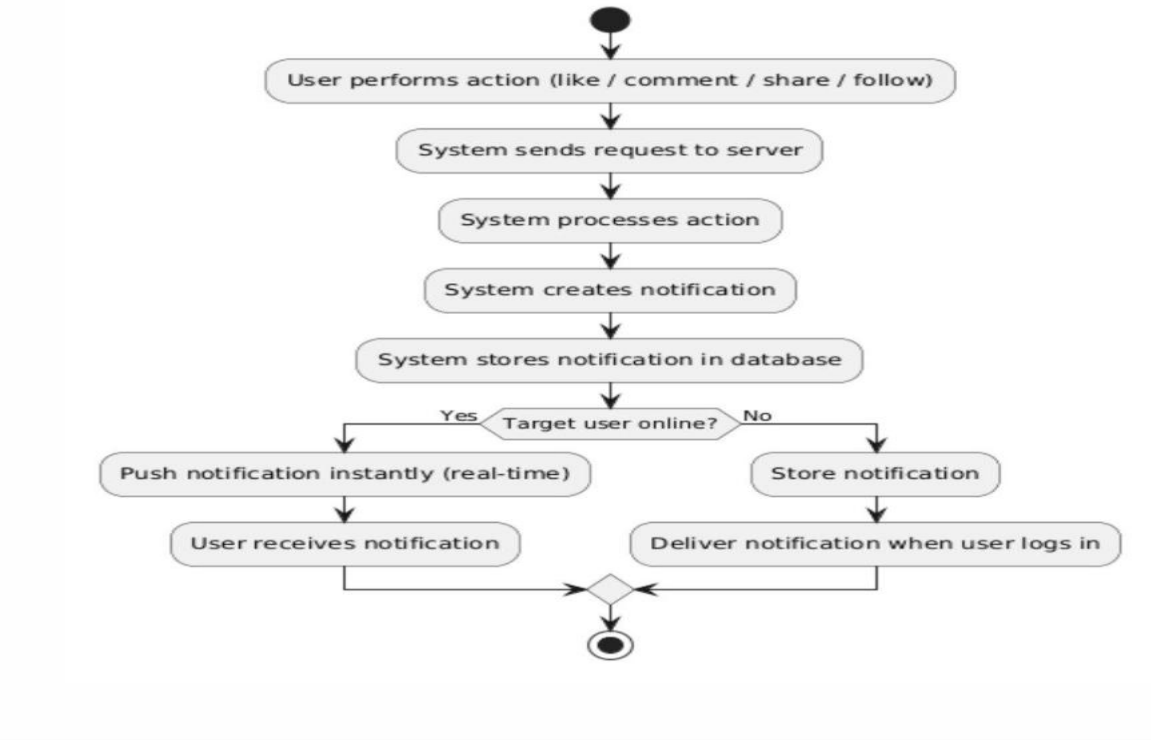


Figure 2.25: Activity Diagram Send Real-Time Notification

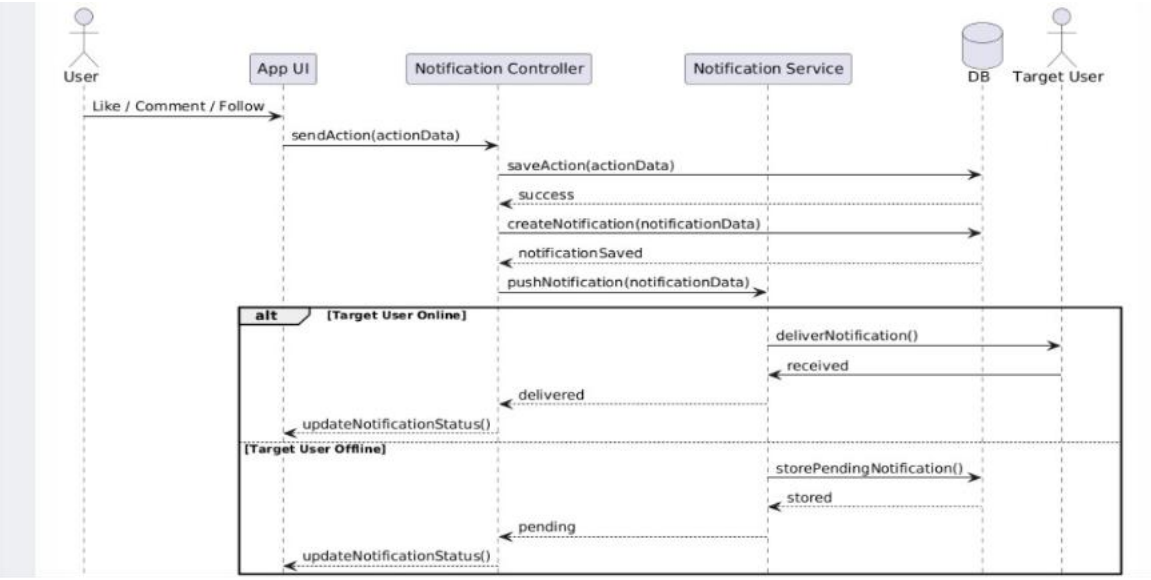


Figure 2.26: Sequence Diagram Send Real-Time Notification

Table 2.12: Use Case Diagram View All Notification

Use Case Name/ID	View All Notification UC_29
Actors	User
Description	This use case allows the user to view all Notification on a dedicated Notification page , including interactions and system alerts .
Precondition	1. The user must be logged into the system. 2. Notification must exists in the system.
Main flow	1. The user opens the Notification page. 2. The system sends the request to the server , retrieves all notifications , displays the notification lists.
Alternative flow	A2: Notification Available : The system displays the list of notifications. A2: Notification Not Available : The system displays “No Notification Available”.
Postconditions	Notification are displayed to the user and Notification may be marked as read .

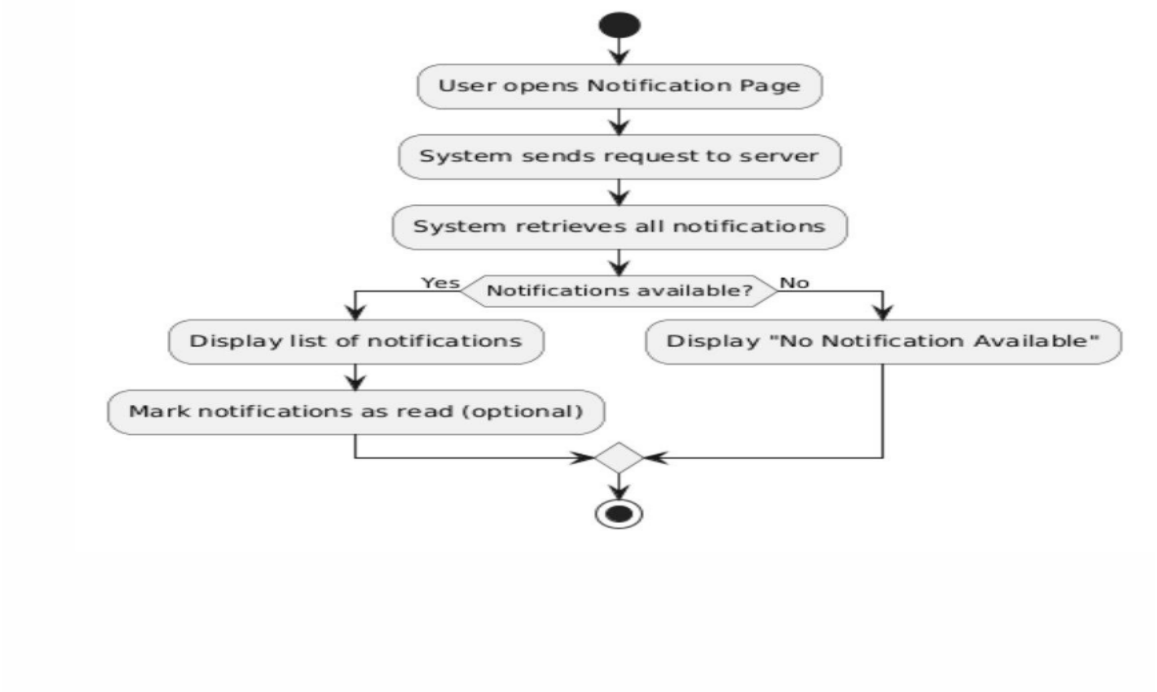


Figure 2.27: Activity Diagram View All Notification

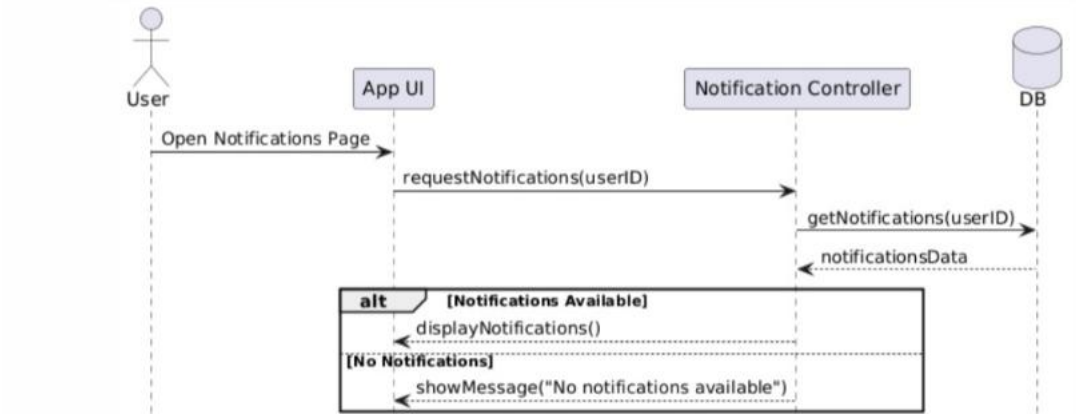


Figure 2.28: Sequence Diagram View All Notification

Table 2.13: Use Case Diagram Delete Notification or Mark as Read

Use Case Name/ID	Delete Notification or Mark as Read UC_30
Actors	User
Description	This use case allows the user to either delete notification or mar k them as read from the notification page.
Precondition	1. The user must be logged into the system. 2. Notification must exists in the system.
Main flow	1. The user opens the Notification page and selects the notification than can choose an action (Delete or Mark as Read). 2. The system sends the request to the server , updates the database and UI.
Alternative flow	A1 : Delete Notification : if the user select delete the system removes the notification from the database and the notification disappears from the list. A2 : if the user select Mark as Read the system updates the notification status and the notification is marked as read
Postconditions	Notification are either deleted or updated as read , UI reflects the updated notification state.

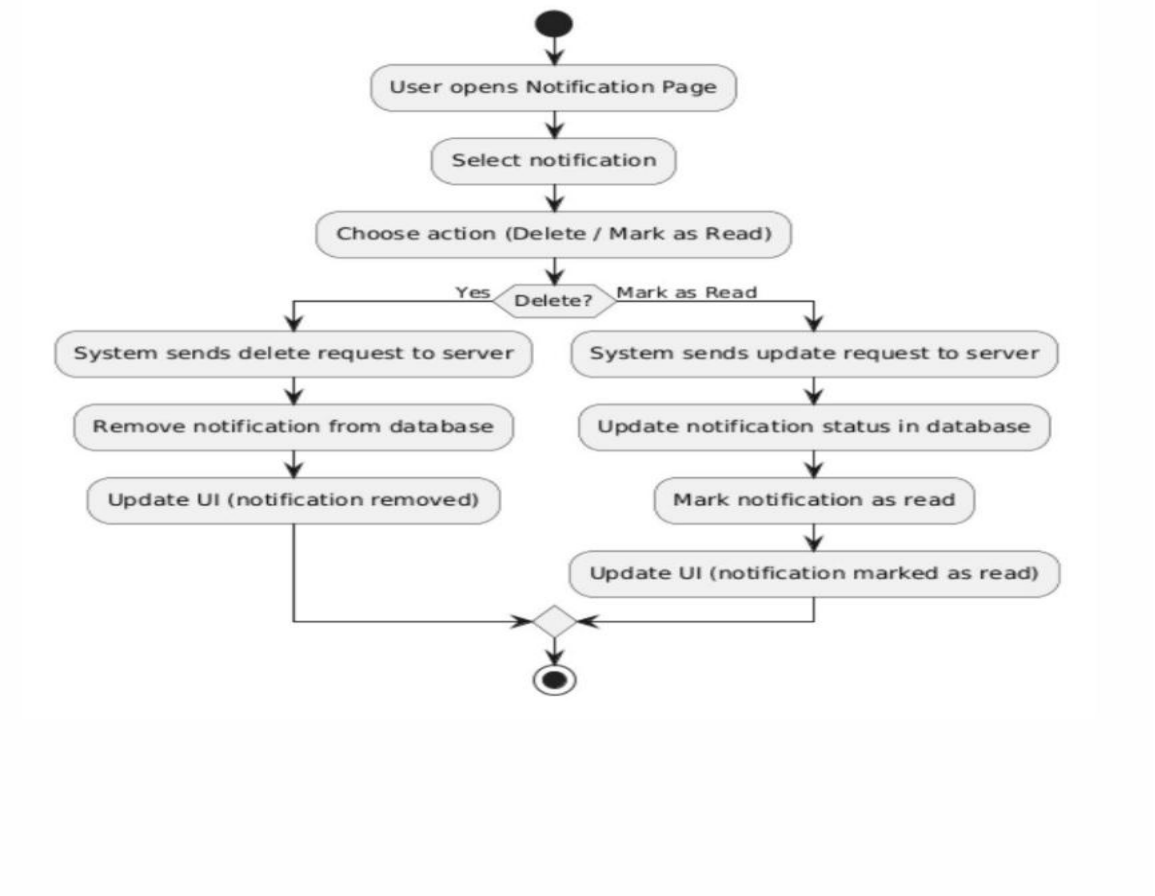


Figure 2.29: Activity Diagram Delete Notification or Mark as Read

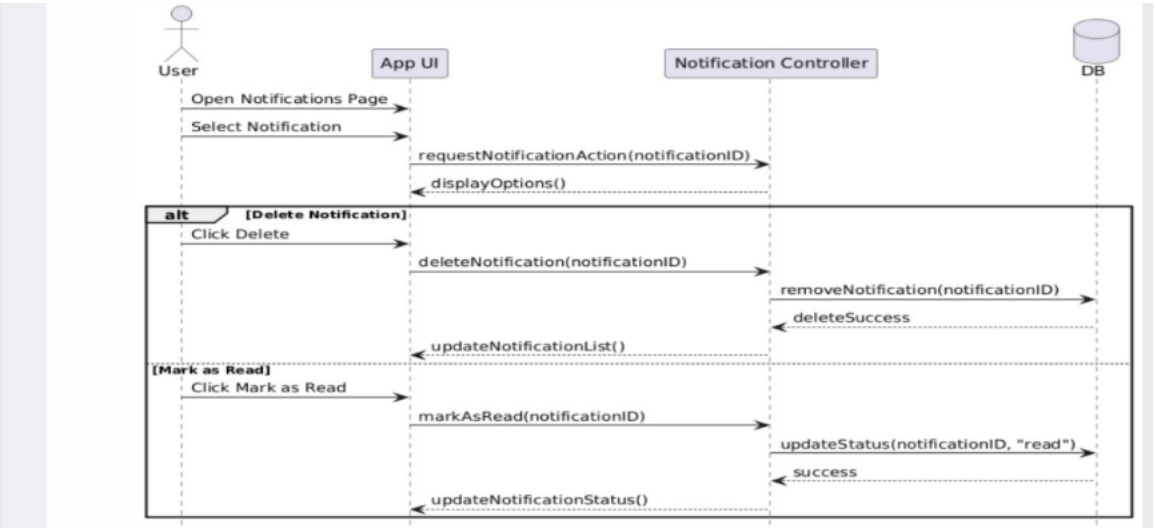


Figure 2.30: Use Case Diagram Delete Notification or Mark as Read

Table 2.14: Use Case Diagram Upload and Optimize Images

Use Case Name/ID	Upload and Optimize Images UC_31
Actors	User
Description	This use case allows the user to upload the images , and the system automatically optimizes them and support multiple format.
Precondition	1. The user must be logged into the system. 2. The image file must be selected .
Main flow	1. The user selects an image , clicks Upload. 2. The system sends the image to the server , validates the format , optimizes and store the image . 3. The system updates the UI.
Alternative flow	A2 |: Supported Format : The system process and optimize the images. A2 : Unsupported Image : The system displays “Unsupported Format”.
Postconditions	The image is uploaded successfully , stored and optimized for display.

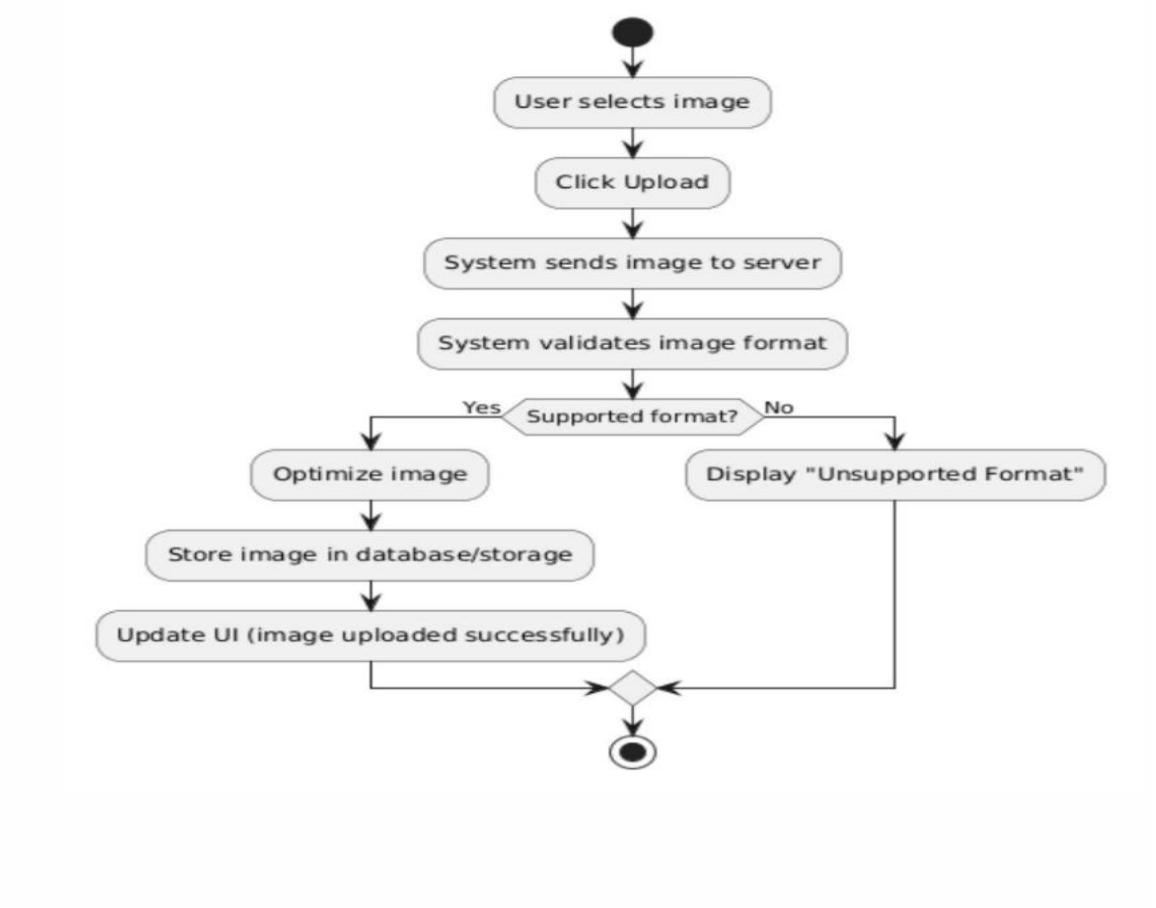


Figure 2.31: Activity Diagram Upload and Optimize |Images

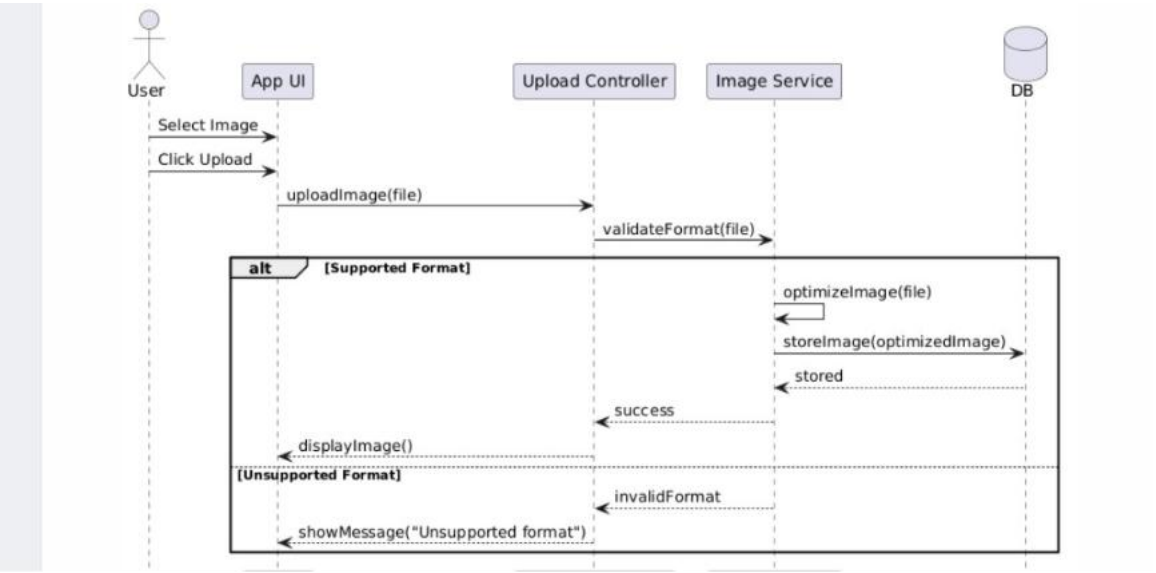


Figure 2.32: Sequence Diagram Upload and Optimize |Images

Table 2.15: Use Case Diagram Protect Media Links From Unauthorized Access

Use Case Name/ID	Protect Media Links From Unauthorized Access UC_32
Actors	User
Description	This use case ensures that media files (images, videos) are accessible only to authorized users by validating access permissions before allowing access.
Precondition	1. The user must be logged into the system. 2. The image file must be selected . 3. The user must have permission to access the media.
Main flow	1. The user requests a media file (image/video). 2. The system sends the request to the serve ,validates the user's authentication token, checks access permissions and allows access to the media.
Alternative flow	A2: Authorized Access :The system returns the media file. A2: Unauthorized Access :The system denies access and shows an error message.
Postconditions	Authorized users can access media , Unauthorized users are denied access. Access attempts may be logged for security.

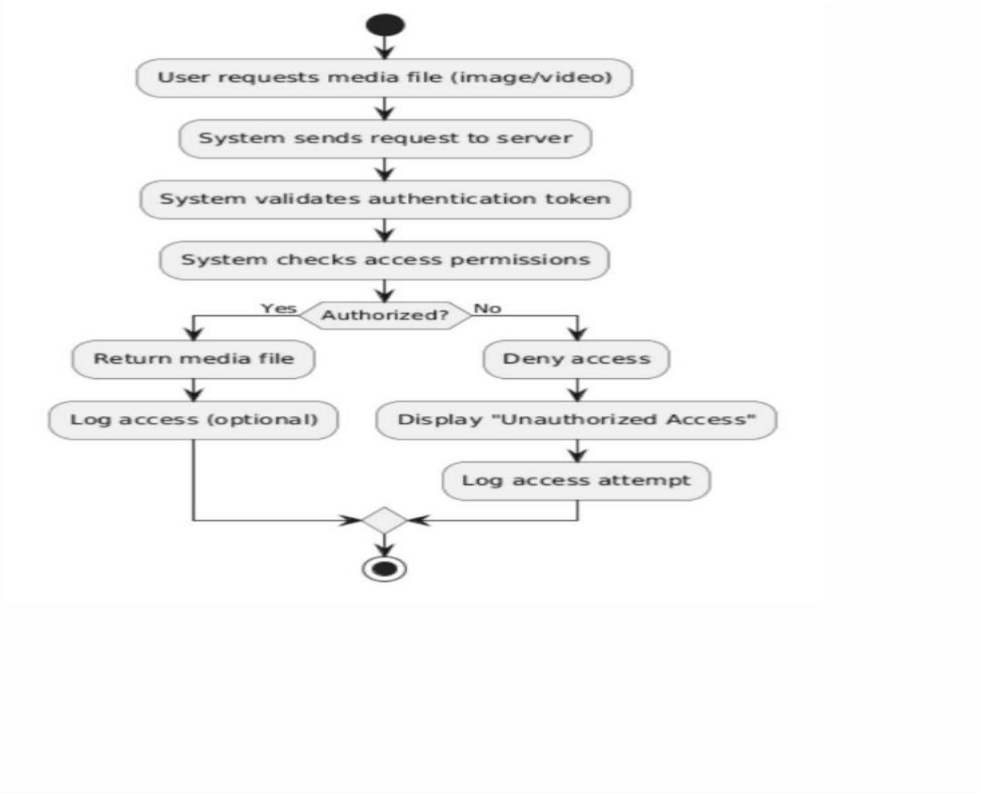


Figure 2.33: Activity Diagram Protect Media Links From Unauthorized Access

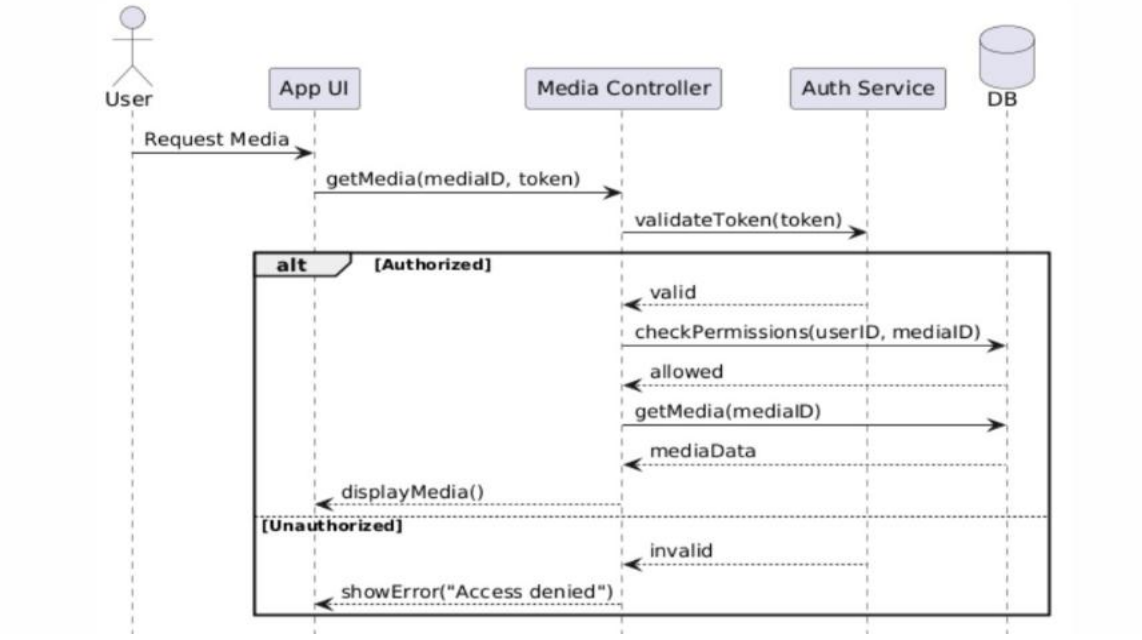


Figure 2.34: Sequence Diagram Protect Media Links From Unauthorized Access

Table 2.16: Use Case Diagram Log Errors and System Activities

Use Case Name/ID	Log Errors and System Activities UC_33
Actors	User
Description	This use case allows the system to log errors and system activities for monitoring, debugging, and auditing purposes.
Precondition	1. The system is running. 2. Logging service is available.
Main flow	1. A system event or user action occurs. 2. The system processes the request , detects an event (error or activity) , sends log data to the logging service. 3. The logging service stores the log in the database.
Alternative flow	A3 |: Activity Log : The system records normal system activity. A3 : Error Log : The system records error details.
Postconditions	Errors and activities are recorded in the system logs and Logs can be reviewed for debugging or auditing.

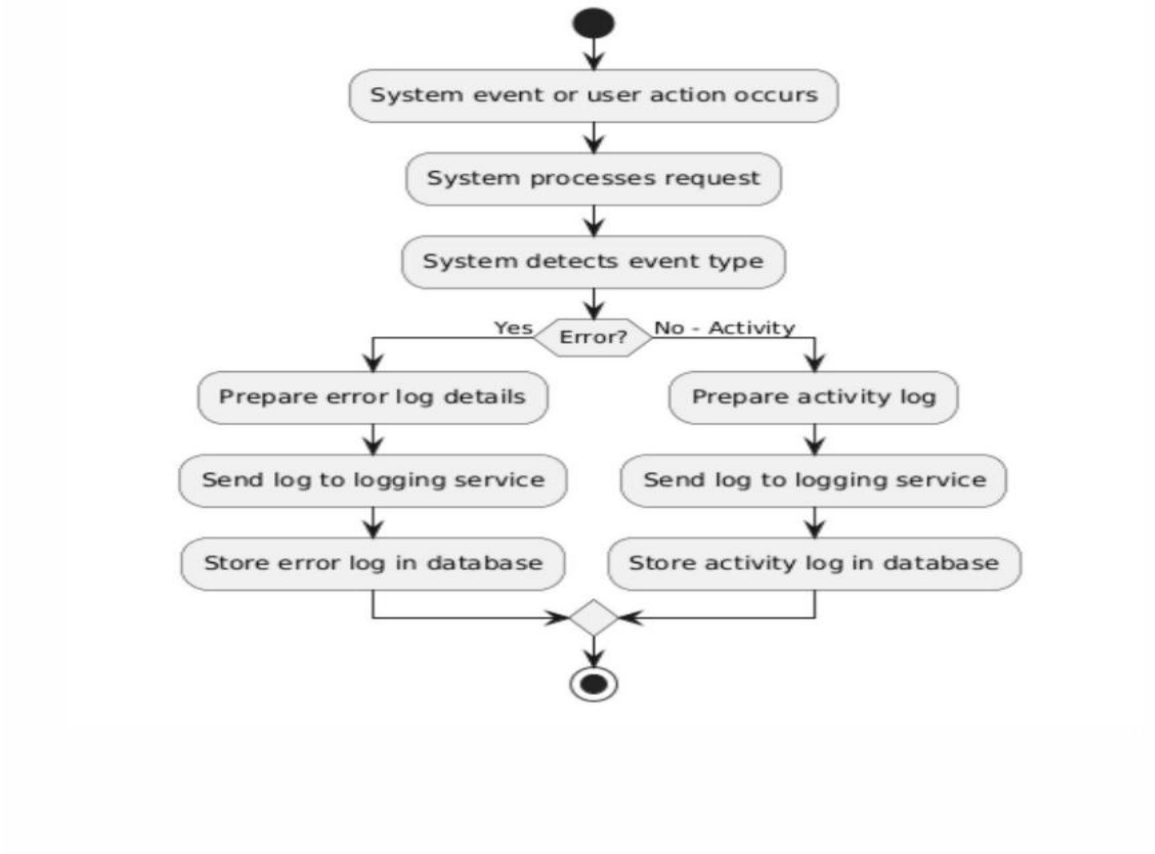


Figure 2.35: Activity Diagram Log Errors and System Activities

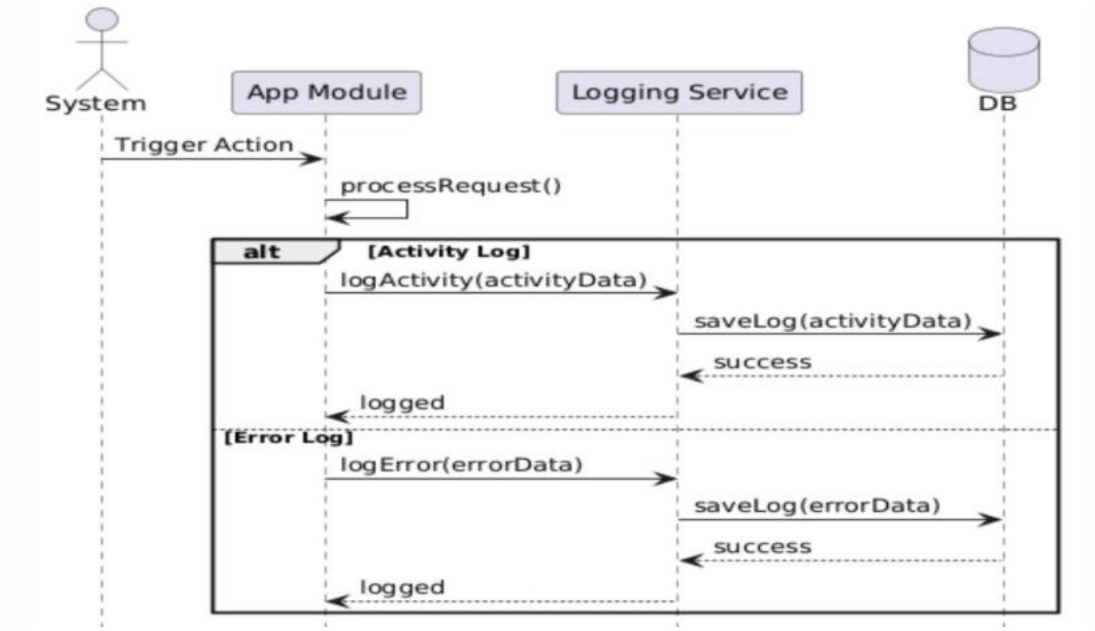


Figure 2.36: Sequence Diagram Log Errors and System Activities

Table 2.17: Use Case Diagram *Execute Scheduled Tasks*

Use Case Name/ID	Execute Scheduled Tasks UC_34
Actors	User
Description	This use case allows the system to automatically execute scheduled tasks at predefined times (e.g., deleting expired data, sending reminders, updating status.
Precondition	1. The system is running. 2. The scheduler service is active. 3. Scheduled tasks are predefined.
Main flow	1. The scheduler triggers at a predefined time. 2. The system identifies the scheduled task, executes the task , updates the database and logs the execution result.
Alternative flow	A2 : Task Executed Successfully : The system completes the task and logs success. A2: No Task to Execute : The system does nothing and logs "No task available".
Postconditions	The scheduled task is executed successfully. The system data is updated accordingly and logs the execution result.

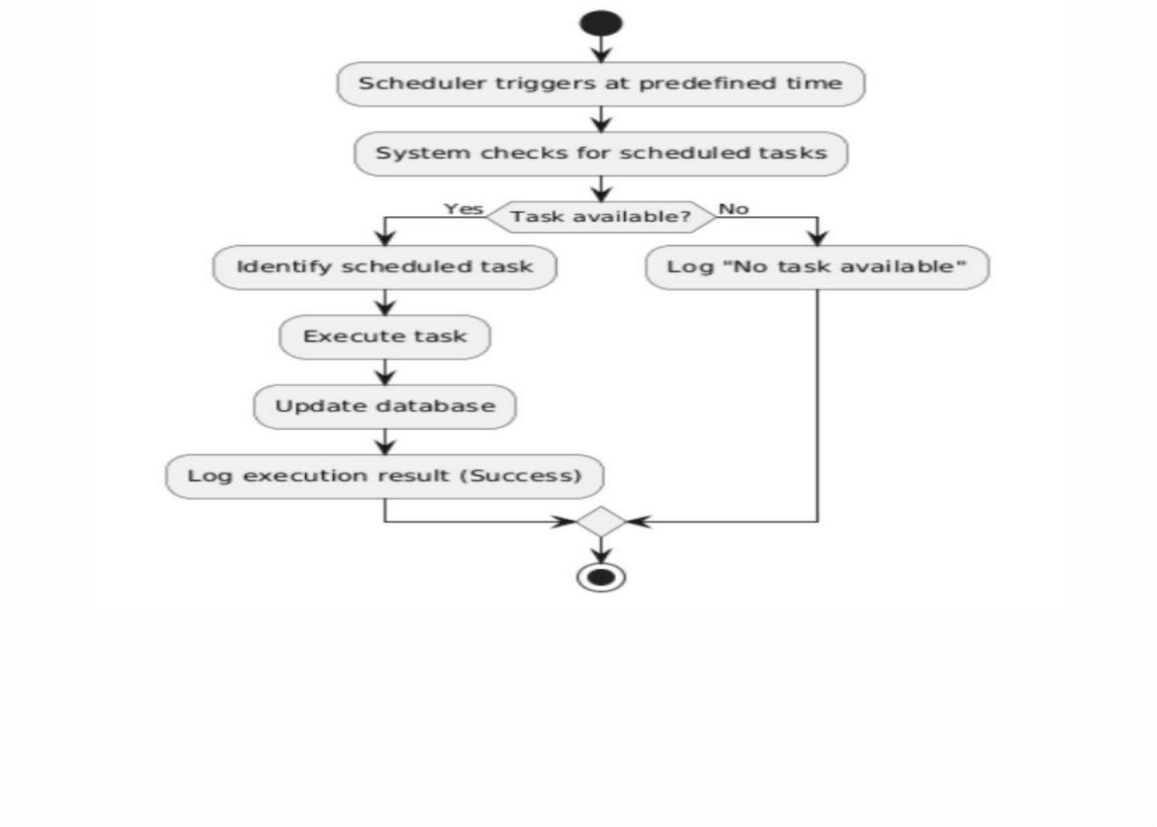


Figure 2.37: Activity Diagram *Execute Scheduled Tasks*

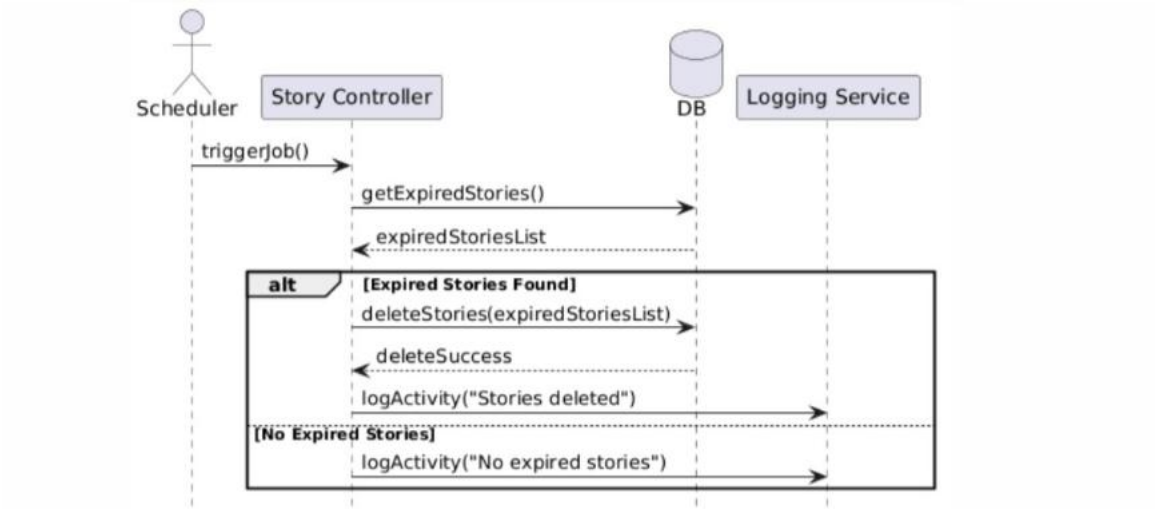


Figure 2.38: Sequence Diagram *Execute Scheduled Tasks*

Table 2.18: Use Case Diagram *Report Inappropriate or Violating Posts*

Use Case Name/ID	Report Inappropriate or Violating Posts UC_35
Actors	User
Description	This use case allows the user to report posts that violate platform rules (e.g., spam, abuse, inappropriate content).
Precondition	1. The user must be logged into the system. 2. The post must exist.
Main flow	1. The user opens a post , selects Report. 2. The system displays violation options. 3. The user selects a reason and submits. 4. The system sends the report to the server , stores the report in the database and confirms submission.
Alternative flow	A4 : A1: Valid Report : The system saves the report successfully. A4: Invalid Input :The system shows "Invalid report data
Postconditions	The report is saved in the system and the system may flag the post for review.

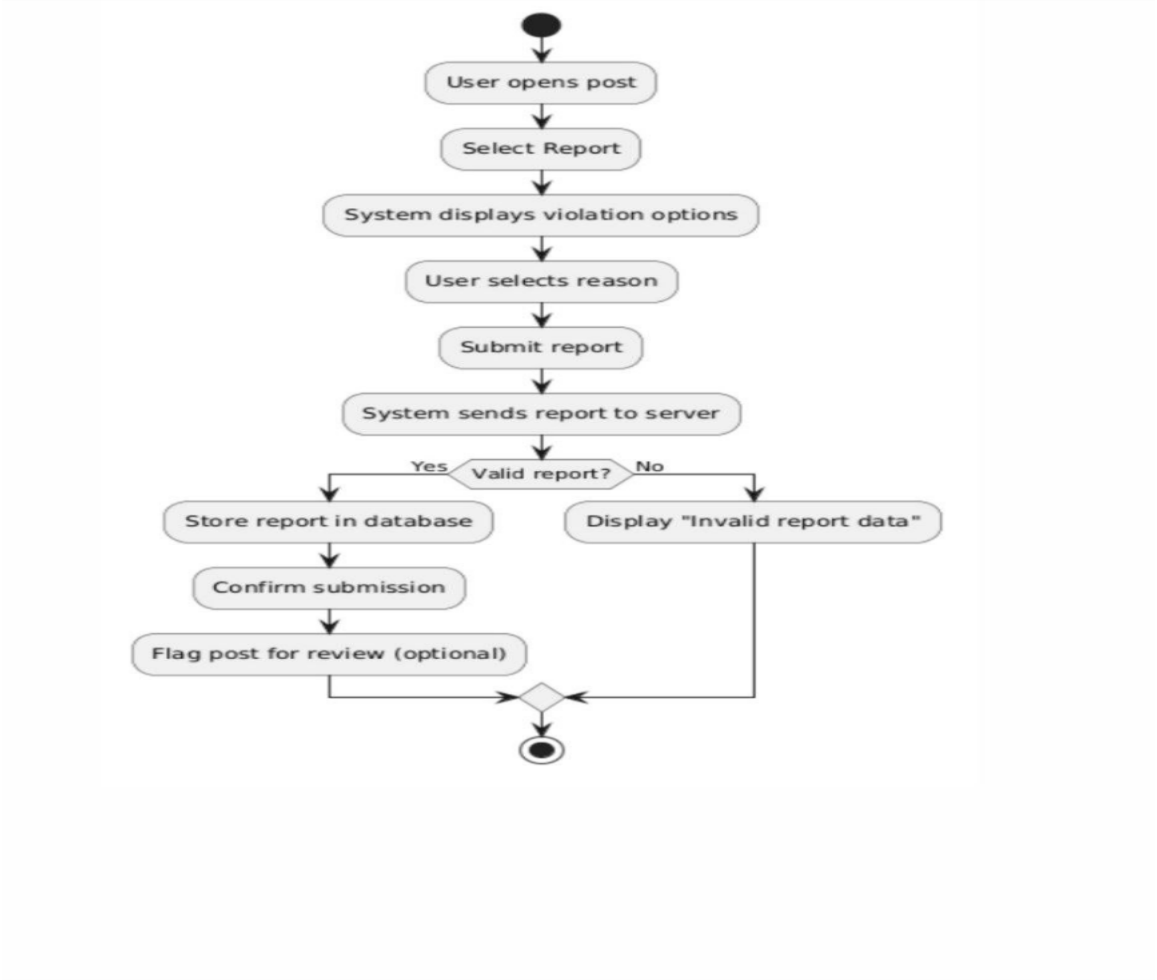


Figure 2.39: Activity Diagram Report Inappropriate or Violating Posts

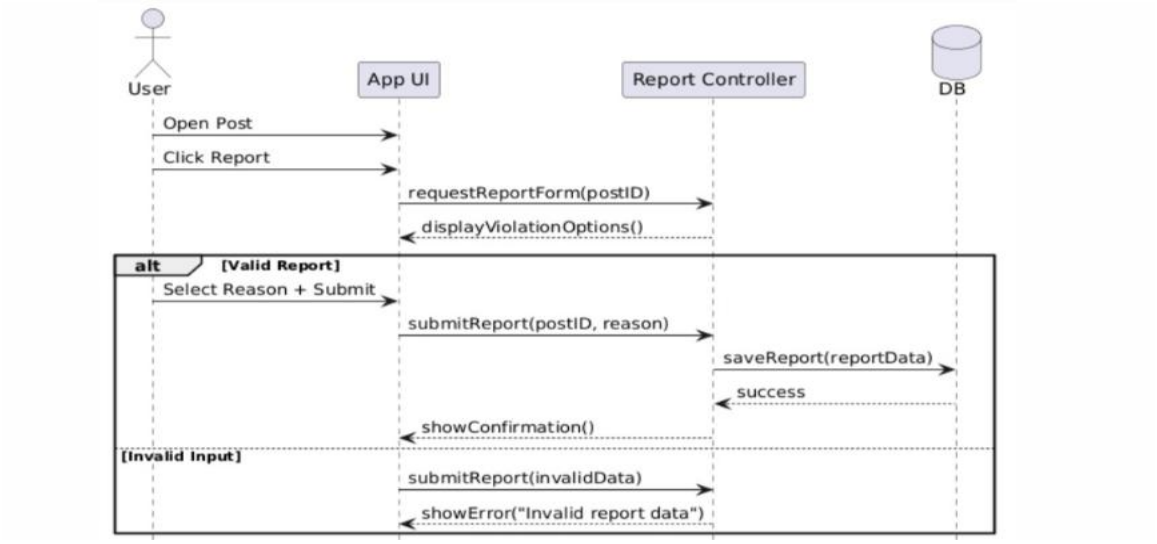


Figure 2.40: Sequence Diagram Report Inappropriate or Violating Posts

Table 2.19: Use Case Diagram Suggest Friends or Groups Based on Shared Interests

Use Case Name/ID	Suggest Friends or Groups Based on Shared Interests UC_36
Actors	User
Description	This use case allows the system to suggest friends or groups to the user based on shared interests, activities, or interactions.
Precondition	1. The user must be logged into the system. 2. User profile and interests must exist.
Main flow	1. The user opens the suggestions page. 2. The system sends a request to the server , retrieves user interests , analyzes similar users/groups , generates recommendations and displays suggestions.
Alternative flow	A2: Suggestions Available The system displays recommended users/groups. A2: No Suggestions The system displays "No suggestions available".
Postconditions	Relevant suggestions are displayed. Suggestions are based on similarity or system recommendations.

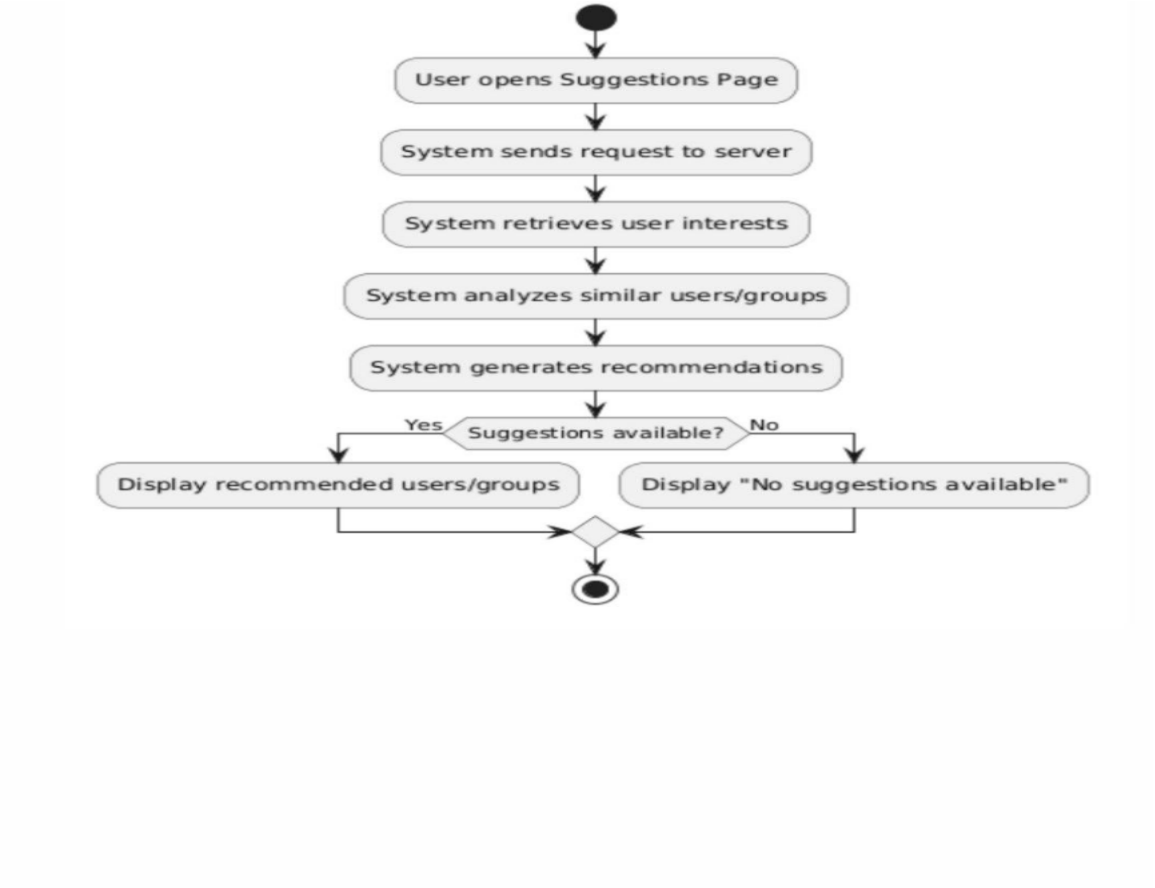


Figure 2.41: Activity Diagram Suggest Friends or Groups Based on Shared Interests

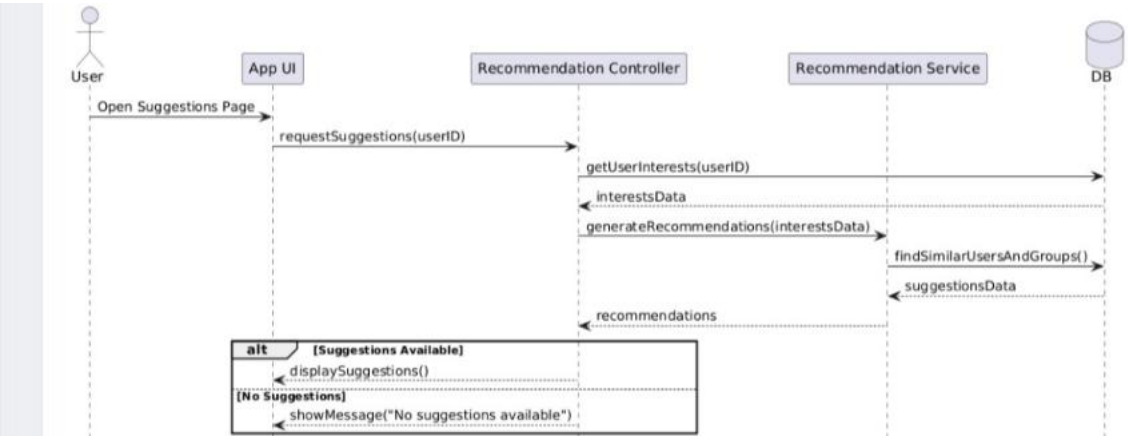


Figure 2.42: Sequence Diagram Suggest Friends or Groups Based on Shared Interests

Table 2.20: Use Case Diagram Discover Trending Content in Real Time

Use Case Name/ID	Discover Trending Content in Real Time UC_37
Actors	User
Description	This use case allows the user to discover trending content in real time based on interactions such as likes, comments, and shares..
Precondition	1. The user must be logged into the system. 2. Content and user interactions must exist. 3. Real-time or analytics service must be active.
Main flow	1. The user opens the Explore / Trending page. 2. The system sends a request to the server, retrieves interaction data (likes, comments, shares). 3. The system analyzes trending content , sends results to the UI and updates content dynamically.
Alternative flow	A2: Trending Content Available The system displays trending posts. A2: No Trending Content The system displays "No trending content".
Postconditions	Trending content is displayed. Content updates dynamically in real time.

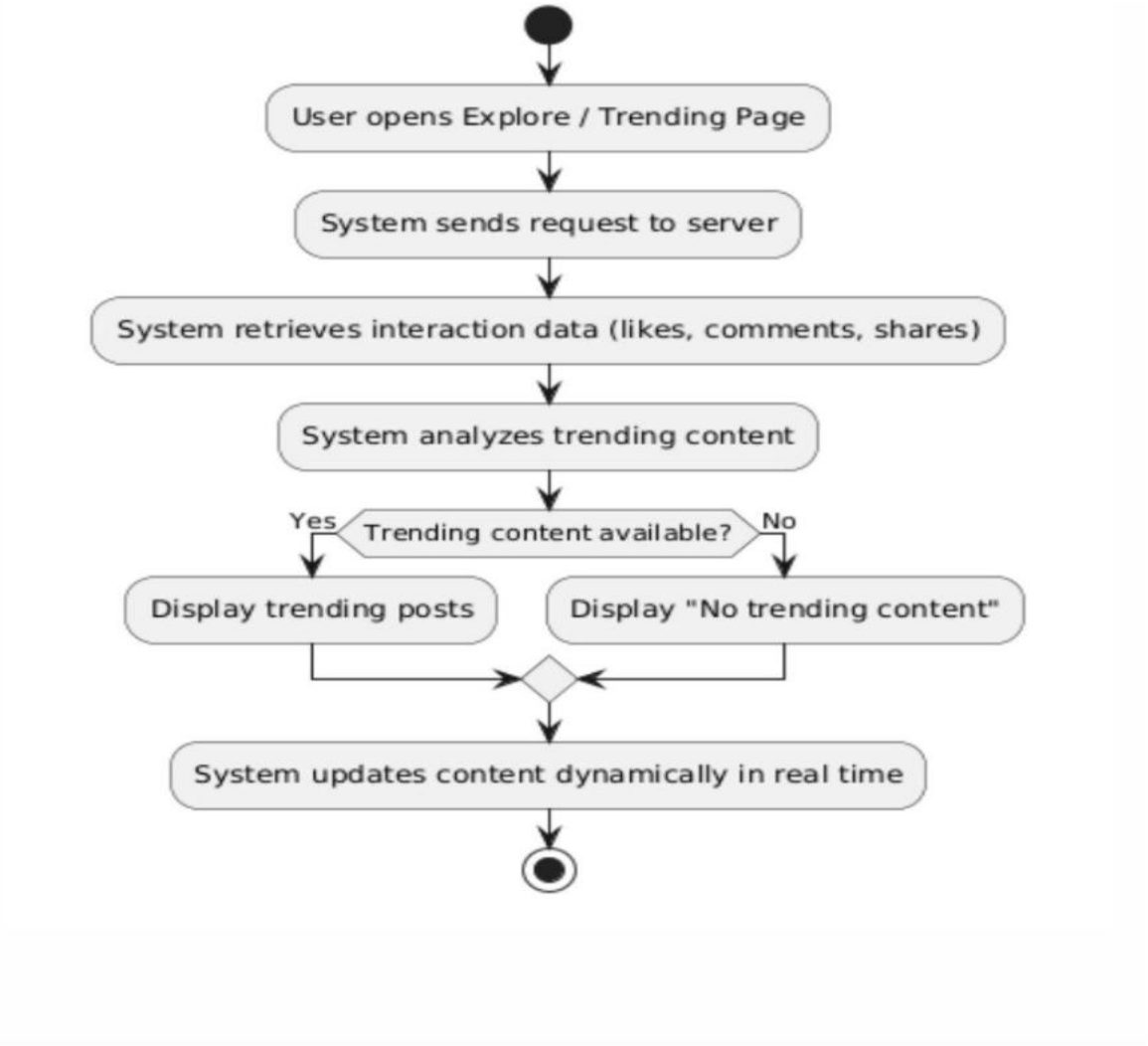


Figure 2.43: Activity Diagram Discover Trending Content in Real Time

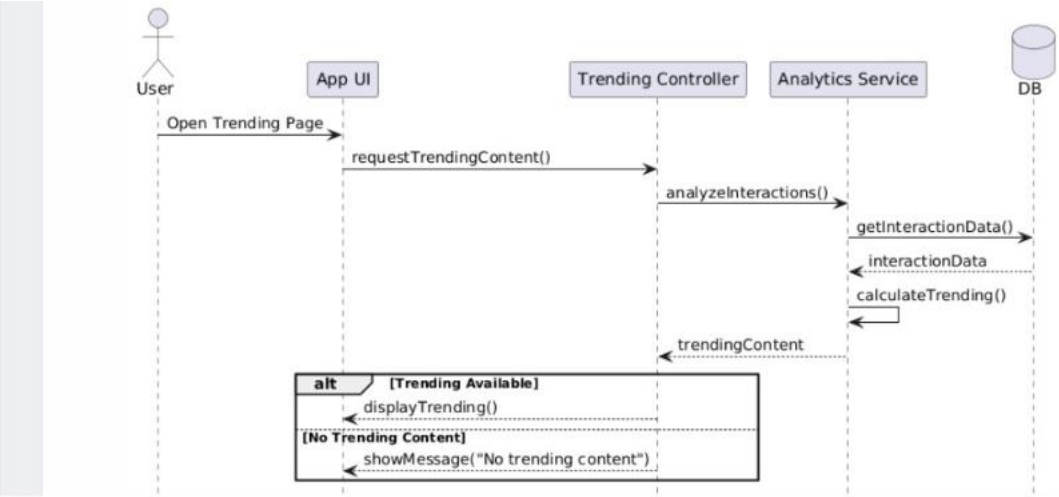


Figure 2.44: Sequence Diagram Discover Trending Content in Real Time

3. Chapter Three: Design Study

3.1- Site Map Diagram

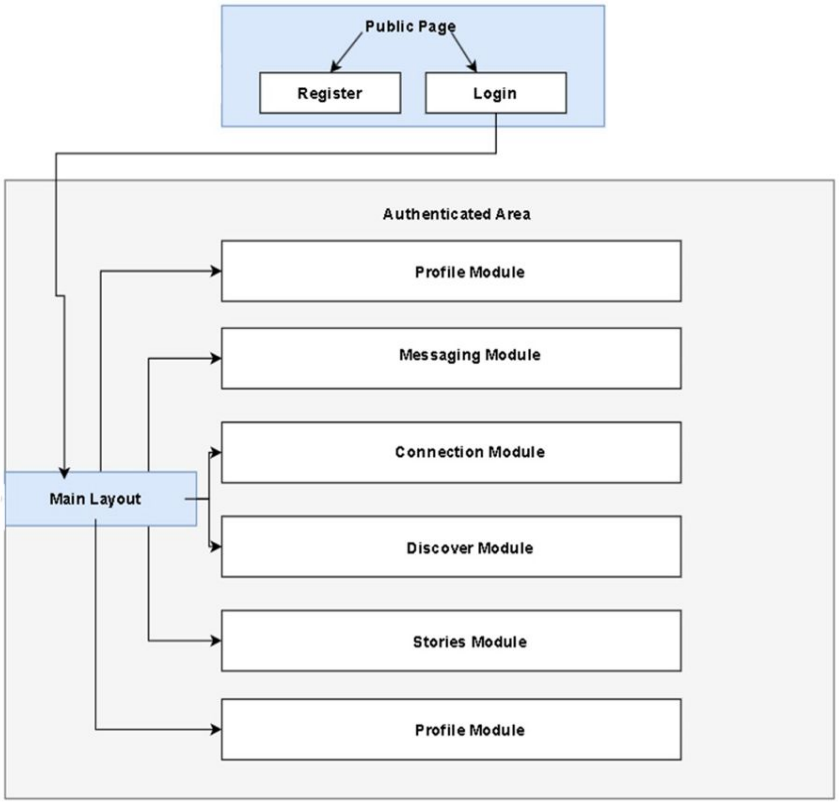


Figure 3.45: Sit Map Diagram

3.2- Database ERD Diagram (Entity-Relationship Diagram)

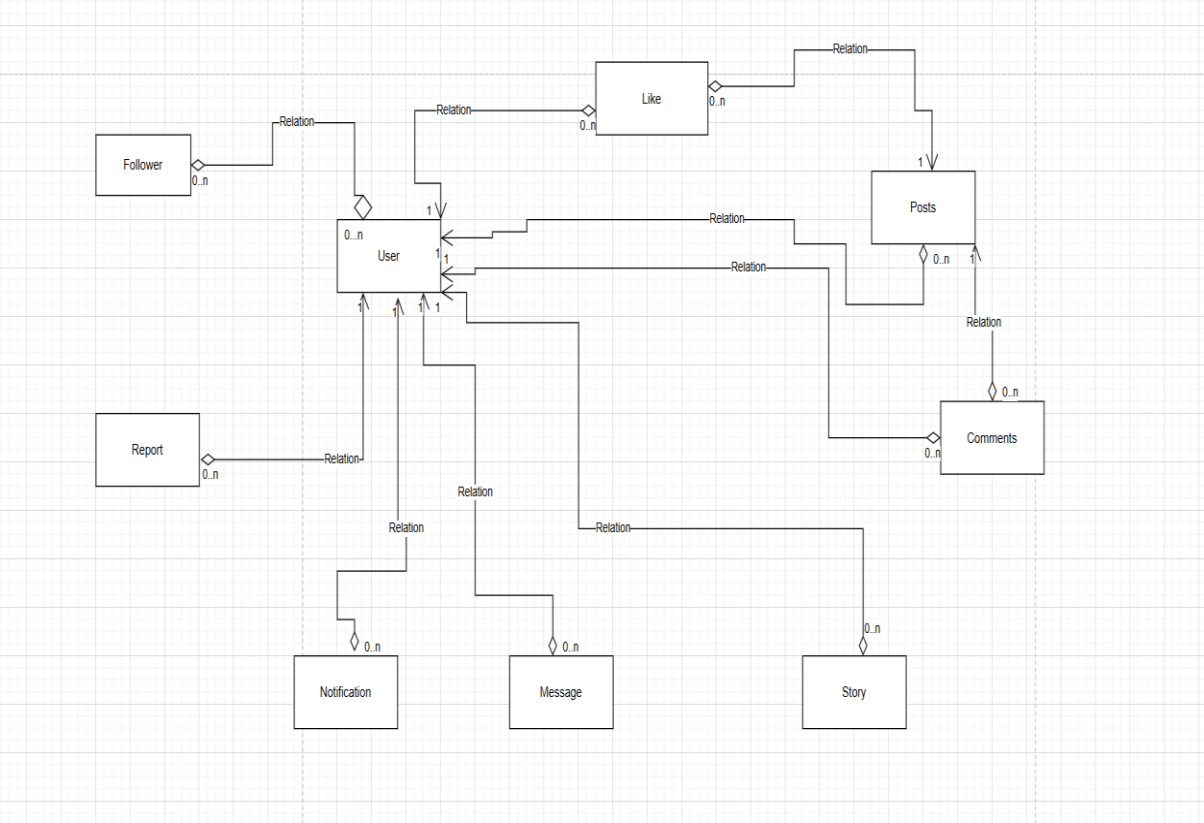


Figure 3.46: Database ERD Diagram (Entity-Relationship Diagram)

3.3- Class Diagram

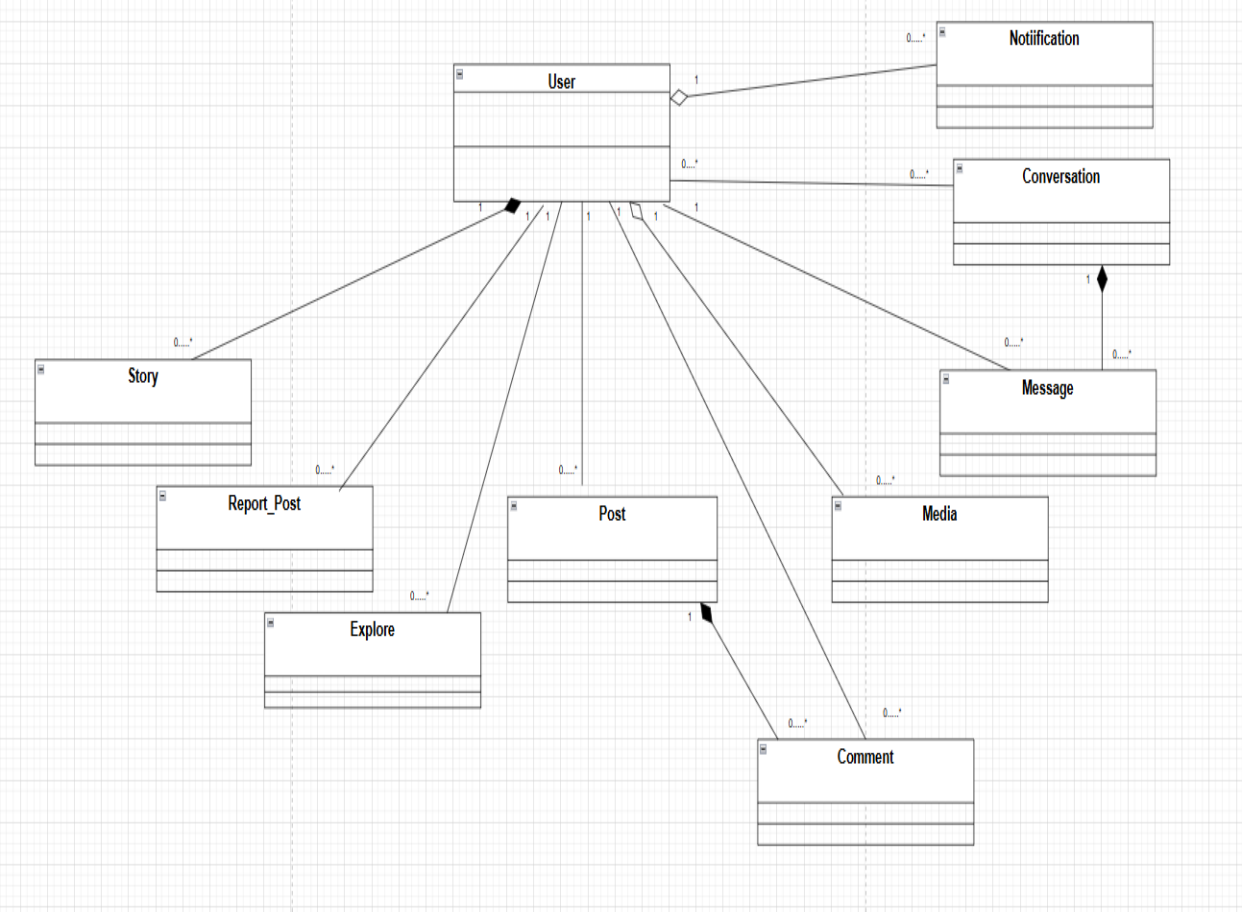


Figure 3.47: Class Diagram

3.4- Test Cases Table

TC-01: Start a new conversation between users

Test Case ID	Title	Pre-Conditions	Test Data	Steps	Expected Result	Post-Conditions
TC-01	Start New Conversation with Valid User	User A logged in, User B exists, messaging enabled	Receiver: userB; Message: Hello	1) Open Messages 2) Click New Message 3) Search userB 4) Select user 5) Enter message 6) Submit	New conversation created successfully and first message displayed	New conversation stored in DB
TC-02	Start Conversation with Non-Existing User	User A logged in	Receiver: invalidUser	1) Open New Message 2) Search invalidUser 3) Try select/send	System shows 'user not found' and conversation is not created	No new conversation created
TC-03	Start Conversation with Empty First Message	User A logged in, User B exists	Receiver: userB; Message: empty	1) Open New Message 2) Select userB 3) Leave message empty 4) Submit	Validation appears and sending is rejected	No conversation/message saved
TC-04	Start Conversation with Blocked User	User A logged in, target blocked	Receiver: userB	1) Open New Message 2) Search blocked user 3) Try to send	System prevents conversation creation	No new chat created

TC-02: Send and receive text messages and emojis

Test Case ID	Title	Pre-Conditions	Test Data	Steps	Expected Result	Post-Conditions
TC-05	Send Valid Text Message	Conversation exists, sender logged in	Hi	1) Open chat 2) Enter text 3) Send	Message sent and displayed correctly	Message saved in conversation
TC-06	Send Emoji Message	Conversation exists	□	1) Open chat 2) Select emoji 3) Send	Emoji appears correctly for sender and receiver	Emoji message saved
TC-07	Send Mixed Text and Emoji	Conversation exists	Hi □	1) Enter mixed content 2) Send	Mixed message displayed correctly	Message saved
TC-08	Send Empty Message	Conversation exists	Empty	1) Open chat 2) Leave field empty 3) Send	Validation shown and message rejected	No message saved

TC-03: Support real-time messaging

Test Case ID	Title	Pre-Conditions	Test Data	Steps	Expected Result	Post-Conditions
TC-09	Receive Message in Real Time	User A and User B online on separate devices	Real-time test	1) A opens chat 2) B opens app/chat 3) A sends message	B receives message instantly without refresh	Message synced in both sessions
TC-10	Real-Time Update in Active Chat	Same conversation open for both users	test live	1) A sends message 2) Observe B screen	Message appears immediately in active chat	Chat updated live

Test Case ID	Title	Pre-Conditions	Test Data	Steps	Expected Result	Post-Conditions
TC-11	Real-Time Notification When Receiver Outside Chat	Receiver logged in but on another page	Ping	1) A sends message 2) B stays on Home page	Notification/unread badge appears immediately	Unread count updated
TC-12	Recover Messages After Temporary Disconnect	B connection drops temporarily	Reconnect message	1) Disconnect B 2) A sends message 3) Reconnect B	Message appears after reconnection and sync is restored	Message synchronized

TC-04: Display message status (Sent - Delivered - Read)

Test Case ID	Title	Pre-Conditions	Test Data	Steps	Expected Result	Post-Conditions
TC-13	Show Sent Status	Sender online, receiver may be offline	Status test	1) Send message	Initial status shown as Sent	Status stored
TC-14	Show Delivered Status	Receiver connected to service	Status test	1) Send message 2) Wait for delivery	Status changes to Delivered	Delivery event logged
TC-15	Show Read Status	Receiver opens conversation	Status test	1) Receiver opens chat after delivery	Status changes to Read	Read event saved
TC-16	Status Remains Delivered if Not Opened	Receiver online but does not open chat	Hello	1) Send message 2) Receiver does not open chat	Status stays Delivered, not Read	Read status not triggered

TC-05: Delete or mute conversations

Test Case ID	Title	Pre-Conditions	Test Data	Steps	Expected Result	Post-Conditions
TC-17	Delete Existing Conversation	Conversation exists	selected chat	1) Open messages 2) Select conversation 3) Delete 4) Confirm	Conversation removed from list	Conversation hidden/deleted per design
TC-18	Cancel Delete Conversation	Conversation exists	selected chat	1) Delete conversation 2) Cancel confirmation	Conversation remains unchanged	No delete action performed
TC-19	Mute Conversation	Conversation exists	selected chat	1) Open chat settings 2) Mute	Notifications for this chat are disabled	Mute state saved
TC-20	Unmute Conversation	Conversation already muted	selected chat	1) Open settings 2) Unmute	Notifications enabled again	Mute state removed

TC-06: Block or unblock users

Test Case ID	Title	Pre-Conditions	Test Data	Steps	Expected Result	Post-Conditions
TC-25	Block User TCom Profile	User logged in, target exists	target=userB	1) Open profile 2) Click Block 3) Confirm	Target user blocked successfully	Block record saved
TC-26	Block User TCom Chat	Conversation exists	target=userB	1) Open chat 2) More options 3) Block	User blocked and chat interaction restricted	Block state saved
TC-27	Unblock User	User is already blocked	target=userB	1) Open blocked list 2) Click Unblock	User removed from blocked list	Block removed
TC-28	Verify Blocked User Cannot Send Message	Target blocked	target=userB	1) Block user 2) Attempt messaging between users	Messaging is prevented according to rules	No message created

TC-07: Display online status (Online / Offline)

Test Case ID	Title	Pre-Conditions	Test Data	Steps	Expected Result	Post-Conditions
TC-29	Show Online Status	Target user logged in	target=userB	1) Open chat/profile	Status displayed as Online	Presence data visible
TC-30	Show Offline Status	Target user logged out	target=userB	1) Open chat/profile	Status displayed as Offline	Presence data visible
TC-31	Update Status on Login	User B currently offline	target=userB	1) Observe Offline 2) B logs in	Status changes to Online	Presence updated
TC-32	Update Status on Logout	User B currently online	target=userB	1) Observe Online 2) B logs out	Status changes to Offline	Presence updated

TC-08: Search for posts using hashtags

Test Case ID	Title	Pre-Conditions	Test Data	Steps	Expected Result	Post-Conditions
TC-33	Search Existing Hashtag	Posts with hashtag exist	#travel	1) Open search 2) Enter #travel 3) Search	Relevant posts displayed	Search results returned
TC-34	Search Non-Existing Hashtag	Logged in	#xyz123notfound	1) Search hashtag	No results message displayed	No records returned
TC-35	Search Without Hash Sign	Posts may exist	travel	1) Search text without #	Results handled according to design	Search behavior logged
TC-36	Search Hashtag with Special Characters	Logged in	#tr@vel	1) Enter invalid hashtag 2) Search	Validation or empty/filtered results shown	Invalid query handled

TC-09: View the Explore page containing trending posts and suggested users

Test Case ID	Title	Pre-Conditions	Test Data	Steps	Expected Result	Post-Conditions
TC-37	Open Explore Page Successfully	User logged in	-	1) Navigate to Explore	Explore page loads correctly	Page content displayed
TC-38	Show Trending Posts	Trending data available	-	1) Open Explore	Trending posts shown	Trending list rendered
TC-39	Show Suggested Users	Recommendation data available	-	1) Open Explore	Suggested users shown	Suggestions visible
TC-40	Handle Empty Explore Data	No trending/suggestions available	-	1) Open Explore	Empty state displayed gracefully	No crash occurs

TC-10: Receive real-time notifications for interactions and new followers

Test Case ID	Title	Pre-Conditions	Test Data	Steps	Expected Result	Post-Conditions
TC-41	Receive Like Notification	User A has post, User B logged in	Like event	1) B likes A's post	A receives real-time notification	Notification stored
TC-42	Receive Comment Notification	Post exists	Comment=Nice!	1) B comments on A's post	A receives comment notification instantly	Notification saved
TC-43	Receive New Follower Notification	User B follows A	Follow event	1) B follows A	A receives follower notification	Notification saved
TC-44	Avoid Duplicate Notification Handling	Same action repeated	repeated like/follow	1) Trigger repeated event	System handles duplicates according to design	No unexpected duplicates

TC-11: View all notifications on a dedicated page

Test Case ID	Title	Pre-Conditions	Test Data	Steps	Expected Result	Post-Conditions
TC-45	Open Notifications Page	Notifications exist	-	1) Navigate to Notifications	Notification page opens correctly	Notifications listed
TC-46	Display Notifications in Correct Order	Multiple notifications exist	-	1) Open page	Notifications shown in expected order (e.g. newest first)	Ordering confirmed
TC-47	Distinguish Unread Notifications	Unread items exist	-	1) Open page	Unread notifications visually marked	Unread state visible

TC-12: Delete notifications or mark them as read

Test Case ID	Title	Pre-Conditions	Test Data	Steps	Expected Result	Post-Conditions
TC-48	Delete Single Notification	Notification exists	selected notification	1) Open notifications 2) Delete item	Notification removed	Item no longer visible
TC-49	Mark One Notification as Read	Unread notification exists	selected notification	1) Open notifications 2) Mark as read	Notification state changes to read	Read state saved
TC-50	Mark All Notifications as Read	Multiple unread items exist	all unread items	1) Click Mark All as Read	All unread notifications become read	Unread count reset
TC-51	Delete Multiple Notifications	Several notifications exist	selected set	1) Select multiple 2) Delete	Selected notifications deleted	List updated

TC-13: Upload images and automatically optimize them

Test Case ID	Title	Pre-Conditions	Test Data	Steps	Expected Result	Post-Conditions
TC-52	Upload JPG Image	User logged in	file.jpg	1) Open upload 2) Select JPG 3) Submit	Image uploaded successfully	Media record created
TC-53	Upload PNG Image	User logged in	file.png	1) Upload PNG	Upload succeeds	Media stored
TC-54	Upload Unsupported File Type	User logged in	file.exe	1) Attempt upload	Validation rejects unsupported format	No media created
TC-55	Verify Image Optimization	User logged in	large image file	1) Upload large valid image	Image uploaded and optimized/compressed	Optimized version stored
TC-56	Upload Very Large Image	User logged in	oversized file	1) Attempt upload	System handles limit correctly with message	No invalid upload stored

TC-14: Protect media links From unauthorized access

Test Case ID	Title	Pre-Conditions	Test Data	Steps	Expected Result	Post-Conditions
TC-57	Authorized User Opens Media Link	User has access rights	valid media URL	1) Login as authorized user 2) Open link	Media displayed successfully	Access logged
TC-58	Unauthorized User Opens Media Link	User lacks permission	same URL	1) Open URL as unauthorized user	Access denied / 401 or protected response	No unauthorized access granted
TC-59	Open Media Link After Logout	Protected media exists	direct media URL	1) Logout 2) Open copied URL	Access denied or redirected	No media shown

TC-15: Log errors and user activities

Test Case ID	Title	Pre-Conditions	Test Data	Steps	Expected Result	Post-Conditions
TC-60	Log Successful Login Activity	Logging enabled	valid login	1) User logs in	Login action recorded in logs	Log entry created
TC-61	Log Content Creation Activity	Logging enabled	create post/upload	1) Perform action	Activity recorded correctly	Log entry created
TC-62	Log System Error	Logging enabled	invalid operation	1) Trigger controlled error	Error details recorded	Error log created
TC-63	Log Failed Login Attempt	Logging enabled	wrong password	1) Attempt invalid login	Failed attempt recorded	Security log updated

TC-16: Execute scheduled tasks such as deleting expired stories

Test Case ID	Title	Pre-Conditions	Test Data	Steps	Expected Result	Post-Conditions
TC-64	Delete Expired Story Automatically	Story exists with expiration time	expired story	1) Wait/trigger scheduler after expiry	Expired story deleted automatically	Story removed
TC-65	Keep Non-Expired Story	Active story exists	non-expired story	1) Run scheduler before expiry	Story remains available	Story unchanged
TC-66	Delete Multiple Expired Stories	Several expired stories exist	multiple records	1) Run scheduler	All expired stories removed successfully	Records cleaned
TC-67	Scheduler Runs Without Errors	Scheduled job configured	-	1) Execute task 2) Observe logs/results	Job completes successfully without crash	Task execution logged

TC-17 Report inappropriate or violating posts

Test Case ID	Title	Pre-Conditions	Test Data	Steps	Expected Result	Post-Conditions
TC-68	Report Post with Valid Reason	User logged in, post exists	Reason: inappropriate content	1) Open post 2) Click Report 3) Choose reason 4) Submit	Report submitted successfully	Report record created
TC-69	Report Without Selecting Reason	Post exists	empty reason	1) Open report form 2) Submit without reason	Validation message shown	No report created
TC-70	Report Same Post More Than Once	Prior report may exist	same post	1) Submit report twice	System handles duplicate reports according to design	Duplicate handling applied

TC-18:Suggest Friends or groups based on shared interests

Test Case ID	Title	Pre-Conditions	Test Data	Steps	Expected Result	Post-Conditions
TC-71	Show Friend Suggestions Based on Interests	User profile/interests available	shared interests	1) Login 2) Open suggestions section	Relevant Friend suggestions displayed	Recommendation list generated
TC-72	Show Group Suggestions Based on Interests	User activity/interests exist	shared interests	1) Open recommendations	Relevant groups displayed	Recommendation list generated
TC-73	Update Suggestions After User Interest Changes	User changes interests or follows new topics	updated interests	1) Modify interests 2) Refresh suggestions	Suggestions update accordingly	New recommendation state generated

TC-19: Discover trending content in real time

Test Case ID	Title	Pre-Conditions	Test Data	Steps	Expected Result	Post-Conditions
TC-74	View Current Trending Content	Trending engine active	-	1) Open Trending section	Current trending content displayed	List rendered
TC-75	Trending Content Updates Dynamically	Popularity changes in system	new trending item	1) Keep Trending page open 2) Trigger/populate trending changes	Trending list updates in near real time	Updated ranking visible
TC-76	Handle Empty Trending State	No trending data available	-	1) Open Trending section	Empty state shown correctly	No crash/errors

3.5- Requirements Traceability Matrix (RTM)

Req ID	Requirement Description	Use Case	Type	Design Description	Test Case ID
ID-01	Start a new conversation between users	UC-01	Functional	User selects a user and initiates chat	TC-01
ID-02	Send and receive text messages and emojis	UC-02	Functional	Message input and send mechanism	TC-02

ID-03	Support real-time messaging	UC-03	Functional	Live message transmission using sockets	TC-03
ID-04	Display message status (Sent–Delivered–Read)	UC-04	Functional	Track and display message states	TC-04
ID-05	Delete or mute conversations	UC-05	Functional	User can mute or delete conversations	TC-05
ID-06	Block or unblock users	UC-06	Functional	User blocking functionality	TC-06
ID-07	Display online/offline status	UC-07	Functional	Show user availability status	TC-07
ID-08	Search posts using hashtags	UC-08	Functional	Hashtag-based filtering system	TC-08
ID-09	View Explore page (trending content)	UC-09	Functional	Display trending posts dynamically	TC-09
ID-10	Receive real-time notifications	UC-10	Functional	Push notifications for events	TC-10
ID-11	View notifications on a dedicated page	UC-11	Functional	Display all notifications	TC-11
ID-12	Delete notifications or mark as read	UC-12	Functional	Mark or remove notifications	TC-12
ID-13	Upload images with multi-format support	UC-13	Functional	Handle image uploads and formats	TC-13
ID-14	Protect media from unauthorized access	UC-14	Functional	Secure media access system	TC-14
ID-15	Log system activities and errors	UC-15	Functional	Record system actions and errors	TC-15

ID-16	Execute scheduled tasks (delete expired stories)	UC-16	Functional	Auto-delete stories after 24h	TC-16
ID-17	Report inappropriate or violating posts	UC-17	Functional	Report system for posts	TC-17
ID-18	Suggest users/groups based on interests	UC-18	Functional	Suggest users algorithmically	TC-18
ID-19	Discover trending content in real time	UC-19	Functional	Real-time trending detection	TC-19

4. Chapter Four: Project Implementation

4.1- User Interfaces

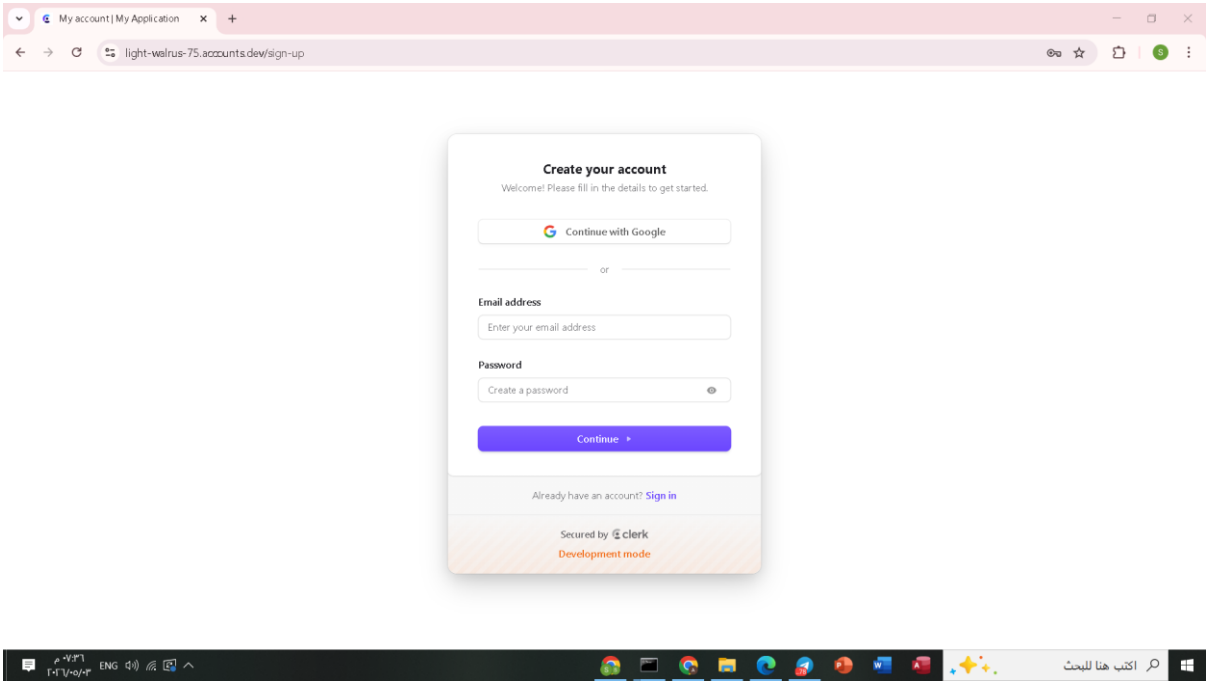


Figure 4.47: Register Page

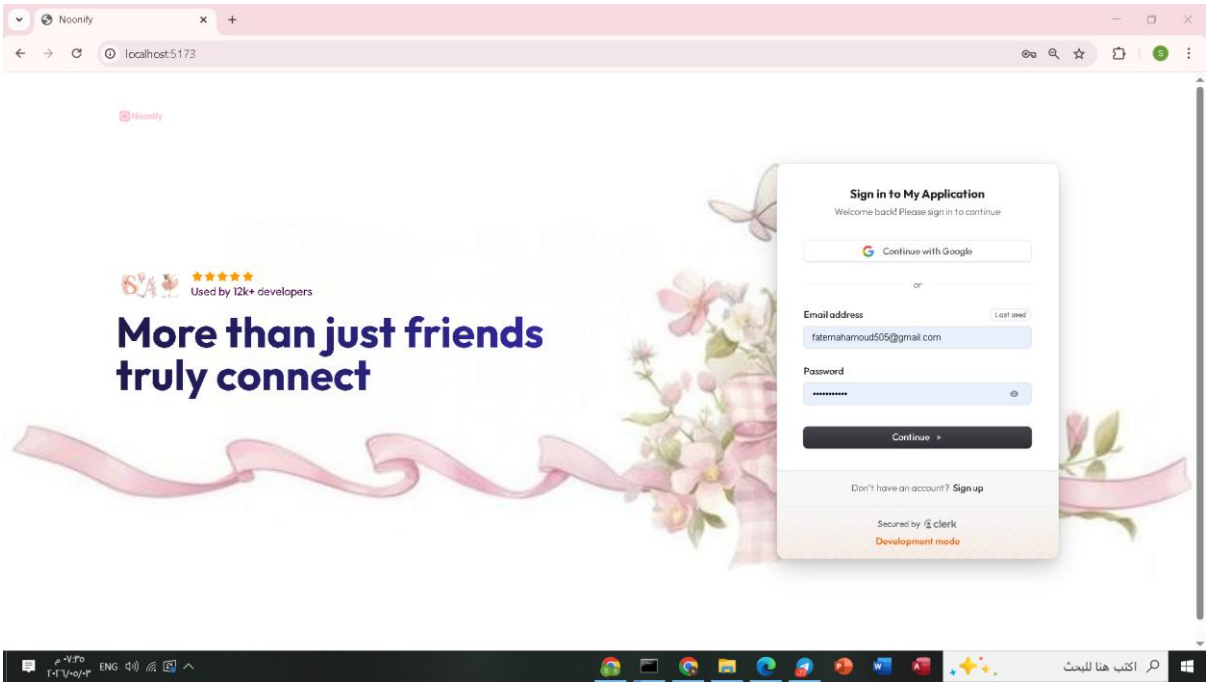


Figure 4.48: Login Page

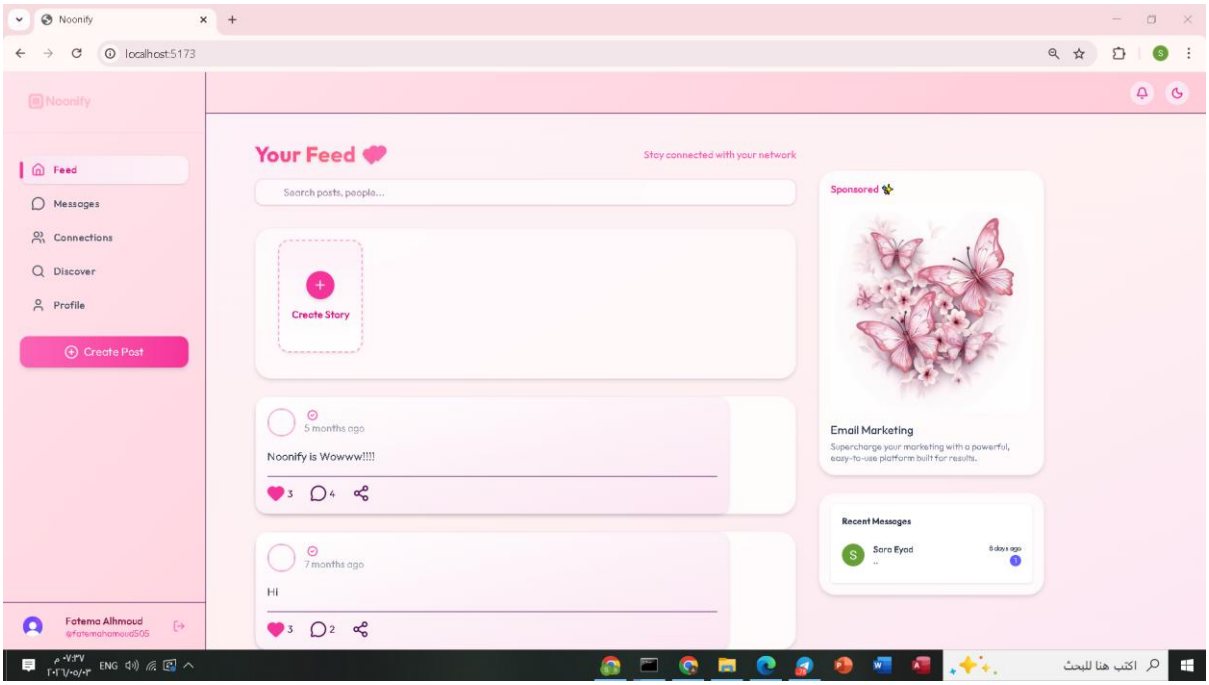


Figure 4.48: Feed Page

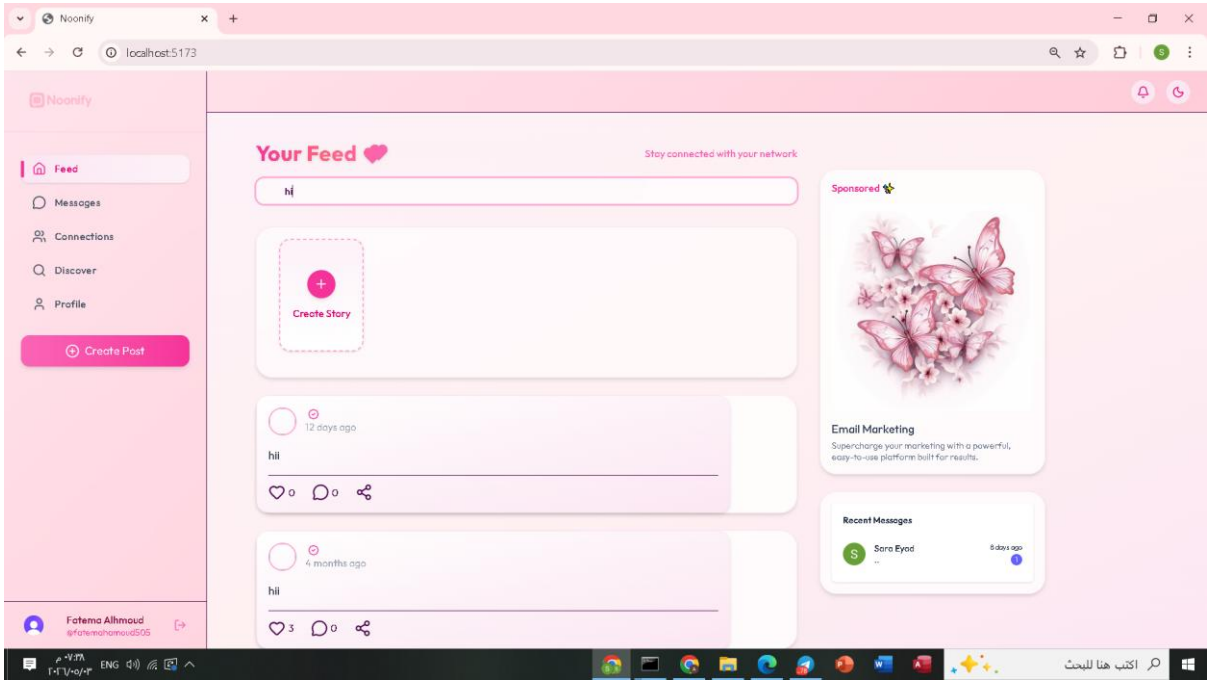


Figure 4.49: Search Hashtags

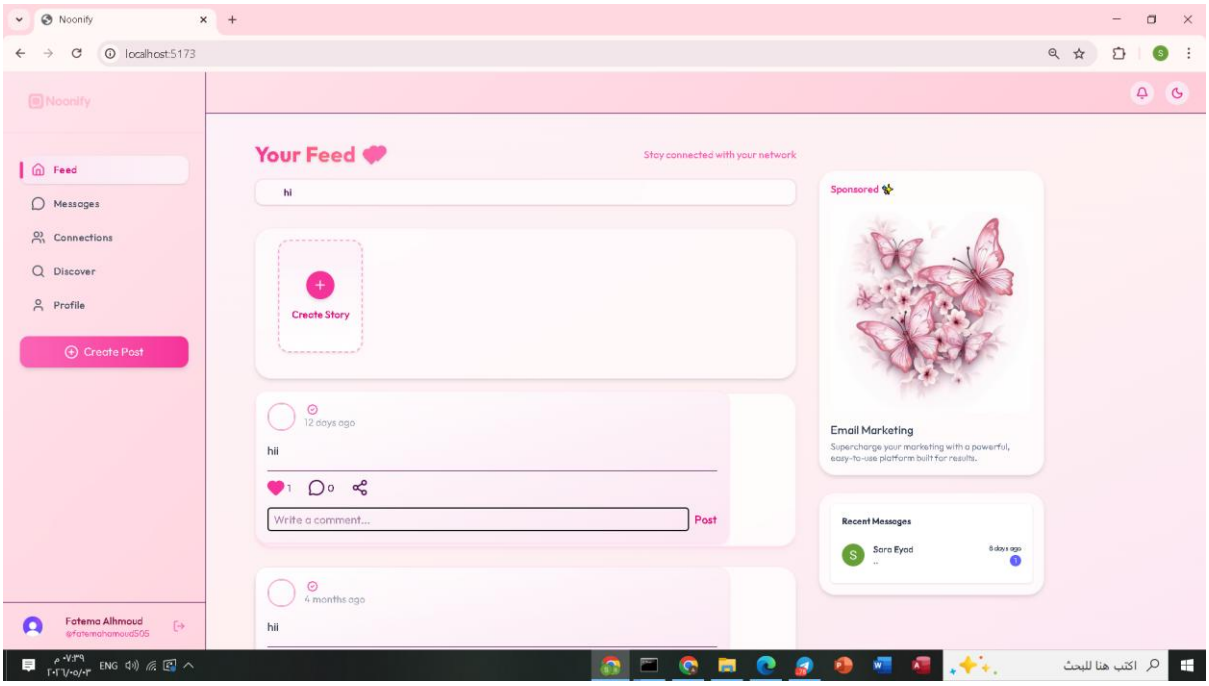


Figure 4.50: Comment / Like / Share

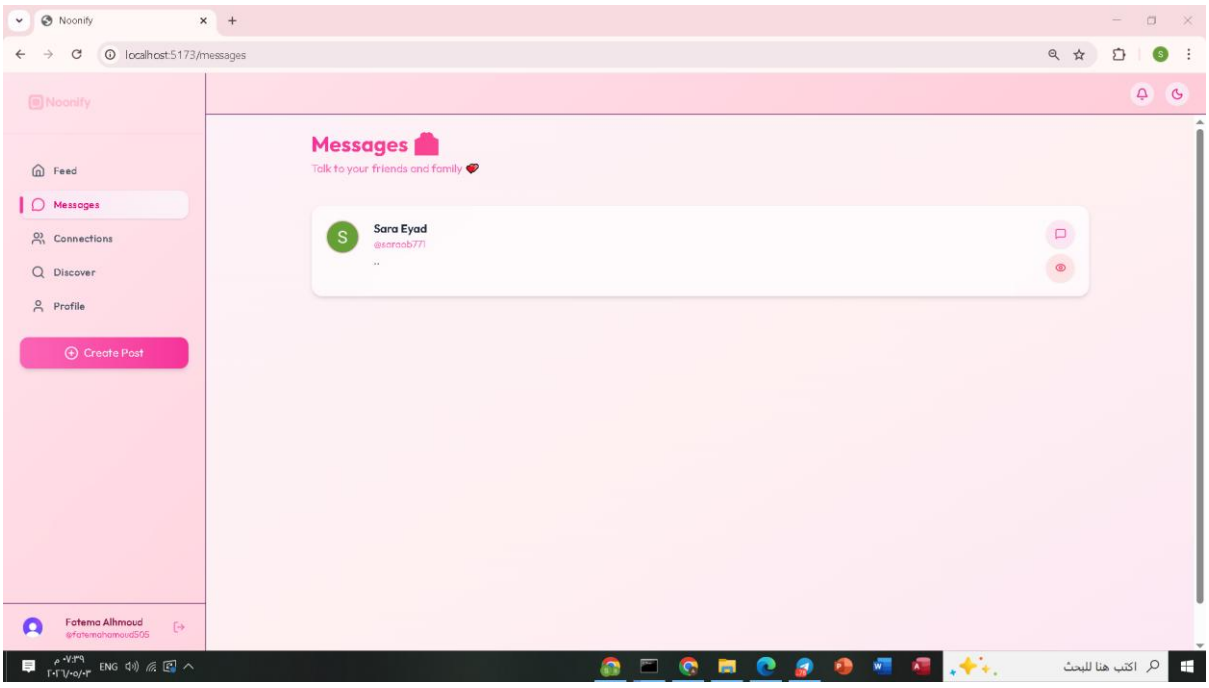


Figure 4.51: Message Page

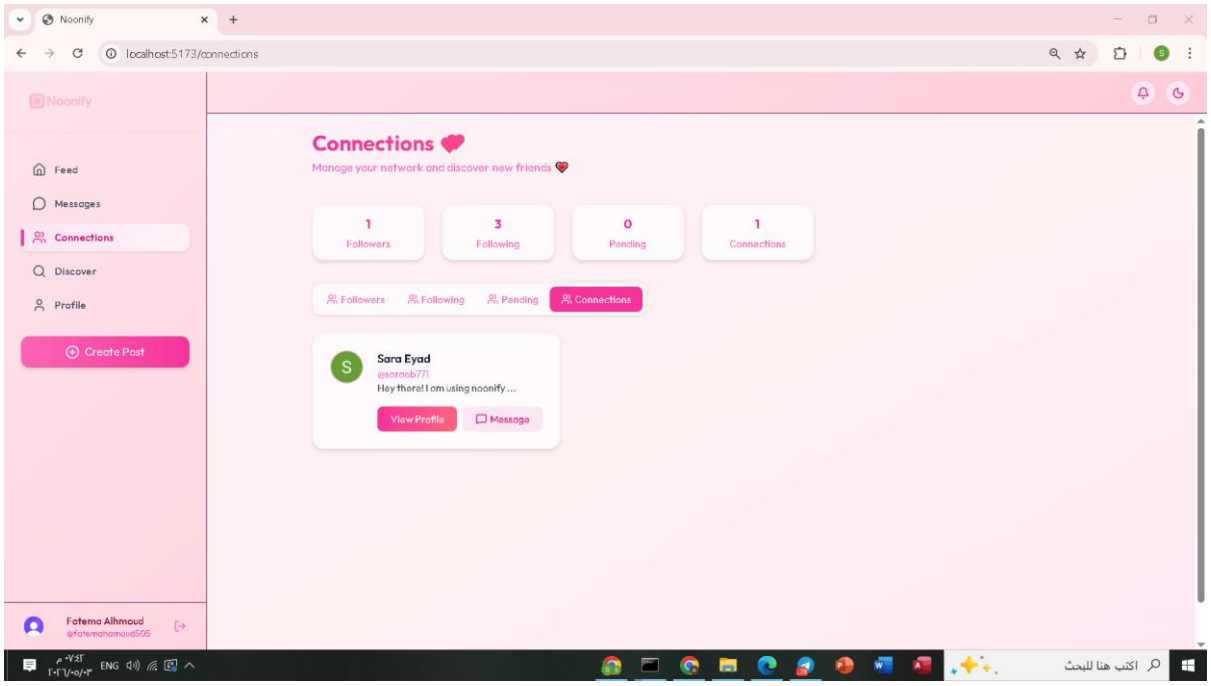


Figure 4.52: Connections Page

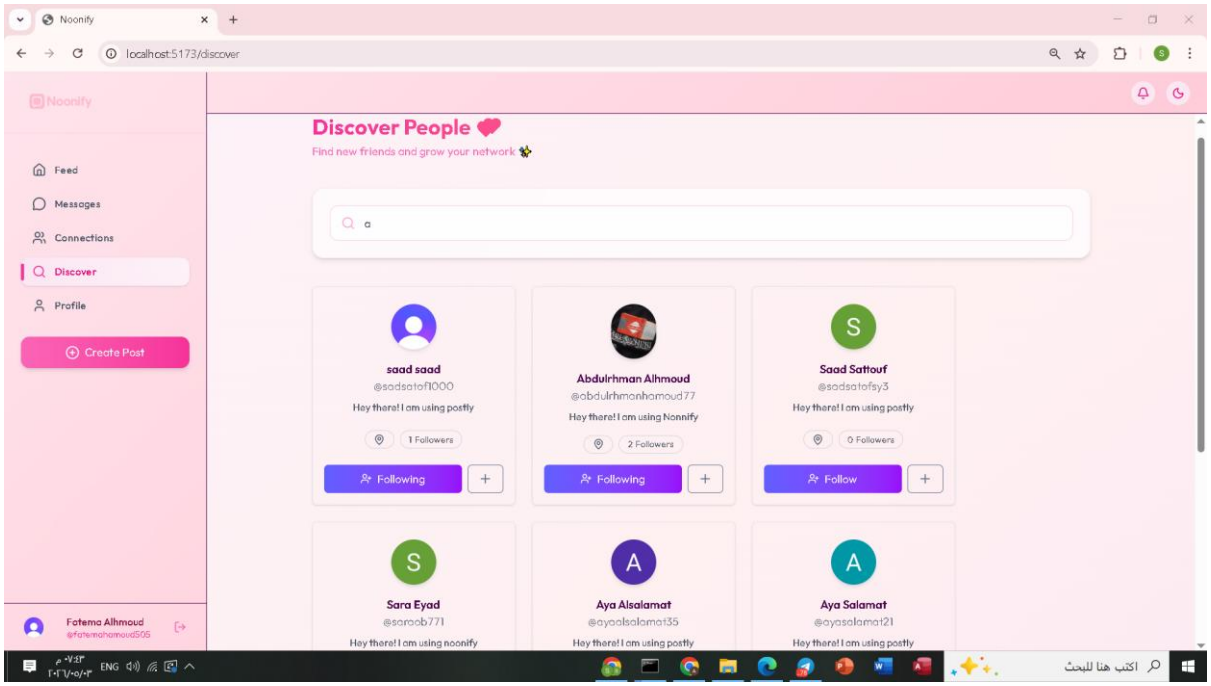


Figure 4.53: Discover People / Friend Suggestion

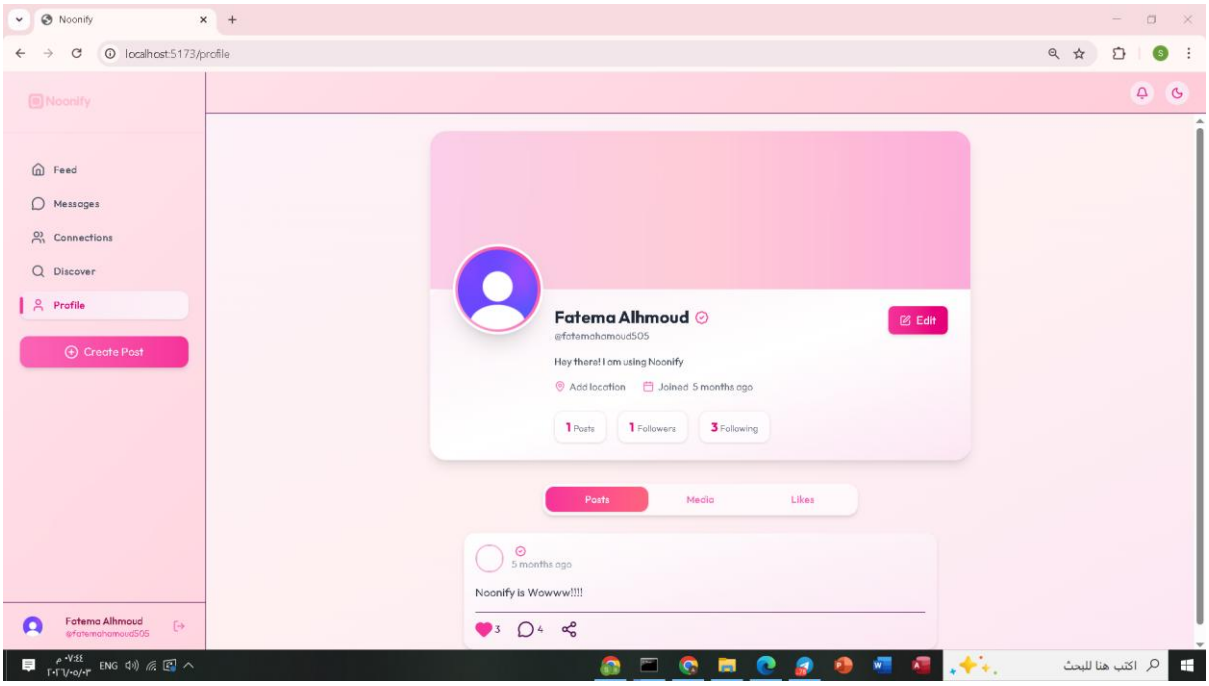


Figure 4.54: Profile Page

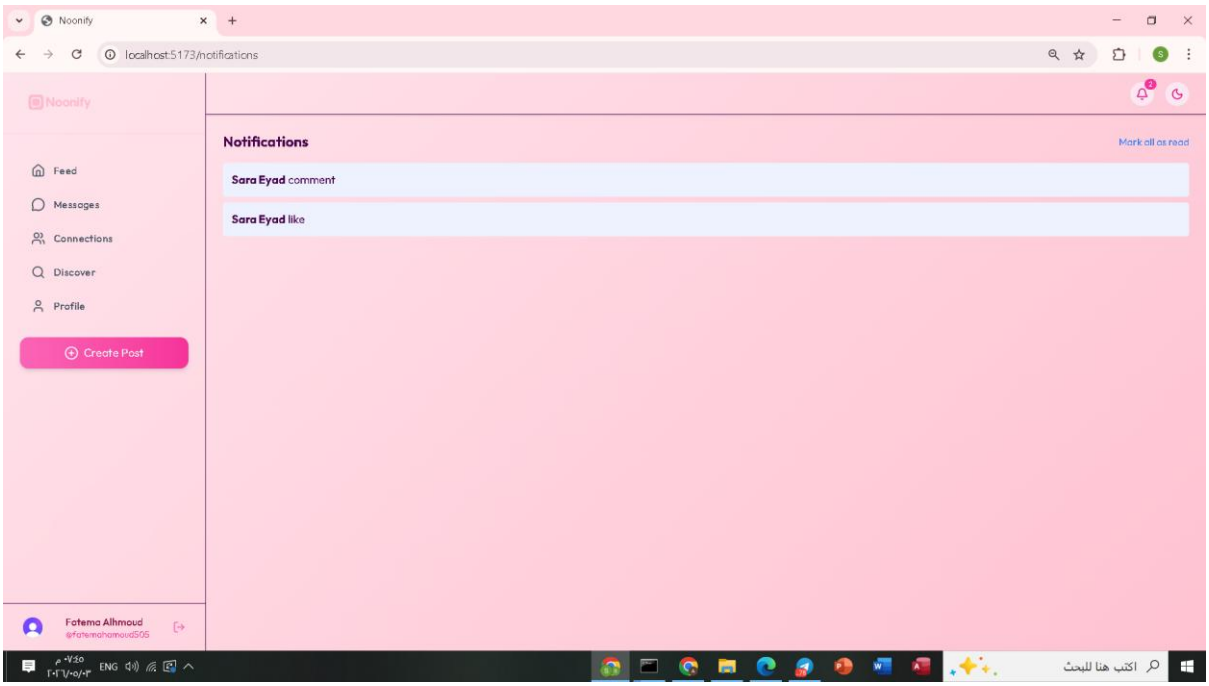


Figure 4.55: Notification Page

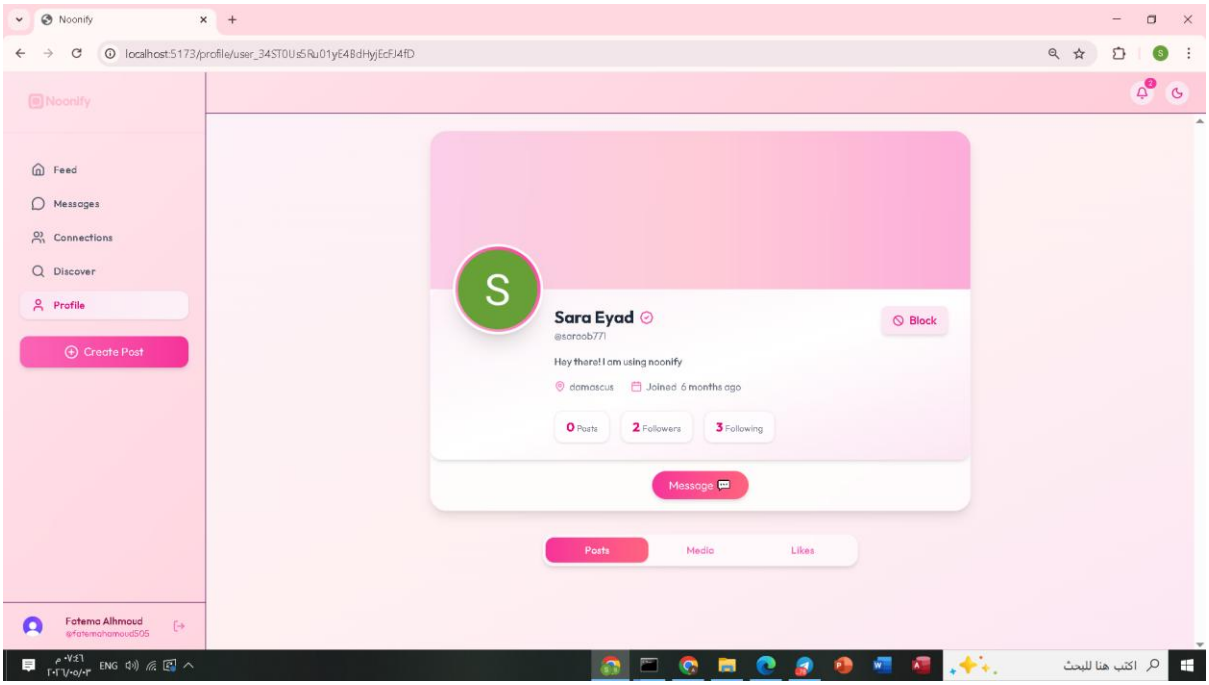


Figure 4.56: User Card Information

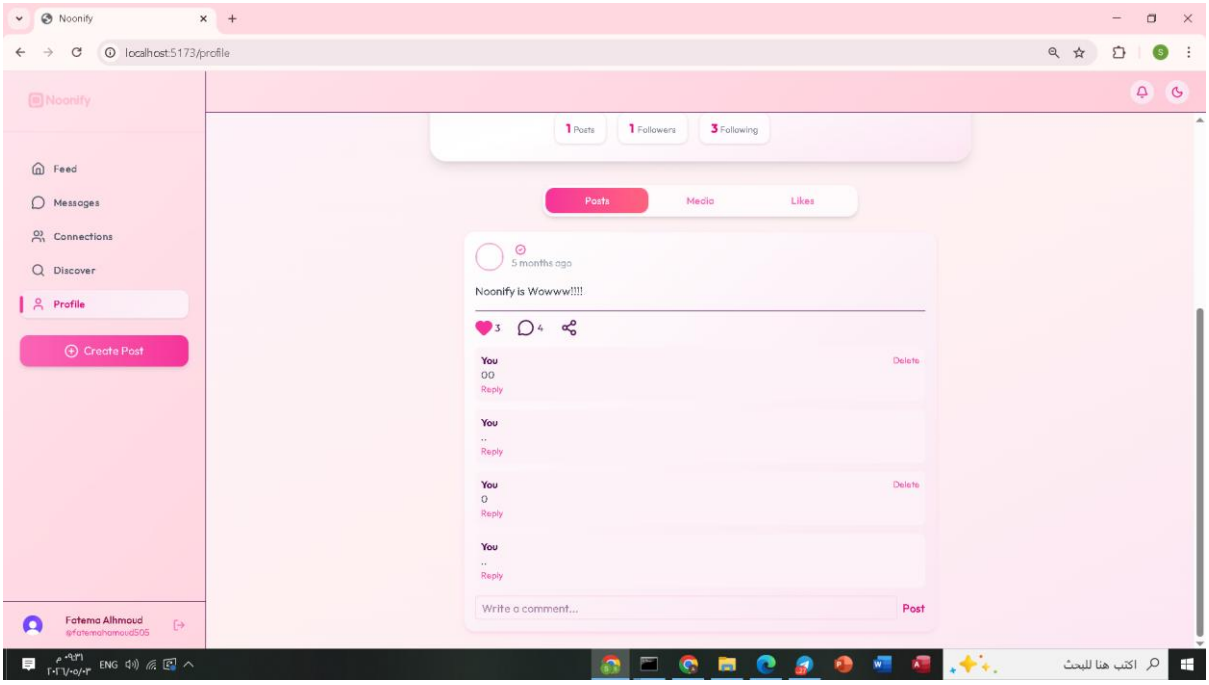


Figure 4.57: Post card Information

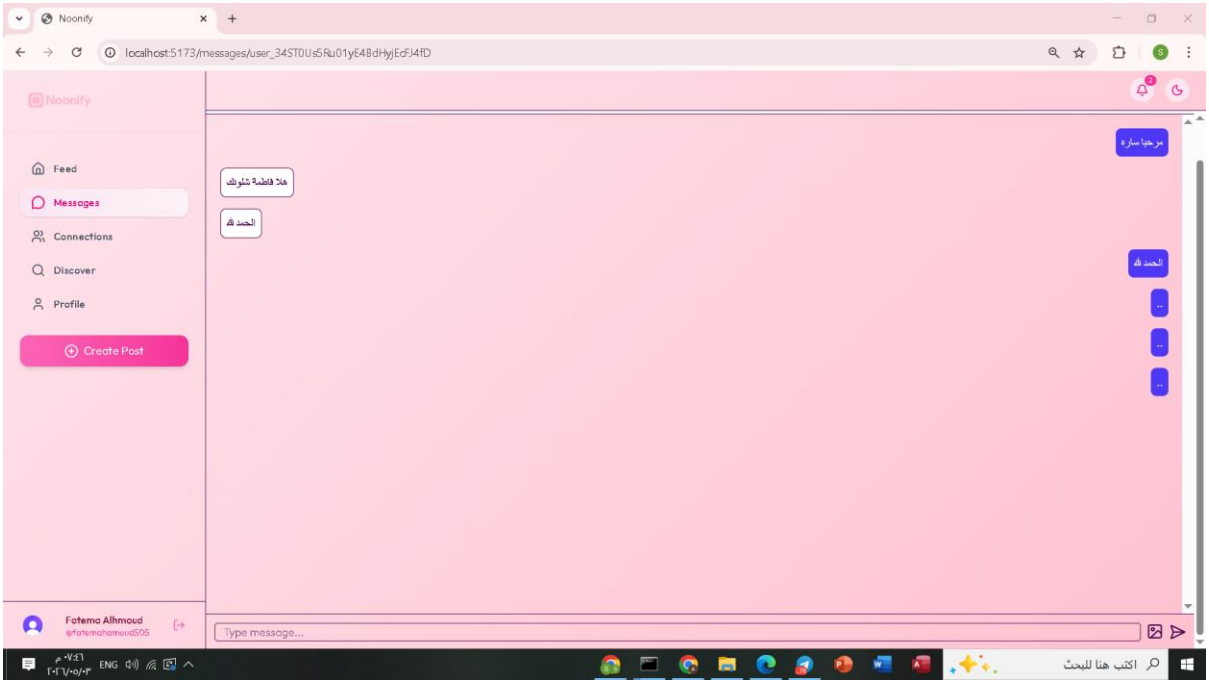


Figure 4.58: Message Page

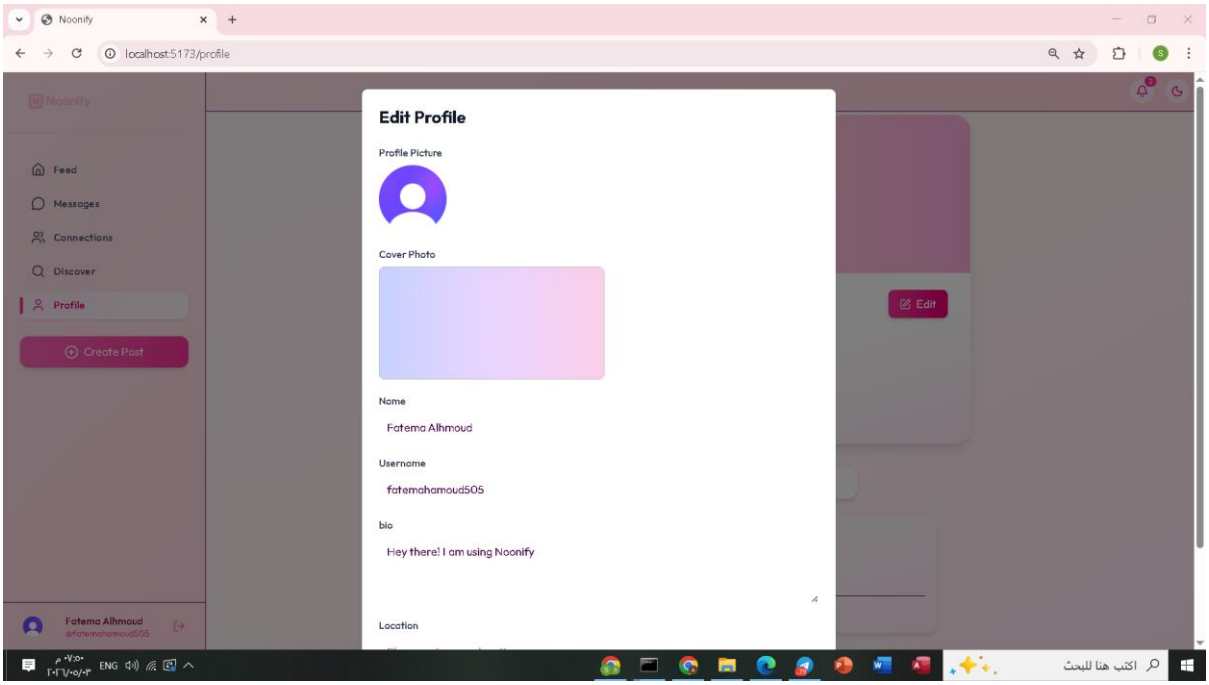


Figure 4.59: Edit Profile

4.2- Conclusion

In conclusion, this project successfully designed and developed a comprehensive social media platform named **NOONIFY**, aiming to replicate the core features of modern social networking systems while addressing both academic and practical aspects. The project was built using the MERN Stack architecture, which integrates an interactive Front-end developed with React.js, a back-end powered by Node.js and Express.js, and a MongoDB database. This combination enabled the creation of a flexible, well-structured, and scalable system.

The project achieved its defined objectives by allowing users to create personal accounts, manage their profiles, share textual and multimedia content, interact with posts, use the stories feature, and communicate through private messaging. Additionally, key architectural concepts such as Client–Server Architecture and MVC Architecture were implemented, contributing to better code organization, improved maintainability, and easier future feature development.

Furthermore, this project enhanced practical skills in modern web application development, working within the Scrum methodology, and requirements analysis. It serves as a practical model that bridges the gap between theoretical knowledge and real-world application in the fields of software engineering and information systems.

4.3- Future Work

Despite achieving its primary goals, there are several potential improvements and extensions that can be implemented in the future to enhance the platform's efficiency and expand its capabilities, including:

- **Enhancing Security**
- Implement advanced security techniques such as Two-Factor Authentication and improved encryption policies to protect user data.
- **Improving Performance and Scalability**

Support caching techniques and utilize advanced cloud services to enable the platform to handle a larger number of users.

- **Integrating Artificial Intelligence**

Incorporate recommendation algorithms to suggest Friends and relevant content based on user interests.

- **Developing Mobile Applications**

Build dedicated applications for Android and iOS to provide a smoother user experience.

- **Expanding Social Interaction Features**

Support live streaming, push notifications, and enhance the commenting and interaction systems.

- **Improving User Experience (UX/UI)**

Conduct real user studies to improve interface design and make it more intuitive and user-Friendly.

References

1. Books and Articles

- Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill Education.
- Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson Education.

2. Online Resources

- MongoDB Inc. (n.d.). *MongoDB Documentation*. Retrieved from https://www.mongodb.com/docs/
- React. (n.d.). *React Official Documentation*. Retrieved from https://react.dev/
- Node.js Foundation. (n.d.). *Node.js Documentation*. Retrieved from https://nodejs.org/en/docs
- Express.js. (n.d.). *Express Framework Documentation*. Retrieved from https://expressjs.com/

3. Tools and Software Documentation

- Vite. (n.d.). *Vite – Next Generation frontend Tooling*. Retrieved from https://vitejs.dev/
- Mongoose. (n.d.). *Mongoose ODM Documentation*. Retrieved from https://mongoosejs.com/docs/
- NodeMailer. (n.d.). *NodeMailer Documentation*. Retrieved from https://nodemailer.com/about/

4. Additional References

- Scrum.org. (n.d.). *Scrum Guide*. Retrieved from https://www.scrumguides.org/

- Mozilla Developer Network (MDN). (n.d.). *Web Technologies Documentation*. Retrieved from https://developer.mozilla.org/
- ImageKit. (n.d.). *ImageKit Media Optimization Documentation*. Retrieved from https://imagekit.io/docs

Appendix: Junior Project Report

The appendix includes supplementary materials related to the Junior project, such as system screenshots, interface designs, diagrams (Use Case, ERD, Class Diagram), test cases, and additional implementation details that support the main content of the report.